

Utah State University

DigitalCommons@USU

All Graduate Theses and Dissertations

Graduate Studies

8-2021

Alternative Doppler Extraction for Indoor Communication Signals

Thomas L. Bradshaw
Utah State University

Follow this and additional works at: <https://digitalcommons.usu.edu/etd>



Part of the [Electrical and Computer Engineering Commons](#)

Recommended Citation

Bradshaw, Thomas L., "Alternative Doppler Extraction for Indoor Communication Signals" (2021). *All Graduate Theses and Dissertations*. 8111.

<https://digitalcommons.usu.edu/etd/8111>

This Thesis is brought to you for free and open access by the Graduate Studies at DigitalCommons@USU. It has been accepted for inclusion in All Graduate Theses and Dissertations by an authorized administrator of DigitalCommons@USU. For more information, please contact digitalcommons@usu.edu.



ALTERNATIVE DOPPLER EXTRACTION FOR INDOOR COMMUNICATION
SIGNALS

by

Thomas L. Bradshaw

A thesis submitted in partial fulfillment
of the requirements for the degree

of

MASTER OF SCIENCE

in

Electrical Engineering

Approved:

Todd Moon, Ph.D.
Major Professor

Jacob Gunther, Ph.D.
Committee Member

Reyhan Baktur, Ph.D.
Committee Member

D. Richard Cutler, Ph.D.
Interim Vice Provost of Graduate Studies

UTAH STATE UNIVERSITY
Logan, Utah

2021

Copyright © Thomas L. Bradshaw 2021

All Rights Reserved

ABSTRACT

Alternative Doppler Extraction for Indoor Communication Signals

by

Thomas L. Bradshaw, Master of Science

Utah State University, 2021

Major Professor: Todd Moon, Ph.D.

Department: Electrical and Computer Engineering

Passive bistatic radar has become a topic of recent research. This radar method uses opportunistic transmitters, such as radio broadcast towers and WiFi routers, as the transmitters in the bistatic radar system. Since opportunistic signals are widespread in society today, new opportunities for using these signals for passive bistatic radar are arising. Recent research has found ways to use these signals to track moving objects. WiFi signals are of particular interest for tracking people in a building for both surveillance and healthcare applications. WiFi signals are useful because most buildings have WiFi routers, and these signals are able to penetrate walls.

The common method of tracking a moving person using WiFi radar is the Doppler only method. This method extracts the Doppler frequencies from the Doppler-shifted echo received at a receiver. A Kalman filter or a particle filter can use the Doppler frequencies extracted at multiple receivers to track the moving person. Typically, the Doppler frequencies are extracted using the cross-ambiguity function (CAF). This function requires a reference signal without the Doppler echo in it, which means an extra antenna is needed to obtain this signal. This extra antenna requires more hardware and is not always feasible in application.

This thesis provides an alternative method to extract the Doppler frequencies on a communication signal. This alternative method removes the need for a reference antenna by using knowledge of the communication system and additional signal processing in order to separate the Doppler echo signal from the direct path signal. The method explored relies on the ability to remove the communication signal symbols from the received signal and accurately estimate the carrier frequency offset on the communication signal.

GNU Radio was used for implementing and testing this system. The system was tested with various devices and antenna configurations. Algorithms, such as the MUSIC algorithm, Viterbi algorithm, and BCJR algorithm, were applied to the signal in MATLAB in order to further improve the Doppler frequency extraction. In addition, various new techniques were developed to overcome limitations on the system.

(216 pages)

PUBLIC ABSTRACT

Alternative Doppler Extraction for Indoor Communication Signals

Thomas L. Bradshaw

Radar is a common detection system for detecting the speed and velocity of moving objects. Radar is performed by transmitting a radio signal in the direction of a moving object and then processing the received reflected radio waves. The radar system processes the received signal to extract the Doppler frequency of the reflected waves, which reveals information about the velocity of the object. In traditional radar, the system uses one antenna for both the transmitter and the receiver.

In passive bistatic radar, the transmit antenna and receiver antenna are separated. In addition, the radar system has no control over the transmitter. The transmitted signal is instead a signal of opportunity. Signals of opportunity include FM and AM radio signals, satellite GPS signals, and WiFi signals. The receiver in the radar system receives a signal of opportunity and extracts the Doppler frequency from the signal. The Doppler frequency can then be used to get the velocity of the object, which can be used for tracking the object.

In previous research, the methods of traditional radar were applied to passive bistatic radar using a WiFi signal in order to track a moving object within a building. Traditional radar methods are to compare the Doppler-shifted signal to the original signal. In passive radar, the original signal needs to be measured without being affected by the Doppler-shifted signal, which requires a second antenna to be placed by the transmitter.

The research presented suggests a new method of retrieving the Doppler frequencies from a communication signal. This new method does not require a separate antenna to get the original signal but uses knowledge about the communication signal to successfully measure the Doppler frequency on the signal. The system was implemented using GNU Radio, a software-defined radio library. Additional processing was done using MATLAB.

Dedicated to my wife and best friend, Kelsey.

ACKNOWLEDGMENTS

This thesis would not have come about without the support of many different people in my life. I am grateful for the education I have received because of them.

First, I would like to thank my wife, Kelsey, who comforted me through my many late nights of studying and supported me in my many endeavors over the past three years.

I would also like to thank my parents, who helped me love to learn and encouraged me to push beyond my comfort zone in order to accomplish more than I ever thought myself capable of.

I give a special thanks to my professor, Dr. Todd Moon, for sharing his knowledge with me. The countless hours he has spent teaching and mentoring me have shaped me into the engineer I am today.

I also thank my committee, Dr. Jacob Gunther and Dr. Reyhan Baktur, whose advice helped me push forward in my research.

I would like to give thanks to the countless friends and co-workers who also supported me through my research: Joseph Ipson, Mirelle DeSpain, Cayden Anderson, and Colton Lindstrom.

Finally, I would like to thank the Laboratory for Telecommunication Sciences for funding our research and helping me troubleshoot the many hardware problems I faced.

Thomas L. Bradshaw

CONTENTS

	Page
ABSTRACT	iii
PUBLIC ABSTRACT	v
ACKNOWLEDGMENTS	vii
LIST OF TABLES	x
LIST OF FIGURES	xii
ACRONYMS	xvi
1 INTRODUCTION	1
1.1 Literature Review	2
1.1.1 Passive Bistatic Radar	2
1.1.2 FM-Based Passive Radar	2
1.1.3 WiFi-Based Passive Radar	3
1.2 Background	4
1.2.1 Introduction to Digital Communications	4
1.2.2 Bistatic Radar	6
1.3 Chapter Outlines	9
2 COMMUNICATION SYSTEM	11
2.1 Theory	11
2.1.1 Second Doppler Extraction Method	16
2.2 Hardware Description	19
2.2.1 Transmitters and Receivers	20
2.2.2 Antennas	21
2.2.3 Other Hardware	22
2.3 GNU Radio QPSK System	23
2.3.1 Transmitter	23
2.3.2 Receiver	26
2.3.3 Hardware Constraints	29
2.4 Doppler Extraction	30
3 SPECTRAL ESTIMATION	32
3.1 Discrete Fourier Transform	32
3.2 Multiple Signal Classification	36
3.3 Carrier Frequency Offset Removal	40
3.4 DC Component Removal	42

4	PATH EXTRACTION	45
4.1	Viterbi Algorithm	45
4.2	BCJR Algorithm	49
4.3	Balance Factor	52
4.4	Correcting Symmetry	57
5	RESULTS	66
5.1	Simulation Results	66
5.2	Implementation Results	68
5.2.1	June 26, 2020	70
5.2.2	July 13, 2020	76
5.2.3	July 15, 2020	82
5.2.4	July 21, 2020	86
5.2.5	August 10, 2020	90
5.2.6	Simulated Doppler Paths	93
5.2.7	October 2, 2020	98
5.2.8	October 9, 2020	102
5.2.9	November 6, 2020	108
5.2.10	November 13, 2020	114
5.2.11	February 12, 2021	124
5.3	Discussion	134
5.3.1	Communication System Setup	134
5.3.2	Carrier Frequency Offset Removal Tuning	135
5.3.3	Spectrum Estimation Parameter Tuning	135
5.3.4	Doppler Path Extraction Parameter Tuning	136
6	CONCLUSION	138
	REFERENCES	141
	APPENDICES	143
A	Test Parameters	144
B	Derivation of Path Derivative	153
C	Code Listings	156
C.1	Overhead Files	157
C.2	Spectrum Estimation Functions	178
C.3	Viterbi Functions	183
C.4	BCJR Functions	193

LIST OF TABLES

Table	Page
2.1 Transmitter and receiver devices.	20
2.2 Antenna parameters.	21
5.1 Device Parameters, June 26, 2020	70
5.2 Symbol Timing Synchronization Parameters, June 26, 2020	71
5.3 Carrier phase synchronization parameters, June 26, 2020	71
5.4 Tests, June 26, 2020	71
5.5 Device Parameters, July 13, 2020	76
5.6 Symbol Timing Synchronization Parameters, July 13, 2020	76
5.7 Carrier phase synchronization parameters, July 13, 2020	76
5.8 Tests, July 13, 2020	77
5.9 Device Parameters, July 15, 2020	82
5.10 Symbol Timing Synchronization Parameters, July 15, 2020	82
5.11 Carrier phase synchronization parameters, July 15, 2020	83
5.12 Tests, July 15, 2020	83
5.13 Device Parameters, July 21, 2020	86
5.14 Symbol Timing Synchronization Parameters, July 21, 2020	86
5.15 Carrier phase synchronization parameters, July 21, 2020	87
5.16 Tests, July 21, 2020	87
5.17 Device Parameters, August 10, 2020	90
5.18 Symbol Timing Synchronization Parameters, August 10, 2020	90
5.19 Carrier phase synchronization parameters, August 10, 2020	90

5.20 Tests, August 10, 2020	93
5.21 Device Parameters, October 2, 2020	98
5.22 Symbol Timing Synchronization Parameters, October 2, 2020	98
5.23 Carrier phase synchronization parameters, October 2, 2020	99
5.24 Tests, October 2, 2020	99
5.25 Device Parameters, October 9, 2020	102
5.26 Symbol Timing Synchronization Parameters, October 9, 2020	102
5.27 Carrier phase synchronization parameters, October 9, 2020	103
5.28 Tests, October 9, 2020	103
5.29 Device Parameters, November 6, 2020	108
5.30 Symbol Timing Synchronization Parameters, November 6, 2020	108
5.31 Carrier phase synchronization parameters, November 6, 2020	109
5.32 Tests, November 6, 2020	109
5.33 Device Parameters, November 13, 2020	114
5.34 Symbol Timing Synchronization Parameters, November 13, 2020	114
5.35 Carrier phase synchronization parameters, November 13, 2020	115
5.36 Tests, November 13, 2020	115
5.37 Device Parameters, February 12, 2021	124
5.38 Symbol Timing Synchronization Parameters, February 12, 2021	125
5.39 Carrier phase synchronization parameters, February 12, 2021	125
5.40 Tests, February 12, 2021	125

LIST OF FIGURES

Figure	Page
1.1 QPSK constellation.	5
1.2 Diagram of the direct and Doppler echo paths of a transmitted communication signal.	6
1.3 Complete system high-level overview	10
2.1 Receiver high level overview.	12
2.2 Doppler rotations around symbols in the signal space.	16
2.3 Doppler rotations around symbols in the signal space with a rotating symbol constellation.	17
2.4 Flowchart of the carrier phase PLL.	18
2.5 Hardware overview.	19
2.6 Building antenna positions.	22
2.7 Transmitter system high level overview.	24
2.8 Transmitter system GNU Radio flowgraph.	24
2.9 Constellation Modulator properties.	25
2.10 Constellation Object properties.	25
2.11 UHD:USRP Sink general properties.	26
2.12 UHD:USRP Sink RF properties.	26
2.13 The first part of the receiver system.	27
2.14 QPSK receiver high level overview.	28
2.15 Root Raised Cosine Filter properties.	29
2.16 Symbol Sync properties.	29

2.17	Interpolation and decimation blocks in the transmitter and receiver GNU Radio flowcharts.	30
2.18	First DEM implemented in GNU Radio.	31
2.19	Second DEM implemented in GNU Radio.	31
3.1	Example of a spectrogram.	33
3.2	Attempts to estimate a signal at 13 Hz with only 10 Hz of resolution. . . .	35
3.3	Spectral leakage may result in the loss of the ability to differentiate signals. .	37
3.4	Comparison of spectrograms generated by the DFT or MUSIC algorithm. .	39
3.5	The overwhelming strength of the carrier frequency offset component covers the Doppler frequency.	41
3.6	The Doppler frequency causing changes to the main lobe of the carrier frequency offset path.	43
4.1	Maximum value path extracted from a MUSIC-generated spectrogram. . . .	46
4.2	Trellis diagram of a Viterbi algorithm.	47
4.3	Path extraction using the Viterbi algorithm through a MUSIC generated spectrogram.	50
4.4	Path extraction using a Viterbi algorithm on a BCJR probability matrix formed using a MUSIC generated spectrogram.	53
4.5	A scaled balance factor vector plotted on top of the MUSIC spectrum the balance factor represents.	55
4.6	Adjustment frequency spectrum used to pull the Doppler frequency closer to 0 Hz.	56
4.7	Trying to apply the BCJR algorithm to a spectrogram without the balance factor adjustment when there are portions of the signal at 0 Hz.	58
4.8	Trying to apply the BCJR algorithm to a spectrogram with the balance factor adjustment when there are portions of the signal at 0 Hz.	59
4.9	Diagram of how Doppler echo symbols rotate on a non-rotating signal constellation.	60
4.10	Comparison of $r - R$ and θ for both the positive and negative Doppler cases. .	61

4.11	Diagram of how Doppler frequencies rotate on a rotating signal constellation.	62
4.12	Comparison of $r - R$ and θ with a carrier phase offset, for both the positive and negative Doppler cases.	63
4.13	A symmetric MUSIC spectrum of the Doppler echo signal.	64
4.14	Result of applying symmetry correction.	65
5.1	GNU Radio simulated channel with a simulated Doppler echo.	66
5.2	Simulation of the double frequency issue caused by the carrier phase PLL. .	67
5.3	Test 1 extracted path over a DFT spectrogram.	73
5.4	Test 5 extracted path over a DFT spectrogram.	74
5.5	Test 6 extracted path over a DFT spectrogram.	75
5.6	Test 1 extracted path over a DFT spectrogram.	78
5.7	Test 3 extracted path over a DFT spectrogram.	79
5.8	Test 1 extracted path over a MUSIC spectrogram.	80
5.9	Test 3 extracted path over a MUSIC spectrogram.	81
5.10	Test 3 using the First DEM, with the optimal path plotted on the MUSIC spectrum.	84
5.11	Test 3 using the Second DEM, with the optimal path plotted on the MUSIC spectrum.	85
5.12	Test 1 with optimal path plotted on MUSIC spectrum.	88
5.13	Test 3 with optimal path plotted on MUSIC spectrum.	89
5.14	Test 1 with the optimal path plotted on MUSIC spectrum.	91
5.15	Test 2 with the optimal path plotted on MUSIC spectrum.	92
5.16	Doppler frequency simulated at the SC antenna.	94
5.17	Doppler frequency simulated at the SW antenna.	95
5.18	Doppler frequency simulated at the NC antenna.	96
5.19	Doppler frequency simulated at the NW antenna.	97

5.20	Test 1 with optimal path plotted on MUSIC spectrum, SW antenna.	100
5.21	Test 1 with optimal path plotted on MUSIC spectrum, NW antenna.	101
5.22	Test 1 with optimal path plotted on MUSIC spectrum, SC antenna.	104
5.23	Test 1 with optimal path plotted on MUSIC spectrum, SW antenna.	105
5.24	Test 1 with optimal path plotted on MUSIC spectrum, NC antenna.	106
5.25	Test 1 with optimal path plotted on MUSIC spectrum, NW antenna.	107
5.26	Test 1 with optimal path plotted on MUSIC spectrum, SC antenna.	110
5.27	Test 1 with optimal path plotted on MUSIC spectrum, SW antenna.	111
5.28	Test 1 with optimal path plotted on MUSIC spectrum, NC antenna.	112
5.29	Test 1 with optimal path plotted on MUSIC spectrum, NW antenna.	113
5.30	Test 1 with the optimal path plotted on a MUSIC spectrum, SC antenna. .	116
5.31	Test 2 with the optimal path plotted on a MUSIC spectrum, SC antenna. .	117
5.32	Test 1 with the optimal path plotted on a MUSIC spectrum, SW antenna. .	118
5.33	Test 2 with the optimal path plotted on a MUSIC spectrum, SW antenna. .	119
5.34	Test 1 with the optimal path plotted on a MUSIC spectrum, NC antenna. .	120
5.35	Test 2 with the optimal path plotted on a MUSIC spectrum, NC antenna. .	121
5.36	Test 1 with the optimal path plotted on a MUSIC spectrum, NW antenna. .	122
5.37	Test 2 with the optimal path plotted on a MUSIC spectrum, NW antenna. .	123
5.38	Test 0 with the optimal path plotted on a MUSIC spectrum, SC antenna. .	126
5.39	Test 4 with the optimal path plotted on a MUSIC spectrum, SC antenna. .	127
5.40	Test 0 with the optimal path plotted on a MUSIC spectrum, SW antenna. .	128
5.41	Test 4 with the optimal path plotted on a MUSIC spectrum, SW antenna. .	129
5.42	Test 0 with the optimal path plotted on a MUSIC spectrum, NC antenna. .	130
5.43	Test 4 with the optimal path plotted on a MUSIC spectrum, NC antenna. .	131
5.44	Test 0 with the optimal path plotted on a MUSIC spectrum, NW antenna. .	132
5.45	Test 4 with the optimal path plotted on a MUSIC spectrum, NW antenna. .	133
6.1	Initial target tracking test using Doppler extracted data from the February 12, 2021 tests.	140
B.1	Diagram of a changing path length between two moving objects.	153

ACRONYMS

ADC	analog-to-digital converter
ARD	amplitude-range-Doppler
BCJR	Bahl, Cocke, Jelinek, and Raviv (developers of the algorithm)
BF	Balance Factor
CAF	cross-ambiguity function
DDS	direct digital synthesizer
DEM	Doppler extraction method
DFT	discrete Fourier transform
DTFT	discrete-time Fourier transform
FFT	fast Fourier transform
FM	frequency modulation
IQ	in-phase and quadrature
ISM	industrial, scientific, and medical
LUT	look-up table
MUSIC	multiple signal classification
PLL	phase-locked loop
QPSK	quadrature phase shift keying
RF	radio frequency
SIR	signal-to-interference ratio
SNR	signal-to-noise ratio
TED	timing error detector

CHAPTER 1

INTRODUCTION

Passive bistatic radar has become a topic of research interest in recent years. This method of radar relies on using opportunistic transmitters, such as radio broadcast towers and WiFi routers, as the transmitter in the bistatic radar system. These systems are advantageous over traditional bistatic radar systems as they reduce the hardware cost and can be used discreetly [1]. One particular area of research focuses on using WiFi signals to track human movement in a building. WiFi signals are of interest because of the widespread use of WiFi routers in both public and private buildings and the ability of the signals to penetrate walls [2].

In order to track human movement in a building, one method relies on collecting the Doppler echo signal at multiple receivers and then extracting the Doppler frequency path from this signal. This method collects the Doppler echo at each of the receivers and then inputs the Doppler frequency path into either a Kalman filter or a particle filter [3]. The Doppler frequency path is extracted through a matched filter in a cross-ambiguity function (CAF) [4]. The issue with this extraction method is the reliance on a reference signal in the matched filtering, which is needed to eliminate the direct path signal as it can easily overpower the Doppler echo. This reference signal is obtained by placing a reference antenna by the transmitter or using a narrow beam antenna pointed at the transmitter [5].

This thesis seeks to explore a new method of extracting the Doppler signal at each antenna without the need for a reference signal. This new method relies on demodulating the communication signal and removing the symbols in order to separate out the Doppler signal. This thesis describes the communication system built to test this new Doppler extraction method and additional processing for overcoming obstacles to the Doppler extraction.

1.1 Literature Review

1.1.1 Passive Bistatic Radar

Bistatic radar is a form of radar where the transmitter and receiver use separate antennas. This form of radar has been used since World War II, as recorded by Griffiths [1]. The particular advantage of separating the transmitting and receiving antennas is the greater protection provided to the system. Griffiths discusses the appeal of passive bistatic radar, where the transmitted signal is a signal of opportunity such as a communication signal (e.g. FM radio, WiFi). It is notable that at the time of Griffiths' article there were few passive radar systems implemented outside of research, as there were few situations where passive radar would work better than conventional radar.

Since the publication of Griffiths' paper in 2011, new applications for passive bistatic radar have arisen. FM-based passive radar has found new applications in tracking airborne targets, such as airplanes and drones [6–9]. WiFi-based passive radar has found applications in healthcare situations [4, 10] and surveillance [11].

1.1.2 FM-Based Passive Radar

Howland et al. looked at the feasibility of using FM signals over analog television signals in order to track aircraft. The method of obtaining Doppler information in a traditional radar system involves using the original signal as a matched filter for the Doppler echo signal to produce a range-Doppler matrix [6–8]. The waveform of analog television signals is less ideal for the matched filtering compared to the waveform of FM signals [6, pp. 1].

In passive bistatic radar, the original signal is replaced with an estimate of the original signal known as the reference signal. In order to separate the reference and echo signals, a few methods are employed. The reference signal is collected either by a dedicated receiver [8] or a receiver array with digital beamforming [6, 7]. The echo signal is obtained through an adaptive filter using the reference signal and a received signal at the antenna of interest [8]. Once the reference and echo signals are obtained, then the range-Doppler information is

found using the amplitude-range-Doppler (ARD) surface calculated by [6, pp. 2]

$$|\Psi(\tau, f_d)| = \left| \sum_{n=0}^{N-1} s_{err}(n) s_{ref}^*(n - \tau) e^{j2\pi f_d n/N} \right|. \quad (1.1)$$

In this equation, s_{err} is the echo signal and s_{ref}^* is the conjugate of the reference signal. N is the number of bins on the frequency axis.

1.1.3 WiFi-Based Passive Radar

WiFi-based passive radar has gained recent interest because of the ability of WiFi signals to penetrate walls, which is useful for surveillance applications. A limiting factor in using WiFi is the overpowering strength of the direct path signal. The performance of the system is measured by the signal-to-interference ratio (SIR) rather than the signal-to-noise ratio (SNR) because of the direct path interference [2, pp. 1]. The CAF function computed by

$$CAF(\tau_d, f_d) = \sum_{n=0}^N r[n] s^*[n + \tau] e^{-j2\pi f_d n/N} \quad (1.2)$$

extracts Doppler information from a WiFi signal [12, pp. 2]. This equation is essentially the same as the ARD in (1.1), with $r[n]$ acting as the reference signal and $s[n]$ acting as the echo signal. The WiFi-based approach uses a similar method to the FM-based approach for measuring both of these signals. Typically, the reference signal is obtained by placing an antenna directly adjacent to the transmitter [3].

The CAF is adopted as the typical method in the literature for obtaining the range and Doppler information. Once the range and Doppler information is found, the Doppler information is used in an extended Kalman filter or a particle filter to track a moving person. Tan, Chetty, and Woodbridge used this method to track a person moving in a room [3] and to identify small human motions and hand gestures [4, 12]. Their work has been used and improved upon for various applications [10, 13, 14]. However, the method remains largely the same throughout the literature.

1.2 Background

The objective of the research was to develop a new method to extract the Doppler frequencies from a communication signal for the purpose of tracking a moving human in a building. This section reviews the background theory of the research.

1.2.1 Introduction to Digital Communications

Digital communication theory is described in detail in Dr. Rice's book [15]. This section gives a brief overview of signal space theory as it relates to digital communication theory. Chapter 2 goes into more of the digital communication theory as it relates to the Doppler signal extraction.

In digital communication theory, the communication signal is described using the concept of the signal space. The signal space is a vector space spanned by orthogonal waveforms. An example of a signal space is shown in Figure 1.1, which was the signal space used in this research. This specific signal space uses the orthogonal waveforms [15, pp. 223]

$$\begin{aligned}\psi_0(t) &= \sqrt{2}p(t)\cos(2\pi f_c t) \\ \psi_1(t) &= \sqrt{2}p(t)\sin(2\pi f_c t).\end{aligned}\tag{1.3}$$

In these equations, $p(t)$ is any unit energy pulse and f_c is the carrier frequency of the transmitted signal. The $p(t)$ chosen for this research was the square-root raised cosine pulse [15, pp. 637].

The points in the signal space each represent a number of bits and are known as symbols. The symbols in the signal space are collectively known as the signal constellation. For the signal space in Figure 1.1, each symbol represents two bits as shown. This constellation is known as the quadrature phase shift keying (QPSK) constellation.

The transmitted signal is formed by sending bits through the transmission system (see Section 2.3.1). In this system, the bits are mapped to symbols and then modulated by the waveforms of the signal space. For the QPSK constellation used in the research, each pair of bits is mapped to a constellation point denoted as (a_0, a_1) . The signal created by taking

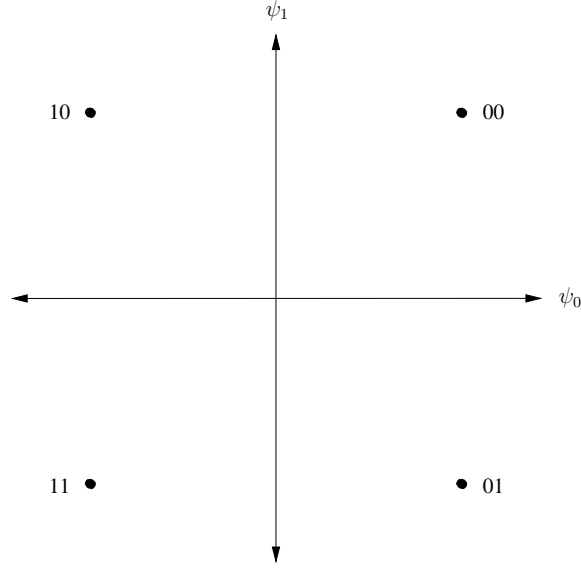


Fig. 1.1: QPSK constellation.

a series of these symbols is

$$s_m(t) = \sqrt{2} \left(\sum_k a_0(k) p(t - kT_s) \cos(2\pi f_c t) - \sum_k a_1(k) p(t - kT_s) \sin(2\pi f_c t) \right), \quad (1.4)$$

where k indexes the symbols. (1.4) can be written in a more convenient form of

$$s_m(t) = \sqrt{2} (I \cos(2\pi f_c t) - Q \sin(2\pi f_c t)). \quad (1.5)$$

In this equation, I is referred to as the in-phase portion of the signal and Q is referred to as the quadrature portion of the signal. Together, they are known as the in-phase and quadrature (IQ) signal.

The IQ signal can be represented as $I + jQ$, which in polar form is

$$s_b(t) = A(t) e^{j\phi(t)}, \quad (1.6)$$

where $I = A(t)\cos(\phi(t))$ and $Q = A(t)\sin(\phi(t))$. This form is convenient for representing the transmitted signal. Using this notation, (1.5) can be written as

$$s_m(t) = A(t)e^{j\phi(t)}e^{j2\pi f_c t} = A(t)e^{j(2\pi f_c t + \phi(t))}. \quad (1.7)$$

This is the transmitted signal in the bistatic radar system.

1.2.2 Bistatic Radar

Figure 1.2 shows a simple model for bistatic radar. There is a fixed transmitter at position $\mathbf{t} = (x_T, y_T)$, which transmits the modulated communication signal of (1.7). Let $\mathbf{q} = (x_R, y_R)$ be the position of a fixed receiver, $\mathbf{g}(t) = (x(t), y(t))$ be the position of a moving object at time t , and $\mathbf{v}(t) = (v_x(t), v_y(t))$ be the velocity vector for the moving object at time t .

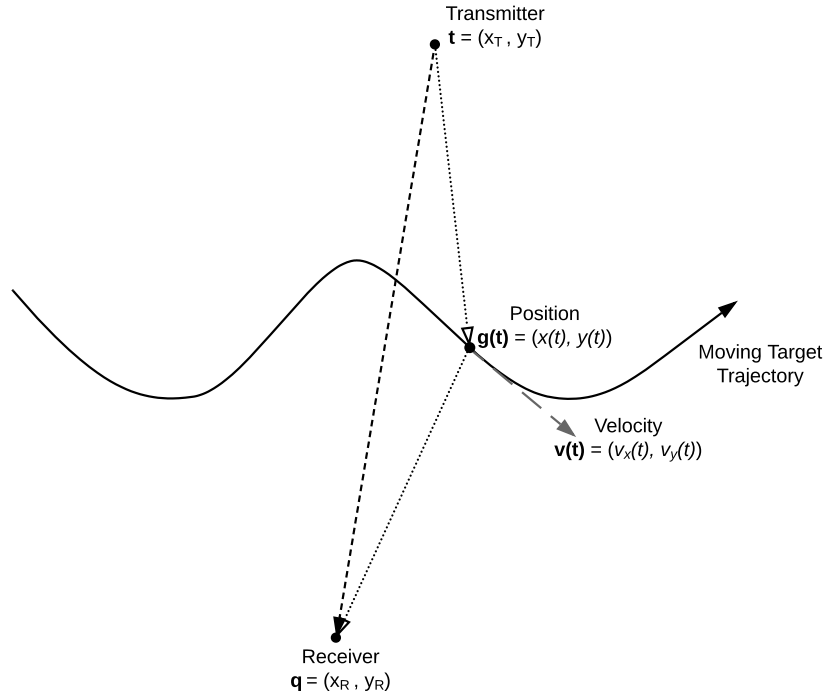


Fig. 1.2: Diagram of the direct and Doppler echo paths of a transmitted communication signal.

Now define $\hat{\mathbf{u}}_{\mathbf{t}}(t)$ as the unit vector pointing from $\mathbf{t}$ to $\mathbf{g}(t)$,

$$\hat{\mathbf{u}}_{\mathbf{t}}(t) = \frac{\mathbf{g}(t) - \mathbf{t}}{\|\mathbf{g}(t) - \mathbf{t}\|} \quad (1.8)$$

and define $\hat{\mathbf{u}}_{\mathbf{q}}(t)$ as the unit vector pointing from $\mathbf{q}$ to $\mathbf{g}(t)$,

$$\hat{\mathbf{u}}_{\mathbf{q}}(t) = \frac{\mathbf{g}(t) - \mathbf{q}}{\|\mathbf{g}(t) - \mathbf{q}\|}. \quad (1.9)$$

In both of these equations, $\|\mathbf{x}\| = \sqrt{x^2 + y^2}$ is the euclidean distance of the vector $\mathbf{x}$. In addition, define $R_T(t)$ as the distance from the target to the transmitter and $R_R(t)$ as the distance from the target to the receiver.

The Doppler signal is found using the bistatic radar equation, [16, pp. 119]

$$f_d(t) = -\frac{1}{\lambda} \left(\frac{dR_T(t)}{dt} + \frac{dR_R(t)}{dt} \right) = -\frac{f_c}{c} \left(\frac{dR_T(t)}{dt} + \frac{dR_R(t)}{dt} \right), \quad (1.10)$$

where f_c is the carrier frequency of the transmitted signal, λ is the wavelength of the carrier frequency, and c is the speed of light. The Doppler signal is the scaled sum of the change in length of $R_T(t)$ and $R_R(t)$. The sign in (1.10) allows a decreasing distance to contribute positive Doppler.

For a general moving object and a moving transmitter or receiver (represented generically as $\mathbf{b}(t)$), the derivative of the path distance can be calculated as

$$\begin{aligned} \frac{dR}{dt} &= (\mathbf{v}_g(t) - \mathbf{v}_b(t)) \cdot \hat{\mathbf{u}}_{\mathbf{b}}(t) \\ &= (\mathbf{v}_g(t) - \mathbf{v}_b(t)) \cdot \frac{\mathbf{g}(t) - \mathbf{b}(t)}{\|\mathbf{g}(t) - \mathbf{b}(t)\|} \\ &= ((v_{gx}(t), v_{gy}(t)) - (v_{bx}(t), v_{by}(t))) \cdot \frac{(x(t), y(t)) - (x_b(t), y_b(t))}{\|(x(t), y(t)) - (x_b(t), y_b(t))\|}, \end{aligned} \quad (1.11)$$

which is derived in Appendix B.

In the human tracking situation, the transmitter and receiver are fixed. The Doppler signal in this situation is computed by applying (1.11) to the derivatives in (1.10) to get

$$\begin{aligned}\frac{dR_T(t)}{dt} &= \mathbf{v}(t) \cdot \hat{\mathbf{u}}_t(t) = (v_x(t), v_y(t)) \cdot \frac{(x(t), y(t)) - (x_T, y_T)}{\|(x(t), y(t)) - (x_T, y_T)\|} \\ \frac{dR_R(t)}{dt} &= \mathbf{v}(t) \cdot \hat{\mathbf{u}}_r(t) = (v_x(t), v_y(t)) \cdot \frac{(x(t), y(t)) - (x_R, y_R)}{\|(x(t), y(t)) - (x_R, y_R)\|}.\end{aligned}\quad (1.12)$$

Applying these derivatives to the bistatic radar equation shows that the Doppler signal is a function of the position and velocity of the moving object,

$$\begin{aligned}f_d(x(t), y(t), v_x(t), v_y(t)) &= -\frac{f_o}{c} \left((v_x(t), v_y(t)) \cdot \left(\frac{(x(t), y(t)) - (x_T, y_T)}{\|(x(t), y(t)) - (x_T, y_T)\|} \right. \right. \\ &\quad \left. \left. + \frac{(x(t), y(t)) - (x_R, y_R)}{\|(x(t), y(t)) - (x_R, y_R)\|} \right) \right).\end{aligned}\quad (1.13)$$

An extended Kalman filter can use multiple Doppler signals to estimate the position and velocity parameters in this equation. This results in the extended Kalman filter tracking a moving object. The objective is to extract the Doppler signals at multiple receivers for this Kalman filter.

When extracting the Doppler frequencies, it is useful to know the limits on the frequencies. The maximum Doppler frequency is determined by

$$|f_{max}| = \frac{2|V|f_c}{c}, \quad (1.14)$$

where V is the velocity of the moving object. For tracking a human, a reasonable maximum velocity is around 4 m/s, which is a little faster than the average running pace but corresponds to quick jerks of human movement. For this research, f_c was chosen to be around 905 MHz. This means the maximum Doppler could be as high as $|f_{max}| = 24$ Hz. Usually, people do not run in a building, but this value is reasonable for the test subject in Chapter 5.

1.3 Chapter Outlines

The following chapters describe an alternative method for extracting the Doppler signal. The system developed for testing the new Doppler extraction method is shown in Figure 1.3. This section gives a brief overview of each chapter. In this thesis, the Doppler echo signal refers to the signal received at the receiver; the Doppler signal or Doppler frequency path refers to the path of estimated Doppler frequencies of the Doppler signal.

Chapter 2 introduces the theory for extracting the Doppler signal from the communication system. Two methods of Doppler extraction are introduced. This chapter also goes over the implementation of the communication system, discussing both the hardware and software.

Chapters 3 and 4 discuss additional processing needed to extract the Doppler frequency path from the output of the receiver system. This processing is necessary because of the low SIR of the Doppler echo signal.

Chapter 3 describes the process of estimating the spectrogram of the receiver system output. The spectrogram is important for accurately measuring the frequency of the Doppler signal over time. The two main methods of spectrogram estimation discussed are the discrete Fourier transform (DFT) and the multiple signal classification (MUSIC) algorithm. In addition, a few other processing techniques are discussed.

Chapter 4 describes algorithms for estimating the Doppler frequency path from the spectrogram. There are various path estimation techniques used, such as the Viterbi and BCJR algorithms, which help pick the Doppler signal out of a signal with low SNR or SIR.

Chapter 5 discusses the results of various tests on the system. This chapter also compares the various devices used, the two Doppler extraction methods tested, and the improvements achieved using additional processing techniques.

Chapter 6 concludes the thesis, discussing how the objectives were met. It also describes future work on the subject.

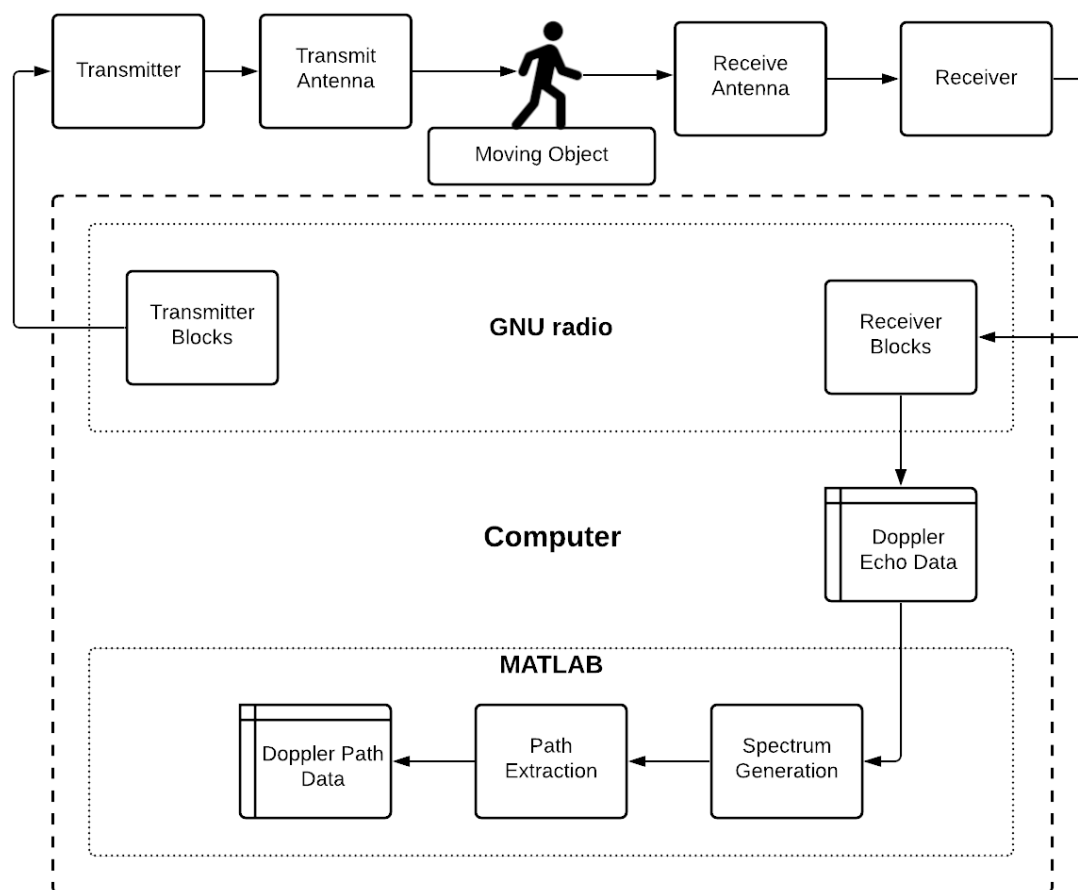


Fig. 1.3: Complete system high-level overview

CHAPTER 2

COMMUNICATION SYSTEM

This chapter looks at the theoretical and physical communication system used to extract the Doppler signal from the received communication signal. Section 2.1 gives an overview of the mathematical theory behind the Doppler extraction. Sections 2.2 and 2.3 review the hardware and software used to implement the theory.

2.1 Theory

The background of Section 1.2.2 introduced the transmitted signal, (1.7), in the bistatic radar system. This signal is transmitted and travels on many different paths to the receiver. The paths of interest, shown in Figure 1.2, are the direct path signal and the Doppler echo signal.

The receiver receives both the direct path signal and the Doppler echo along with additional clutter from other possible reflected paths. The theory ignores the additional clutter as it either is too weak or can be safely assumed to add to the direct path signal as the time delay between the clutter and the direct path is insignificant.

The direct path signal has a time delay of τ and an attenuation of α . The Doppler echo also has a time delay of $\tilde{\tau}$ and an attenuation factor of $\tilde{\alpha}$. The signal received at the receiver can be represented as a superposition of these two signals plus noise $w(t)$,

$$\begin{aligned}
 r(t) = & \alpha A(t - \tau) e^{j(2\pi f_c(t-\tau)(t-\tau) + \phi(t-\tau))} \\
 & + \tilde{\alpha} A(t - \tilde{\tau}) e^{j(2\pi(f_c(t-\tilde{\tau}) + f_d(t-\tilde{\tau}))(t-\tilde{\tau}) + \phi(t-\tilde{\tau}))} + w(t - \tau).
 \end{aligned} \tag{2.1}$$

In this equation, $f_c(t)$ is the carrier frequency, which may change with time due to inconsistencies in the transmitter clock. The value $f_d(t)$ is the Doppler frequency. Substituting

$u = t - \tau$ in (2.1) leads to

$$\begin{aligned} r(t) = & \alpha A(u) e^{j(2\pi f_c(u)u + \phi(u))} \\ & + \tilde{\alpha} A(u + \tau - \tilde{\tau}) e^{j(2\pi(f_c(u + \tau - \tilde{\tau}) + f_d(u + \tau - \tilde{\tau}))(u + \tau - \tilde{\tau}) + \phi(u + \tau - \tilde{\tau}))} + w(u). \end{aligned} \quad (2.2)$$

The value $\tau - \tilde{\tau}$ is the time difference between the signals. In the human tracking application, this time difference will be on the order of nanoseconds when $f_c = 905$ MHz. This time difference is insignificant compared to the length of the symbols. This time difference is also insignificant compared to how slowly the Doppler frequency and carrier frequency offset change. This leads to the approximation

$$r(t) \approx \alpha A(t) e^{j(2\pi f_c(t)(t - \tau) + \phi(t))} + \tilde{\alpha} A(t) e^{j(2\pi(f_c(t) + f_d(t))(t - \tilde{\tau}) + \phi(t))} + w(t). \quad (2.3)$$

The leftover τ and $\tilde{\tau}$ values multiplying $f_c(t)$ and $f_d(t)$ in the complex exponential terms generate phase terms, which can be absorbed into the constants α and $\tilde{\alpha}$,

$$\begin{aligned} r(t) \approx & \alpha e^{j2\pi f_c(t)\tau} A(t) e^{j(2\pi f_c(t)t + \phi(t))} \\ & + \tilde{\alpha} e^{j2\pi(f_c(t) + f_d(t))\tilde{\tau}} A(t) e^{j(2\pi(f_c(t) + f_d(t))t + \phi(t))} + w(t) \\ = & \beta A(t) e^{j(2\pi f_c(t)t + \phi(t))} \\ & + \tilde{\beta} A(t) e^{j(2\pi(f_c(t) + f_d(t))t + \phi(t))} + w(t). \end{aligned} \quad (2.4)$$

The received signal is sent through the receiver of Figure 2.1. This receiver takes in the received signal from (2.4), $r(t)$, and attempts to retrieve the symbols, a_k , from the signal.

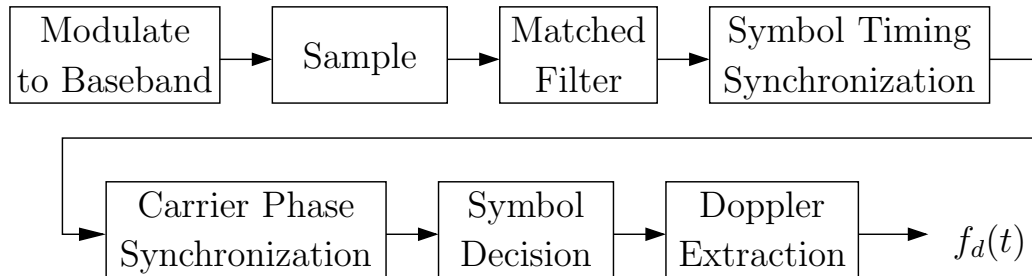


Fig. 2.1: Receiver high level overview.

In the process of retrieving the symbols, the Doppler signal can be extracted. Following the Doppler signal through this system reveals how to extract it.

The receiver first takes the received signal and attempts to demodulate it to baseband. This demodulation uses a set frequency, $\hat{f}_c(t)$, which is not equal to $f_c(t)$. The difference in these values comes from the clock errors on both the transmitter and receiver and from the phase shift of the path. This leads to the received signal not being completely demodulated to baseband,

$$\begin{aligned} r_b(t) &= r(t)e^{-j2\pi\hat{f}_c(t)t} \\ &= \left(\beta A(t)e^{j(2\pi f_c(t)t+\phi(t))} + \tilde{\beta} A(t)e^{j(2\pi(f_c(t)+f_d(t))t+\phi(t))} + w(t) \right) e^{-j2\pi\hat{f}_c(t)t} \\ &= \beta A(t)e^{j(2\pi f_e(t)t+\phi(t))} + \tilde{\beta} A(t)e^{j(2\pi(f_e(t)+f_d(t))t+\phi(t))} + w_b(t), \end{aligned} \quad (2.5)$$

where $f_e(t) = f_c(t) - \hat{f}_c(t)$ is the carrier frequency offset.

After being demodulated to baseband, an ADC samples the signal and sends it through a matched filter. The sampling is done at a sampling period of T ,

$$\begin{aligned} r_b(nT) &= \beta A(nT)e^{j(2\pi f_e(nT)nT+\phi(nT))} \\ &\quad + \tilde{\beta} A(nT)e^{j(2\pi(f_e(nT)+f_d(nT))nT+\phi(nT))} + w_b(nT), \end{aligned} \quad (2.6)$$

and can be also written in discrete form as

$$r_b[n] = \beta A[n]e^{j(2\pi f_e[n]n+\phi[n])} + \tilde{\beta} A[n]e^{j(2\pi(f_e[n]+f_d[n])n+\phi[n])} + w_b[n]. \quad (2.7)$$

The signal is then sent through a matched filter. The matched filter has an impulse response of $h[n] = p[-n]$, with $p[n]$ as the pulse shaping function used by the transmitter. This pulse is known as it is defined by the communication protocol of the system. Sending

(2.7) through this filter results in

$$\begin{aligned}
y[n] &= r_b[n] * p[-n] \\
&= (\beta A[n] e^{j(2\pi f_e[n]n + \phi[n])}) * p[-n] + (\tilde{\beta} A[n] e^{j(2\pi(f_e[n] + f_d[n])n + \phi[n])}) * p[-n] \\
&\quad + w_b[n] * p[-n] \\
&= (\beta A[n] e^{j\phi[n]} e^{j2\pi f_e[n]n}) * p[-n] + (\tilde{\beta} A[n] e^{j\phi[n]} e^{j2\pi(f_e[n] + f_d[n])n}) * p[-n] \\
&\quad + v[n].
\end{aligned} \tag{2.8}$$

Due to the carrier frequency offset and Doppler frequency being relatively low frequencies, (2.8) can be approximated as

$$\begin{aligned}
y[n] &\approx \beta(A[n] e^{j\phi[n]} * p[-n]) e^{j2\pi f_e[n]n} + \tilde{\beta}(A[n] e^{j\phi[n]} * p[-n]) e^{j2\pi(f_e[n] + f_d[n])n} \\
&\quad + v[n].
\end{aligned} \tag{2.9}$$

The symbol timing synchronization block next tries to sample $y[n]$ at the optimal times to retrieve the symbols. Assuming this block samples ideally, then the output of this block is [15, pp. 222]

$$\begin{aligned}
y[kT_s] &= \beta(A[kT_s] e^{j\phi[kT_s]} * p[-kT_s]) e^{j2\pi f_e[kT_s]kT_s} \\
&\quad + \tilde{\beta}(A[kT_s] e^{j\phi[kT_s]} * p[-kT_s]) e^{j2\pi(f_e[kT_s] + f_d[kT_s])kT_s} \\
&\quad + v[kT_s] \\
&= \beta\left(\sum_k a[k] p[k - kT_s] * p[-kT_s]\right) e^{j2\pi f_e[kT_s]kT_s} \\
&\quad + \tilde{\beta}\left(\sum_k a[k] p[k - kT_s] * p[-kT_s]\right) e^{j2\pi(f_e[kT_s] + f_d[kT_s])kT_s} \\
&\quad + v[kT_s] \\
&= \beta \frac{a[k]}{T} e^{j(2\pi f_e[kT_s]kT_s)} + \tilde{\beta} \frac{a[k]}{T} e^{j2\pi(f_e[kT_s] + f_d[kT_s])kT_s} + v[kT_s],
\end{aligned} \tag{2.10}$$

where $a(k)$ is the symbol at time k . The $1/T$ values can be absorbed into the scalar values and (2.10) can be written using the symbol indexing

$$\begin{aligned} y_k &= \beta a_k e^{j2\pi f_{e,k}k} + \tilde{\beta} a_k e^{j2\pi(f_{e,k}+f_{d,k})k} + v_k \\ &= \left(\beta a_k + \tilde{\beta} a_k e^{j2\pi f_{d,k}k} \right) e^{j2\pi f_{e,k}k} + v_k. \end{aligned} \quad (2.11)$$

The next block, the carrier phase synchronization block, uses a phase-locked loop (PLL) to remove the carrier frequency offset value. Ideally, the output of this block is

$$z_k = \beta a_k + \tilde{\beta} a_k e^{j2\pi f_{d,k}k} + \nu_k. \quad (2.12)$$

The final block of the receiver, the symbol decision block, takes the output of the carrier phase synchronization block and makes a decision on the symbols. The output of this block is

$$x_k = \hat{a}_k. \quad (2.13)$$

When making the decision, the Doppler frequency and any additional noise are all lost.

The Doppler signal is extracted by using the output of the carrier phase synchronization block, z_k , and the output of the symbol decision block, x_k ,

$$\begin{aligned} \text{DEM}_1(k) &\triangleq \frac{z_k - x_k}{x_k} = \frac{\beta a_k + \tilde{\beta} a_k e^{j2\pi f_{d,k}k} + \nu_k - \hat{a}_k}{\hat{a}_k} \\ &= \beta - 1 + \tilde{\beta} e^{j2\pi f_{d,k}k} + \text{noise}. \end{aligned} \quad (2.14)$$

This equation assumes the symbol estimation is accurate in order for $a_k = \hat{a}_k$. (2.14) is referred to as the First Doppler Extraction Method (DEM).

Once the data is manipulated in the form of (2.14), additional processing removes the noise and the DC components of this signal and then estimates the Doppler signal, as discussed in Chapters 3 and 4.

2.1.1 Second Doppler Extraction Method

The Doppler extraction theory presented assumes the carrier phase synchronization block ideally removes the carrier frequency offset and outputs the symbols in (2.12). However, this assumption is incorrect and the true output is closer to

$$z_k = \beta a_k + K a_k e^{j2\pi f_{d,k} k} + \tilde{K} a_k e^{j2\pi f_{d,k} k} + \nu_k. \quad (2.15)$$

In this equation, K and $\tilde{K}$ are unknown magnitudes, which vary with the frequency of the Doppler. Their exact values are hard to calculate due to the nonlinear nature of the PLL. A further discussion on how the Doppler signal affects the communication signal and on how the PLL operates sheds insight into why (2.15) is a closer representation of the output.

Figure 2.2 shows how the Doppler affects the symbols in the symbol space. A received symbol a_k has a small rotating Doppler offset added to it, as seen by rewriting (2.12) as

$$z_k = a_k(\beta + \tilde{\beta} e^{j2\pi f_{d,k} k}) + \nu_k. \quad (2.16)$$

This offset rotates the Doppler around the transmitted symbol. When there is a carrier

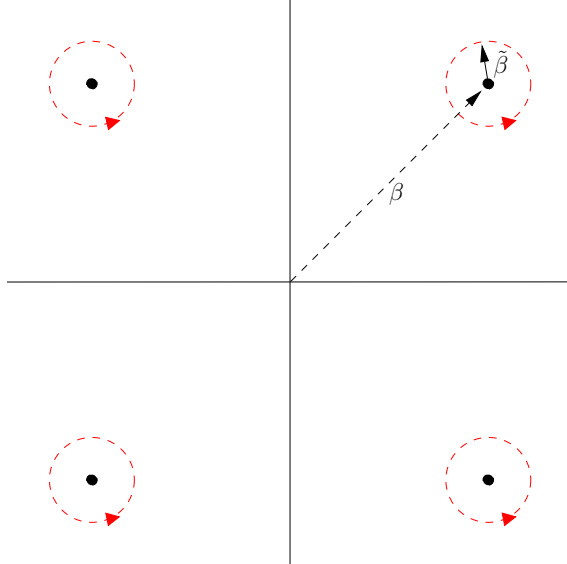


Fig. 2.2: Doppler rotations around symbols in the signal space.

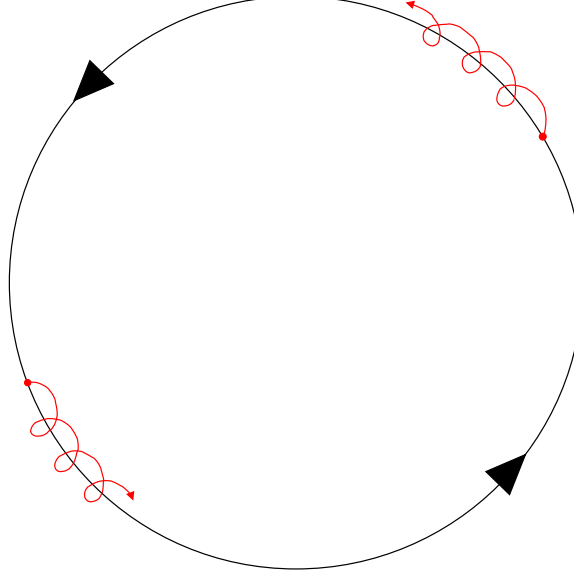


Fig. 2.3: Doppler rotations around symbols in the signal space with a rotating symbol constellation.

frequency offset, this whole constellation is rotating. In this situation, the Doppler rotates as helices, which is seen in Figure 2.3.

The carrier phase PLL, shown in Figure 2.4, attempts to correct for the rotations caused by the carrier frequency offset. The PLL operates by taking in one of the sampled symbols, y_k , and multiplies it by $e^{-j\hat{\theta}}$. The value $\hat{\theta}$ is an estimate of how much the carrier frequency offset has rotated the symbol in the symbol space from the correct value, which is assumed to be the closest symbol. The multiplication result becomes the output z_k .

In order to calculate the θ for the next iteration, z_k is sent through a symbol decision block to estimate which of the symbols z_k might be. Then the error between the estimated symbol and the actual symbol is calculated. This error accounts for errors from the carrier phase and frequency offsets, and from the Doppler frequency as both values pull the symbol away from the decision points. This means the error is a combination of both values.

This error is sent into the loop filter. The loop filter's purpose is to track the frequency and phase of the carrier frequency offset when it is locked on, which can take a few iterations of the loop to do. It has two adjustable parameters, K_1 and K_2 , which control the bandwidth of the filter. If the carrier frequency offset is less than the Doppler frequency shift, then the

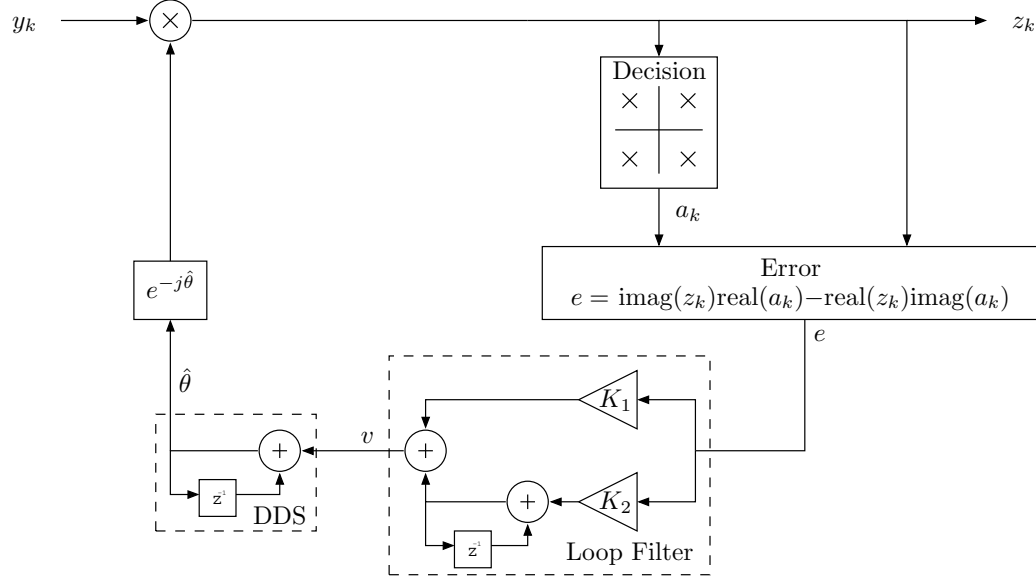


Fig. 2.4: Flowchart of the carrier phase PLL.

filter bandwidth can be set to filter out the Doppler frequency. However, the more realistic situation is the carrier frequency offset is much larger than the Doppler frequency shift and the loop filter tracks both frequencies.

The output of the loop filter is sent through a direct digital synthesizer (DDS) to update the estimate on θ . This means both the carrier frequency offset and the Doppler frequency influences θ . This leads to two Doppler peaks at both the positive and negative Doppler frequencies. It is ambiguous which frequency is real and which was created by the PLL. This becomes a major problem as the sign of the Doppler is crucial in determining if an object is moving towards or away from the receiver. Therefore, another method of extracting the Doppler frequency was developed to avoid this ambiguity.

This new method circumnavigates the PLL by using the value y_k instead of z_k in the Doppler extraction. This value is the output of the symbol timing synchronization block given by (2.11). Taking this value and the estimated symbols, a_k , the Doppler can be

extracted using

$$\begin{aligned} \text{DEM}_2(k) \triangleq \frac{y_k}{x_k} &= \frac{(\beta a_k + \tilde{\beta} a_k e^{j2\pi f_{d,k}k}) e^{j2\pi f_{e,k}k} + \nu_k}{\hat{a}_k} \\ &= \beta e^{j2\pi f_{e,k}k} + \tilde{\beta} e^{j2\pi(f_{e,k}+f_{d,k})k} + \text{noise}. \end{aligned} \quad (2.17)$$

This equation is referred to as the Second DEM.

The Fourier transform of this value is two Dirac deltas, one at the carrier frequency offset and the other at the carrier frequency offset plus the Doppler shift. The difference between these values is the Doppler frequency. Therefore, this Second DEM is another way to extract the Doppler signal and does not suffer from the problems in the PLL. Chapters 3 and 4 go into more detail about how the Doppler frequency path can be extracted from this second method.

2.2 Hardware Description

The hardware in the implemented system consists of a transmitter, one or more receivers, antennas, one or two amplifiers, and one or more computers. The multiple receivers, amplifiers, and computers depend on the test setup. Figure 2.5 shows the hardware for the simplest setup. This section goes over all of the hardware used, but the specific configuration for each test is listed in Chapter 5.

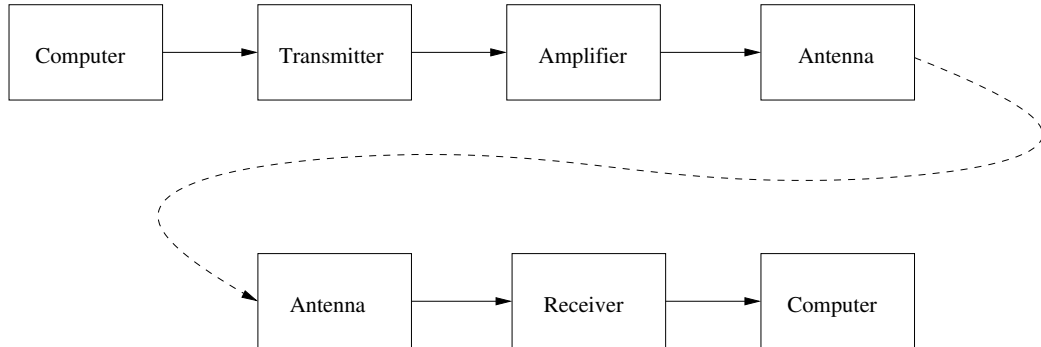


Fig. 2.5: Hardware overview.

2.2.1 Transmitters and Receivers

The transmitter and receiver devices in Table 2.1 were used in various tests. All of these devices except the RTL-SDR are made by Ettus Research. These devices were chosen because GNU Radio has built-in support for them.

The RTL-SDR dongle strictly operates as a receiver and cannot be configured as a transmitter. The RTL-SDR device was used in some of the early tests but suffered the most from clock drift and noise, making it harder to extract the Doppler frequency path. The advantage of the RTL-SDR receiver is its low cost and small package.

The Ettus Research devices are listed in increasing price in Table 2.1. The B200 and B210 devices are very similar in their hardware, differing mostly in the number of ports each device has. The B200mini is a miniature version of the B200 device but lacks a GPSDO clock. This hindered the usefulness of the B200mini as the GPSDO clocks are significantly better than the onboard clocks.

The E320s are more advanced devices capable of running software on the devices themselves. These devices are more expensive than the previous devices and would have had better clocks, except there is a major problem with the devices. The E320s introduce extra

Table 2.1: Transmitter and receiver devices.

Device	Frequency Range	Max Sample Rate	Ports	Clocks
RTL-SDR	24 - 1766 MHz	3.2 MS/s	1 Rx	On-board
B200	70 - 6000 MHz	61.44 MS/s	1 Tx, 1 Rx	On-board, GPSDO, and reference
B200mini	70 - 6000 MHz	61.44 MS/s	1 Tx, 1 Rx	On-board and reference
B210	70 - 6000 MHz	61.44 MS/s	2 Tx, 2 Rx	On-board, GPSDO, and reference
E320	70 - 6000 MHz	61.44 MS/s	2 Tx, 2 Rx	On-board, GPSDO, and reference
N310	10 - 6000 MHz	153.6 MS/s	4 Tx, 4 Rx	On-board, GPSDO, and reference

phase jitter on their clocks. The exact cause of this phase jitter was never determined. However, the E320s were the only devices with an adjustable master clock. It is theorized that the clock on the devices would oscillate between two frequencies in an attempt to get as close to the desired clock value as possible, but this oscillation would yield extra phase jitter on the clock. This jitter shows up as extra noise in the same frequency band as the Doppler and makes it harder to extract the Doppler from the communication signal. Therefore, the E320 devices were used sparingly in testing.

The final Ettus Research devices, the N310s, were obtained late in the testing process. These devices are the most expensive but have the best clocks. These devices also have more ports than the other devices. The N310s have four ports for transmitting or receiving. This was useful for many of the later tests where multiple receivers were used in different positions.

2.2.2 Antennas

Two different types of antennas were used for this system. Table 2.2 shows a few specifications for these antennas. The Heyrtz antenna was used for the earliest tests. Due to the transmit frequency of the tests being in the 902 – 928 frequency range, this antenna attenuated the signal significantly, making it useless when the transmitter and receivers were spread out further than a few meters. The HyperLink antennas were obtained when more transmitter power was needed. These antennas allowed the signal to travel further than a couple of meters.

Table 2.2: Antenna parameters.

Antenna	Frequency Range	Gain	Size
Heyrtz Retractable Antenna	144 - 430 MHz	2.15 dBi	420 mm extended, 140 mm retracted
HyperLink Wireless "Rubber Duck" Antenna	902 - 928 MHz	5 dBi	375 mm

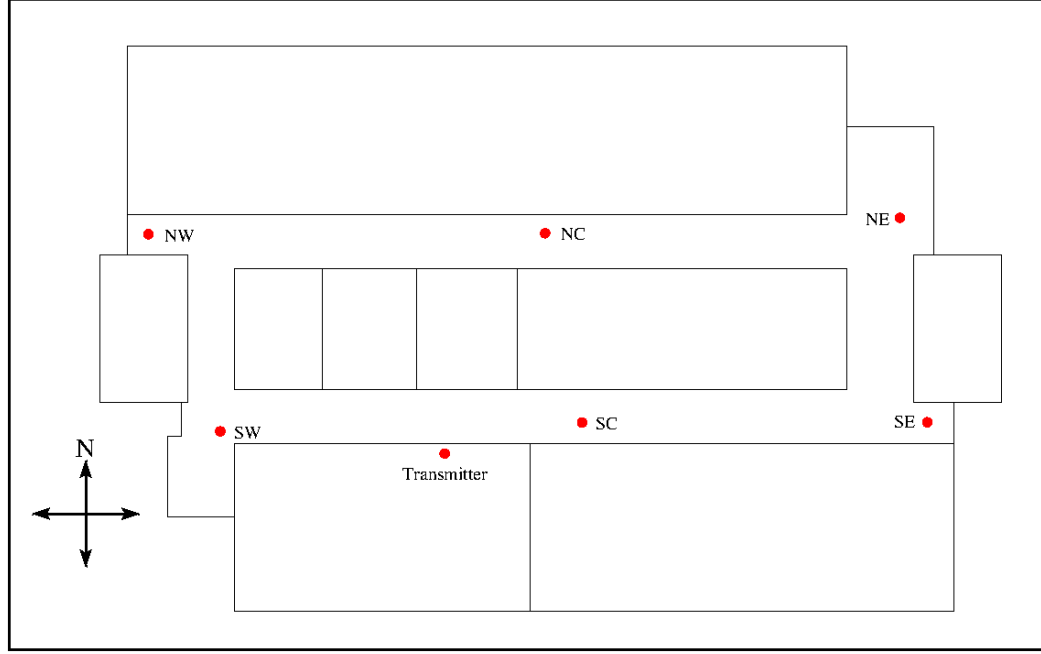


Fig. 2.6: Building antenna positions.

The placement of the antennas on both the transmit and receive side of the communication system varied from test to test. Initially, the antennas were placed side by side to test how much Doppler signal was reflected directly back.

The next tests used antennas in the ceiling throughout the entire second floor of the Sant Engineering Innovation building at Utah State University. The placement of the ceiling antennas is shown in Figure 2.6. The transmit antenna, also shown in this figure, was mounted on a tripod and moved around to different positions.

2.2.3 Other Hardware

The other hardware used were a couple of amplifiers and a couple of computers for sending and receiving the data.

The amplifiers used were Shireen 900 MHz 2 Watt amplifiers. These devices were used on the transmitter in order to increase the power transmitted. This was needed to make sure there was enough power for the receivers to pick up the Doppler echos.

The computers used were either a Lambda Labs workstation or Lambda Labs laptops,

although other desktop or laptop computers could also work. The computers all ran Ubuntu 18.04.5 and were installed with GNU Radio 3.8 and all the required packages to work with the transmitter and receiver devices mentioned in Section 2.2.1.

2.3 GNU Radio QPSK System

The communication protocol chosen for this research was QPSK, which was introduced in Section 1.2.1. QPSK was chosen for the prototype system rather than the WiFi communication protocol as it was a simpler system to test out the basic theory on. Each data symbol in QPSK is one of four possible constellation points transmitted on an IQ signal, with each point encoding two bits of binary. The IQ components are transmitted by modulating the in-phase component with a cosine wave and the quadrature component with a sine wave before adding them together and transmitting them. This allows the two components to be separated at the receiver by a similar demodulation. More information about this standard and basic communication theory can be found in Dr. Rice's book on digital communications [15].

GNU Radio is a software-defined radio library used for easy prototyping and implementation of communication systems. It was chosen as the software used to implement this system because the communication system can be implemented in predefined blocks and easily modified for the desired application. The following sections discuss how this system was implemented in GNU Radio. The parameters listed in this chapter depend on the test being run. The specifics are given in Chapter 5.

2.3.1 Transmitter

The transmitter portion of the QPSK system is easy to implement in GNU Radio. Figure 2.7 shows the operations in the transmitter, and Figure 2.8 shows the two GNU Radio blocks containing all of these operations.

The Constellation Modulator block, shown on the left of Figure 2.8, accomplishes the serial-to-parallel conversion, look-up tables (LUT), and pulse shaping filter operations of

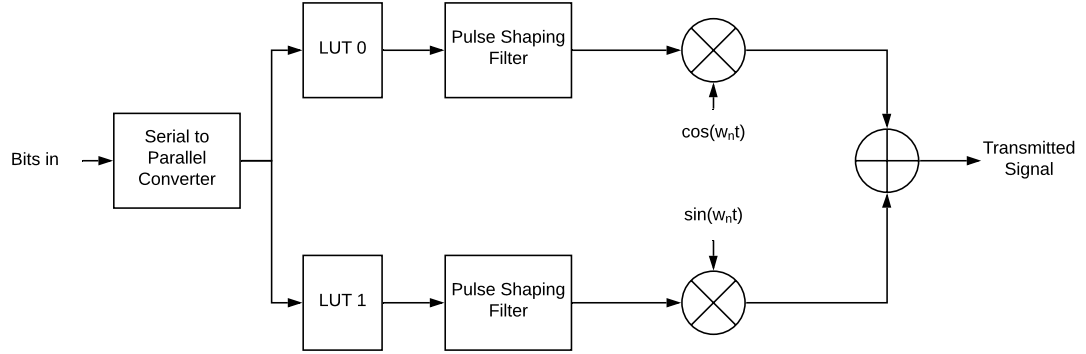


Fig. 2.7: Transmitter system high level overview.

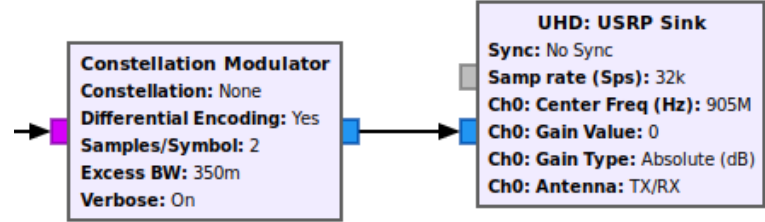
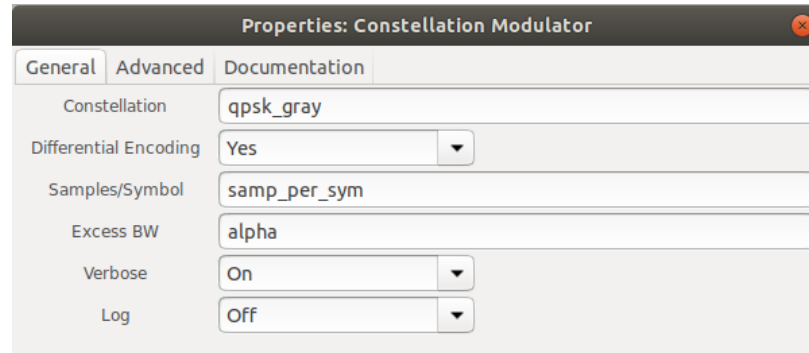


Fig. 2.8: Transmitter system GNU Radio flowgraph.

the transmitter system. The input stream requires a stream of bytes. The output stream is the IQ data in a complex data format ($I + jQ$).

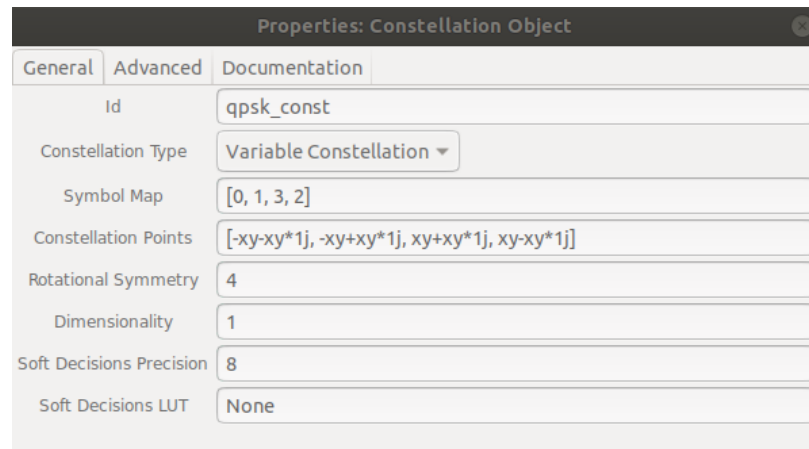
The Constellation Modulator block gives a little control over this process, as seen in Figure 2.9. For the implemented system, the constellation was set as a QPSK constellation using a Constellation Object block (Figure 2.10). The system also used differential encoding, although this parameter did not affect the ability to extract the Doppler signal. The samples per symbol and the excess bandwidth were set as variables as they influenced the performance of the system.

The second block in Figure 2.8 is the UHD: USRP Sink. This block takes care of modulating the IQ data, adding the IQ components together, and transmitting the resulting signal. This block is specifically for the Ettus Research devices mentioned above. Other transmitter devices may have a similar block.



Properties: Constellation Modulator	
General	Advanced
Constellation	qpsk_gray
Differential Encoding	Yes
Samples/Symbol	samp_per_sym
Excess BW	alpha
Verbose	On
Log	Off

Fig. 2.9: Constellation Modulator properties.



Properties: Constellation Object	
General	Advanced
Id	qpsk_const
Constellation Type	Variable Constellation
Symbol Map	[0, 1, 3, 2]
Constellation Points	$[-xy-xy*1j, -xy+xy*1j, xy+xy*1j, xy-xy*1j]$
Rotational Symmetry	4
Dimensionality	1
Soft Decisions Precision	8
Soft Decisions LUT	None

Fig. 2.10: Constellation Object properties.

The goal of the transmitter blocks is to communicate with a transmitter device. The modulation and transmission of the data occur within the device. This block sends the data and any important parameters needed for setting up the transmission. The parameters can be controlled using the properties of the appropriate transmitter block. The important UHD: USRP Sink block properties are shown in Figures 2.11 and 2.12.

This first page of parameters in Figure 2.11 allows for control of the initialization parameters, such as the clock rate, sample rate, and source of the clock. These values all depended on the specific device being used.

The third page of parameters in Figure 2.12 allows for control of the gain and center frequency of the transmission. The gain was allowed to change on run-time from 0 - 90 dB. The center frequency chosen for the tests ranged in the ISM band of 902 - 928 MHz. In

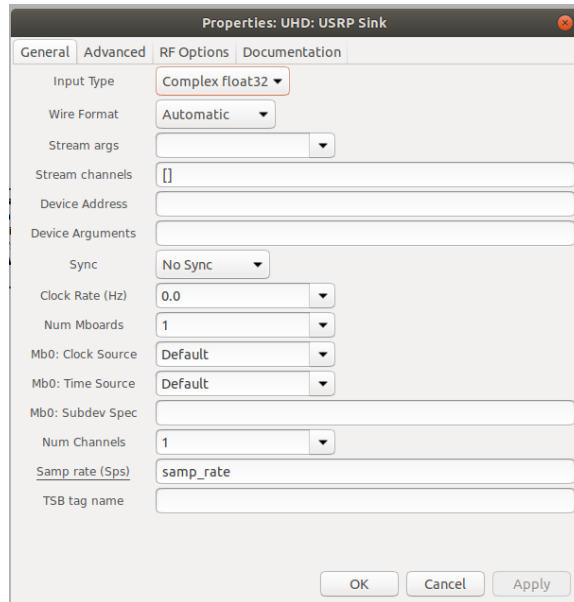


Fig. 2.11: UHD:USRP Sink general properties.

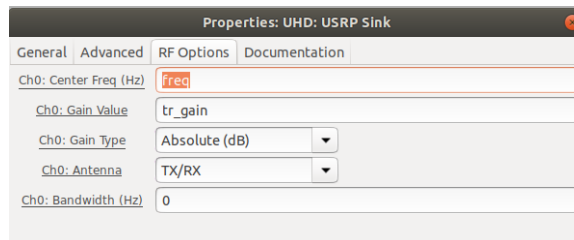


Fig. 2.12: UHD:USRP Sink RF properties.

addition, the desired transmit port for the device needs to be specified.

2.3.2 Receiver

The receiver is more complicated than the transmitter. This is because of the need to perform symbol timing synchronization and carrier phase synchronization in order to recover from errors between the transmitter and receiver clocks, as mentioned in Section 2.1. The overview of the receiver system is shown in Figure 2.14.

GNU Radio provides a lot of functionality to simplify this system and adjust how both the symbol timing synchronization and carrier phase synchronization are implemented. Figure 2.13 shows the flowgraph used to accomplish everything in Figure 2.14.

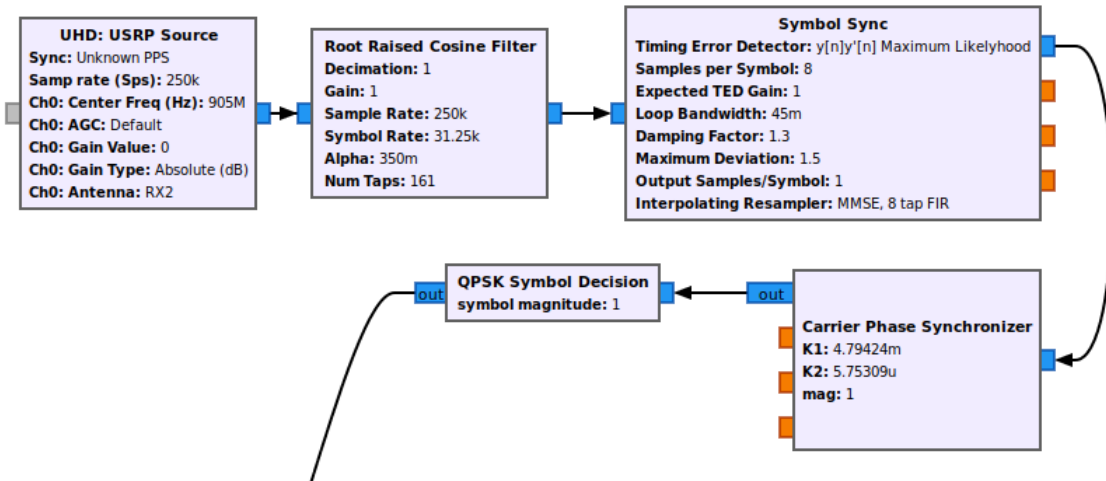


Fig. 2.13: The first part of the receiver system.

The UHD: USRP Source is nearly identical to the UHD: USRP Sink in terms of parameters. The main difference with this block is it receives rather than transmits. However, the parameters will be the same as described in Section 2.3.1 and should match those of the transmitter, except for the gain value. The RTL-SDR receiver uses its own receiver block known as the Soapy Source, which is very similar to the UHD: USRP Source.

The Root Raised Cosine Filter acts as the matched filter of the system. The parameters of this block (Figure 2.15) should match what was set in the transmitter.

The Symbol Sync block takes care of the symbol timing synchronization, ensuring the symbols are sampled at the correct time. The Symbol Sync block gives a lot of control over the synchronization process (Figure 2.16).

The timing error detector, loop bandwidth, and damping factor are all variable quantities used in tuning the system. This allows the system to get the most accurate symbol samples. Most of the variables were adjustable in between tests, except for the loop bandwidth, which was run-time configurable.

The next block in Figure 2.13 is the Carrier Phase Synchronizer. This is a custom GNU Radio block written to perform the carrier phase synchronization. This block replaces GNU Radio's Costas Loop block in order to better track and correct the phase and frequency offsets caused by clock errors in the devices. This block is controlled by setting $K1$, $K2$

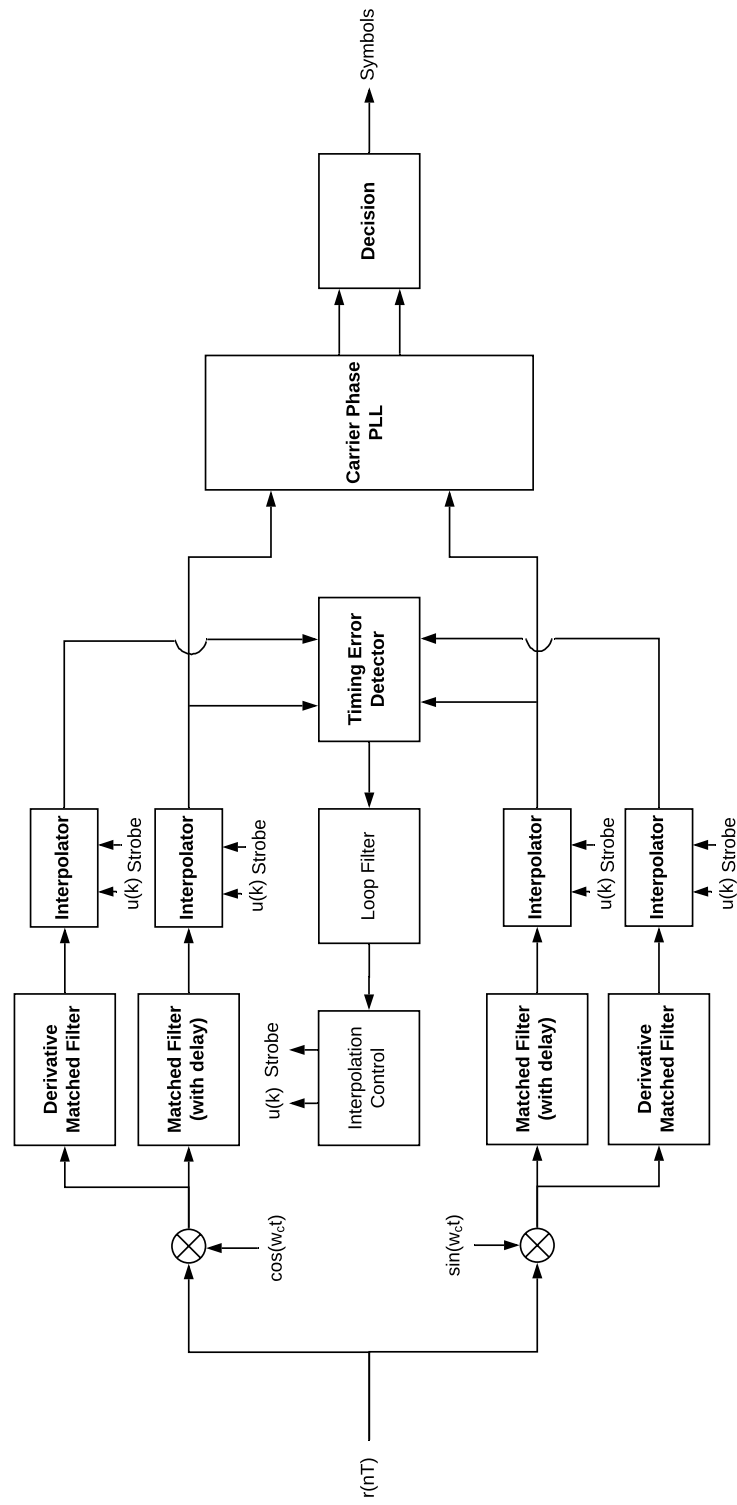


Fig. 2.14: QPSK receiver high level overview.

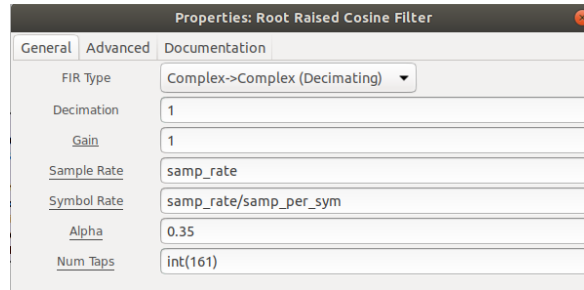


Fig. 2.15: Root Raised Cosine Filter properties.

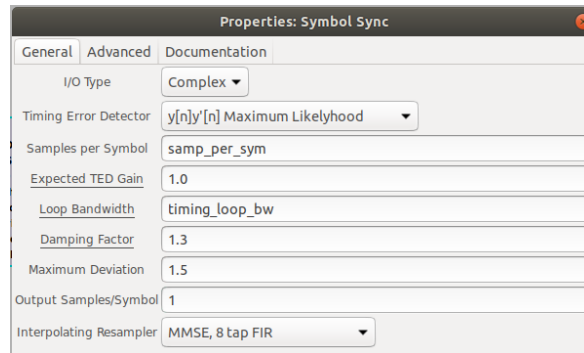


Fig. 2.16: Symbol Sync properties.

to different values, which controls the loop bandwidth of the loop filter, as calculated in [15, pp. 682]. The magnitude variable should correspond to the magnitude of the QPSK constellation points.

After timing and clock errors are corrected, the symbols are sent into the QPSK Symbol Decision block, where the symbols are estimated to be one of the four constellation symbols in Figure 1.1. After this, the symbols could be used to retrieve the bits transmitted, but this is not necessary for the Doppler extraction.

2.3.3 Hardware Constraints

The previous sections described the general communication system, which will work for some hardware configurations. However, most hardware places a constraint on the sample rate it can communicate at. This constraint comes from the clock rate of the device. Table 2.1 lists a few of the constraints on the sample rate for different devices.

Most of the devices in this table prefer a higher sample rate. It can be hard for the GNU

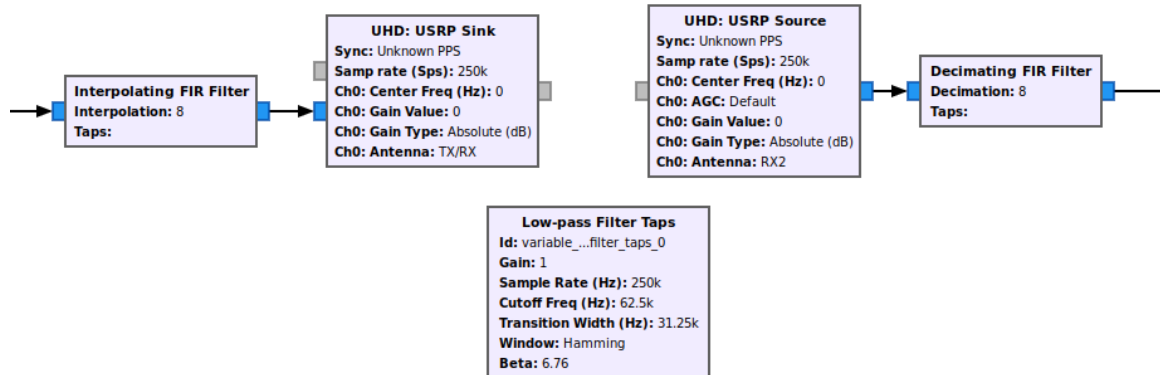


Fig. 2.17: Interpolation and decimation blocks in the transmitter and receiver GNU Radio flowcharts.

Radio receiver system to keep up with this sample rate, depending on the processing power of the computer. Therefore, the GNU Radio communication system can be improved by adding in an Interpolating FIR Filter block before the transmitter block and a Decimating FIR Filter block after the receiver block (Figure 2.17). This allows the sample rate for the hardware to be higher than the sample rate for the GNU Radio transmitter and receiver flowcharts.

2.4 Doppler Extraction

The Doppler frequency extraction was done in the GNU Radio receiver flowchart by adding additional blocks to perform the operations of (2.14) or (2.17), depending on which method was used. Figure 2.18 shows how the Doppler was extracted using the First DEM, while Figure 2.19 shows how the Doppler was extracted using the Second DEM. A File Sink block was also added in order to record the data.

Once GNU Radio extracted the Doppler using either method, the signal was imported into MATLAB and additional signal processing was done to detect and estimate the Doppler frequency path. This processing is the subject of Chapters 3 and 4.

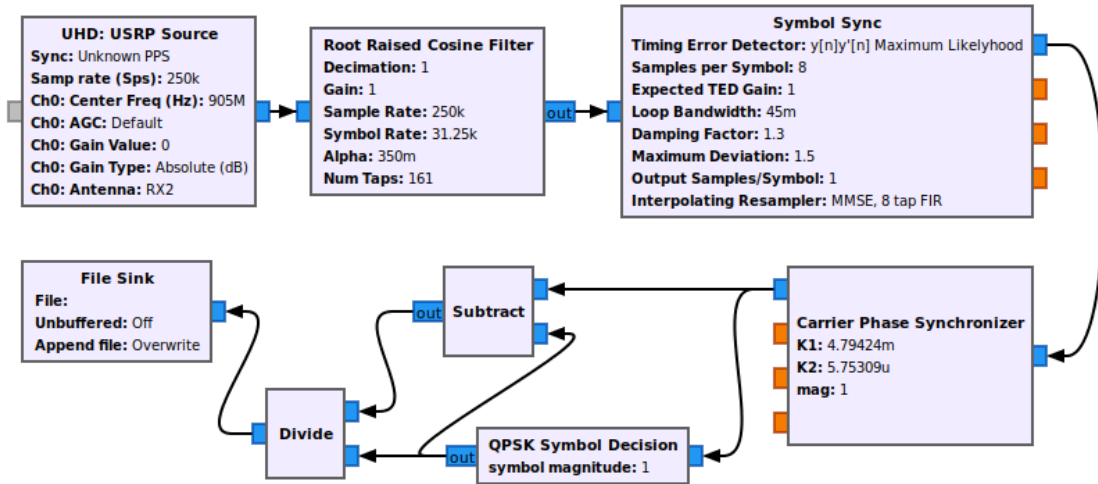


Fig. 2.18: First DEM implemented in GNU Radio.

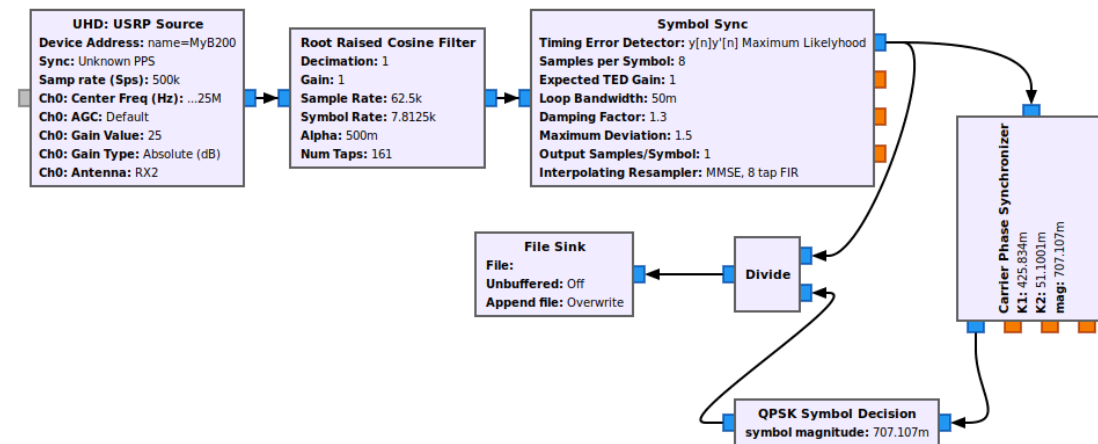


Fig. 2.19: Second DEM implemented in GNU Radio.

CHAPTER 3

SPECTRAL ESTIMATION

The GNU radio communication system is able to remove the symbols from the communication signal using both DEMs of (2.14) and (2.17). Once the symbols are removed, additional signal processing algorithms are used to extract the Doppler signal. This chapter and Chapter 4 explain the signal processing algorithms used to extract the Doppler signal from the output of the GNU radio communication system.

The process of extracting the frequency path consists of transforming the GNU radio output into a spectrogram and then finding the best path through this spectrogram. This chapter discusses methods used to create the spectrogram and improvements made to these methods. These improvements help increase the resolution of the spectrogram and remove the carrier frequency offset and DC components in order to improve the SIR.

The spectrograms discussed in this chapter and the following chapters are plotted in the form of Figure 3.1. The plots are colormap images of magnitude spectra plotted over time. Time progresses downward on the y-axis in the images, and the x-axis shows the frequency bins. These plots are plotted using a dB color scale, with the scale on the right of the image.

Two different methods are employed to create the spectra of the spectrogram: the DFT and the MUSIC algorithm.

3.1 Discrete Fourier Transform

The DFT is the most common way to transform a signal into the frequency domain. It can be implemented using the fast Fourier transform (FFT). The basic equation for the

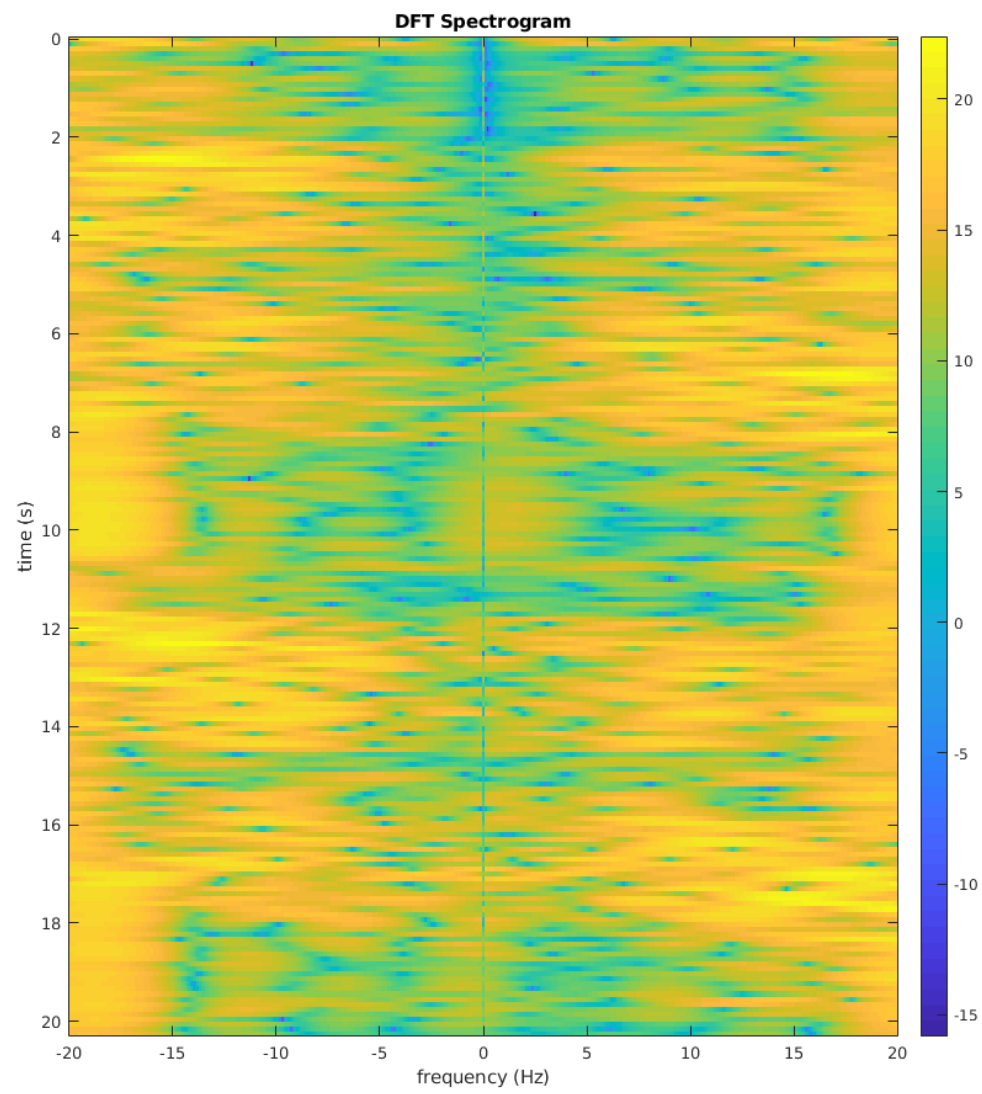


Fig. 3.1: Example of a spectrogram.

DFT is given by [17, Ch. 8]

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] W_N^{kn} \\ W_N &= e^{-\frac{j2\pi}{N}}. \end{aligned} \quad (3.1)$$

This transform takes N samples of data and transforms them into N frequency bins, where frequency bin k represents the actual frequency given by

$$f_k = \begin{cases} \frac{kF_s}{N}, & k \leq N/2 \\ F_s - \frac{kF_s}{N}, & k > N/2. \end{cases} \quad (3.2)$$

F_s is the sample frequency of $x[n]$.

The DFT is simply a sampled version of the DTFT. The DFT samples the DTFT at N points. This means the spectral resolution of the DFT is F_s/N . If N is increased to M by zero-padding the signal to get more bins in the DFT, then the underlying DTFT spectrum will become clearer, as shown in Figure 3.2. However, the underlying spectrum is still the DTFT of the original N signal points, which suffer from windowing effects.

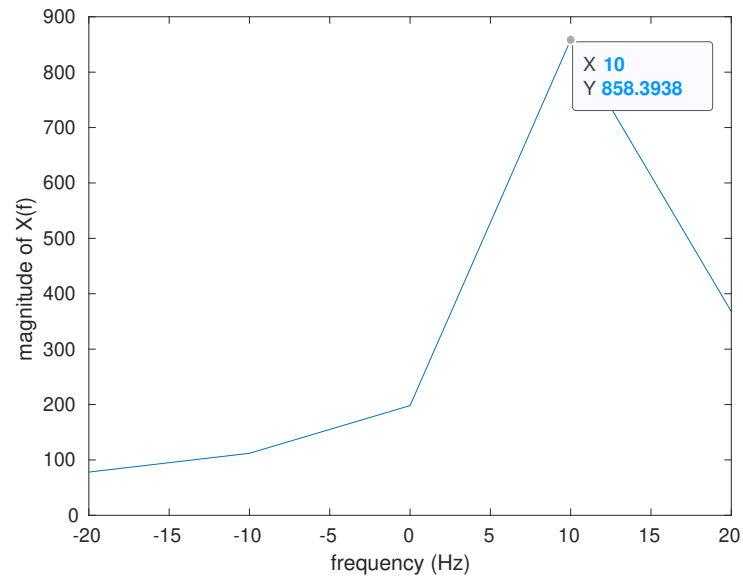
Windowing effects are the results of having limited data. Any signal processed by a computer is considered a windowed signal, as it is limited in length. The basic equation for a windowed signal is

$$x_N[n] = \begin{cases} x[n]w[n], & 0 \leq n < N-1 \\ 0, & \text{otherwise,} \end{cases} \quad (3.3)$$

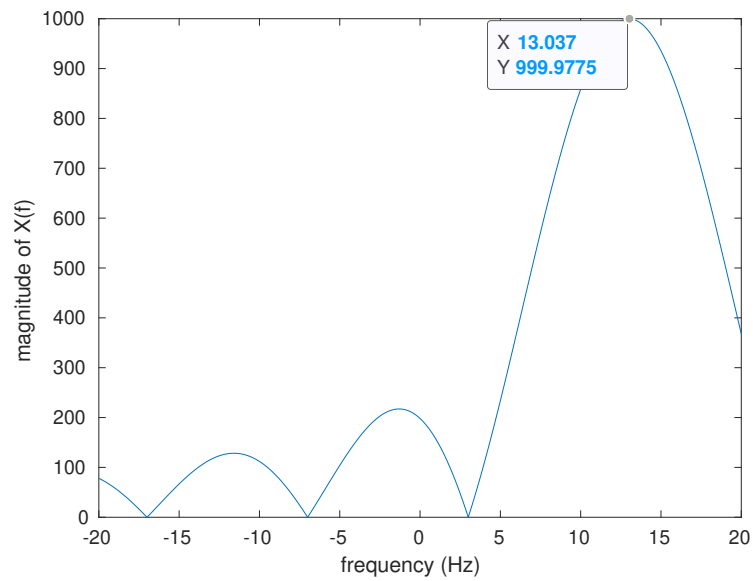
where $x[n]$ is the original signal and $w[n]$ is a windowing function. The simplest windowing function is the rectangular window, $w[n] = 1$. When applying a rectangular window in the frequency domain, this results in convolving the signal with a periodic sinc function in the frequency domain,

$$X_N(f) = X(f) * \frac{\sin(\pi f N)}{\sin(\pi f)} e^{-j\pi f(N-1)}. \quad (3.4)$$

The convolution shifts a copy of the periodic sinc function to each frequency in $X(f)$ and



(a) No zero-padding



(b) Zero padding

Fig. 3.2: Attempts to estimate a signal at 13 Hz with only 10 Hz of resolution.

scales it by the frequency. This means each frequency is represented by a periodic sinc function, as seen in Figure 3.2b.

Each of the periodic sinc functions has a main lobe and side lobes which can interfere with other signals. This interference is known as spectral leakage as each of the frequencies leak into other frequency bins. This is seen in Figure 3.3. In order to limit spectral leakage, a longer window is needed. A longer window decreases the width of the main lobe and decreases the height of the side lobes.

A longer window is a problem for estimating the Doppler frequency. The Doppler frequency changes with time because it varies with the moving object trajectory. The longer the data input into the DFT, the more the Doppler frequency varies over the data resulting in a smearing of frequency content.

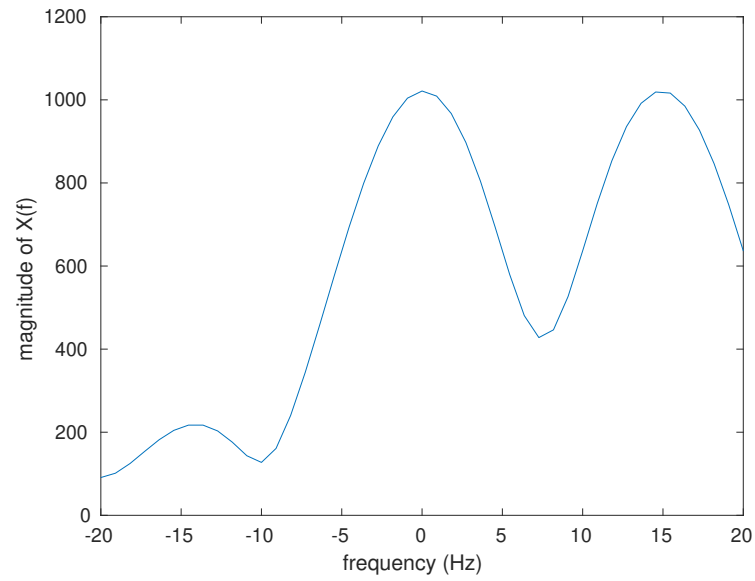
The Doppler frequency for the human tracking application can range from -24 Hz to 24 Hz, as mentioned in Section 1.2.2. It is not unreasonable to assume a human can stop and change direction in about a second, which could cause a lot of smearing if a second of data is input into the DFT.

The trade-off is to find the shortest windowing length possible that keeps the frequency components of interest from being overcome by the spectral leakage of other frequencies in the signal. This can be especially hard in a signal with noise or other large frequency components, like the DC component in the extracted Doppler signal.

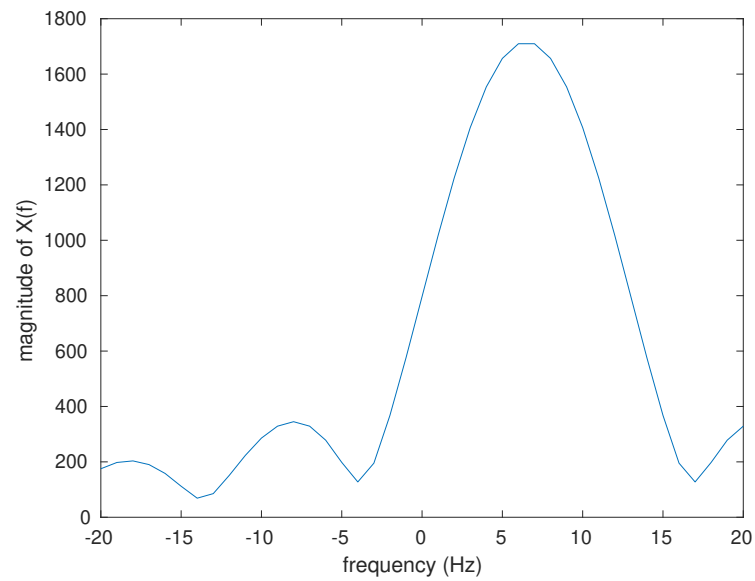
3.2 Multiple Signal Classification

MUSIC is an algorithm capable of improving the spectral resolution of a noisy signal without extending the length of the signal. The MUSIC algorithm establishes a parametric model for the signal as a linear combination of sinusoidal components. By computing the autocorrelation of the data, it is possible to represent the data as a combination of signals in a signal subspace and a noise subspace. This decomposition is valid in situations where the noise is white and has a mean of zero.

The goal of the MUSIC algorithm is to find the noise subspace and then find all of the signals orthogonal to this subspace. The first step to finding the noise subspace is to find



(a) Signals at 0 Hz and 15 Hz



(b) Signals at 5 Hz and 8 Hz

Fig. 3.3: Spectral leakage may result in the loss of the ability to differentiate signals.

the autocorrelation matrix of the data. This matrix is an M by M matrix, with M being an adjustable parameter greater than the number of frequencies in the signal of interest, P . The autocorrelation matrix of the signal $x[n]$ is computed by

$$R_{xx} = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=0}^{N-1} \mathbf{x}[n_i] \mathbf{x}[n_i]^*, \quad (3.5)$$

where

$$\mathbf{x}[n_i] = \begin{bmatrix} x[n_i], x[n_i + 1], \dots \end{bmatrix}^T. \quad (3.6)$$

This equation can be approximated with a large N value for finite signals.

The $M - P$ eigenvectors of R_{xx} corresponding to the $M - P$ lowest eigenvalues span the noise space of $x[n]$ [18, pp. 337]. All frequency vectors orthogonal to this noise space are in the signal space. To determine the frequencies, the inner product

$$M(f) = \sum_{k=p+1}^M |\mathbf{s}(f)^H \mathbf{u}_k|^2, \quad (3.7)$$

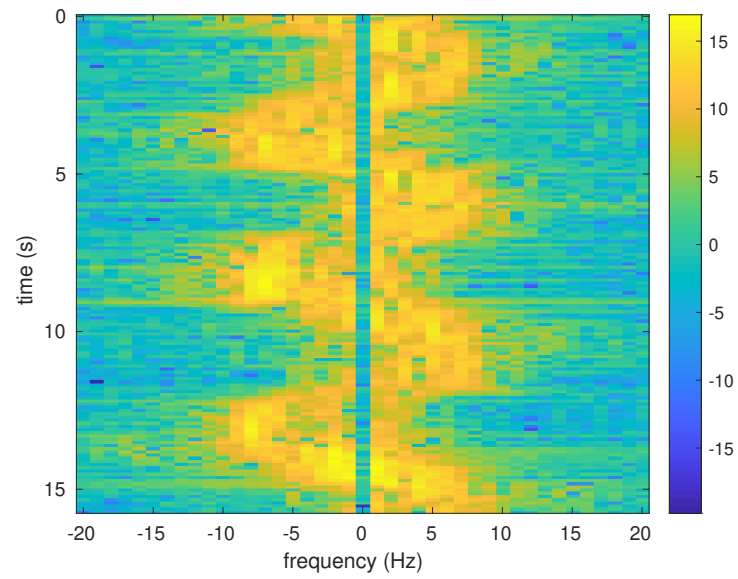
is computed, where

$$\mathbf{s}^T(f) = \begin{bmatrix} 1, e^{j2\pi f}, e^{j2\pi 2f}, \dots, e^{j2\pi(M-1)f} \end{bmatrix}. \quad (3.8)$$

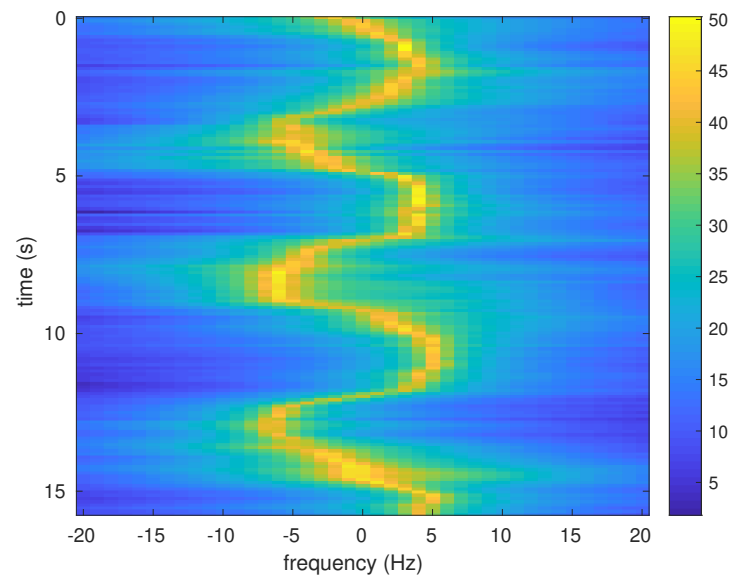
Here, $\mathbf{u}_k$ are the eigenvectors of R_{xx} corresponding to the lowest $M - P$ eigenvalues.

Equation (3.7) computes the inner product between arbitrary frequency vectors and the noise space. The more orthogonal the frequency vector is to the noise space, the closer $M(f)$ is to zero. Plotting the inverse of this function, $P(f) = 1/M(f)$, gives the MUSIC spectrum, which has an appearance similar to the spectrum of the DFT; frequencies in the signal appear as tall slim peaks.

The peaks of MUSIC offer greater spectral resolution without the need for more data. Instead, less data means less accuracy in the computation of the autocorrelation matrix. However, typically MUSIC works better with shorter data lengths than the DFT does. Figure 3.4 shows the difference in the DFT and MUSIC spectrograms when extracting the Doppler frequency path.



(a) DFT spectrogram



(b) MUSIC spectrogram

Fig. 3.4: Comparison of spectrograms generated by the DFT or MUSIC algorithm.

MUSIC works well when the noise is white and zero-mean. When the noise does not fit these assumptions, the MUSIC algorithm gives noisy results. Often, getting a good MUSIC spectrum requires tuning the value of M and sometimes the value of P when it is uncertain how many signals there are.

3.3 Carrier Frequency Offset Removal

This section only applies to the Second DEM of (2.17) as it still has the carrier frequency offset applied to it. As mentioned in Chapter 2, the Second DEM creates a signal with two complex exponential components: one at the carrier frequency offset and one at the carrier frequency offset plus the Doppler shift. Transforming these into the frequency domain would result in two Dirac deltas if both frequencies did not vary with time.

These frequencies do vary with time and, therefore, taking the DFT would require windowing small chunks of the data over a small time step, and doing this over all time steps until a spectrogram is created. Due to limitations with the DFT discussed in Section 3.1 and how small the Doppler shift is, the real DFT plot would contain one large main lobe at the carrier frequency offset, which would cover the Doppler-shifted component (as seen in the spectrogram of Figure 3.5). This means it would be unreasonable to measure both the carrier frequency offset and the Doppler-shifted echo at the same time.

MUSIC would also not be able to separate the two signals because the magnitude of the direct path peak is significantly stronger than the Doppler-shifted peak. The solution is to modulate the signal to baseband and then remove the DC component. Removing the DC is required for both Doppler methods and is discussed in Section 3.4.

Modulating the signal to baseband by removing the carrier frequency offset was initially done in the PLL of the communication system. However, this method resulted in errors because it could not distinguish between the carrier frequency offset and the Doppler frequency.

An alternative method to the PLL is to extract the carrier frequency path from the signal and use this estimated path to demodulate the signal to baseband. This method is not as accurate as using a PLL, but it is able to mostly avoid interference from the Doppler

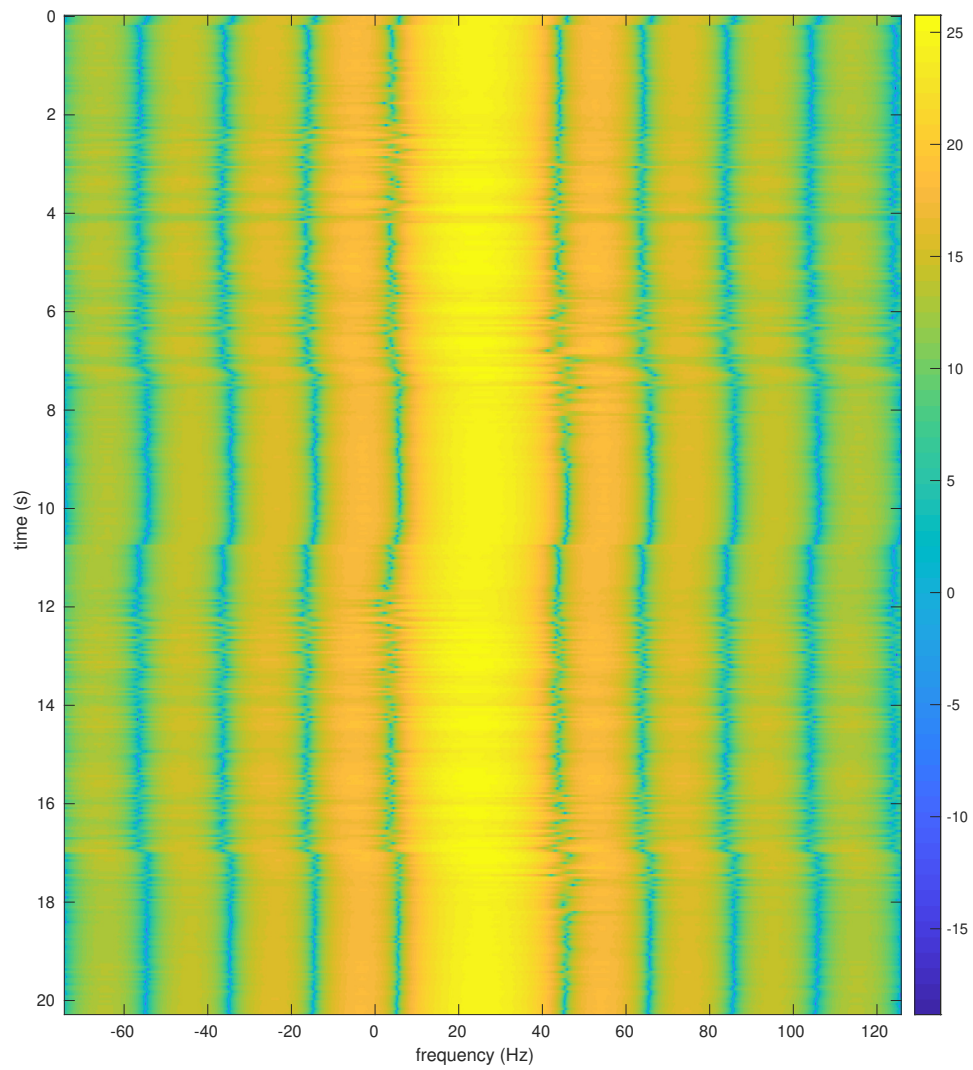


Fig. 3.5: The overwhelming strength of the carrier frequency offset component covers the Doppler frequency.

echo signal. The path extraction techniques mentioned here are discussed more in detail in Chapter 4.

The two main ways to estimate the path of the carrier frequency offset are either to obtain the maximum frequency component at each time step or to use a Viterbi algorithm to find the most likely path.

Obtaining the maximum frequency component at each time step is computed by finding the maximum value of the magnitude spectrum. The frequency of this maximum value is the carrier frequency offset over this window of data. With this frequency known, the window of data used for generating the spectrum can be modulated to baseband. Then it can be combined with the previously modulated data windows.

Finding the maximum works when the Doppler frequency component is too weak to interfere with the carrier frequency offset. However, this is not true for the entirety of most signals. The amplitude of the Doppler echo changes over the course of the signal, and there are instances when the Doppler echo will influence this maximum value. Figure 3.6 shows the Doppler frequency causing small changes in the main lobe of the carrier frequency offset.

These effects can pull the carrier frequency offset estimate from the true value, which would lead to errors in modulating the signal to baseband. The Viterbi algorithm is an alternative method capable of ignoring these slight perturbations. This algorithm is discussed in Section 4.1.

The Viterbi algorithm is more complex but more accurate in estimating the carrier frequency offset. However, once the signal is at baseband, there is still a DC component to remove.

3.4 DC Component Removal

Estimating the Doppler signal with limited data is difficult when there is a large DC component in the signal. Using either DEM in (2.14) or (2.17) (once the carrier frequency offset is removed) will result in an output having a large DC component, usually much larger than the Doppler frequency component.

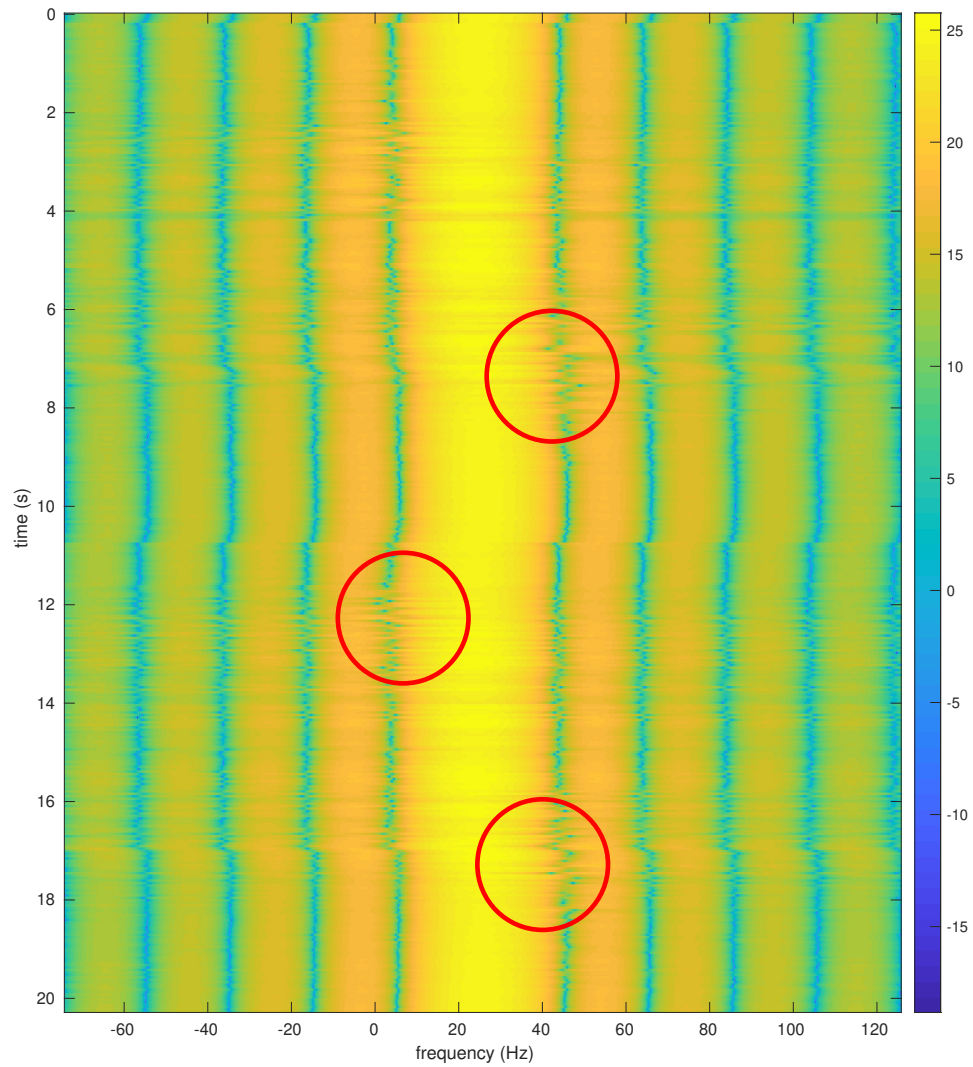


Fig. 3.6: The Doppler frequency causing changes to the main lobe of the carrier frequency offset path.

This component is a result of interference from the direct path signal. This interference, as mentioned in Section 1.1, is a limiting factor to the system. This component causes the Doppler frequency to be overpowered by spectral leakage when the DFT is calculated and causes problems in calculating the MUSIC algorithm, as the MUSIC algorithm expects the noise to have a mean of zero. Therefore, it is necessary to remove the DC component.

Removing the DC component turns out to be trivial as it can be accomplished by subtracting the mean of the signal. It is important to subtract this mean at the correct place, which differs for both DEMs. For the First DEM of (2.14), the mean can be removed either over the whole signal of length D ,

$$\text{DEM}_{1, \text{zero-mean}}(k) = \text{DEM}_1(k) - \frac{1}{D} \sum_{i=0}^{D-1} \text{DEM}_1(i), \quad (3.9)$$

or separately in each window of data used to generate the DFT or MUSIC algorithm,

$$\text{DEM}_{1, \text{zero-mean}}(k) = \text{DEM}_1(k) - \frac{1}{N} \sum_{i=n}^{N-1} \text{DEM}_1(i), \quad \text{for } n \leq k < N. \quad (3.10)$$

Typically, subtracting the DC in each window of data works better. For the Second DEM of (2.17), the best place to subtract the mean is right after modulating a piece of the signal to baseband but before combining the signals together,

$$\text{DEM}_{2, \text{zero-mean}}(k) = \text{DEM}_2(k)e^{j2\pi\hat{f}_c k} - \frac{1}{N} \sum_{i=n}^{N-1} \text{DEM}_2(i)e^{j2\pi\hat{f}_c i}, \quad \text{for } n \leq k < N. \quad (3.11)$$

CHAPTER 4

PATH EXTRACTION

The previous chapter dealt with generating a spectrogram from the extracted Doppler signal. This chapter discusses methods of extracting the best Doppler frequency path from the spectrogram. This chapter will represent the optimal path extracted by the path-finding algorithms as red asterisks on top of the spectrogram.

The simplest extraction method is the maximum value extraction. This method simply finds the maximum value of the spectrum at each time step and records the frequency at the maximum value. This list of recorded frequencies is the Doppler frequency path. This method is the same as the carrier frequency maximum extraction method described in Section 3.3.

The maximum value extraction method works in the ideal case where there is no noise or direct path interference on the signal or where both the SNR and SIR are large. However, the Doppler echo is a weak signal, which the noise or direct path interference can easily bury. This results in the estimated Doppler frequency path becoming noisy and unreliable. Figure 4.1 shows the maximum value extraction applied to a MUSIC spectrogram.

The maximum value extraction is not the best way to extract the Doppler path due to these limitations. The rest of this chapter is dedicated to exploring more reliable ways of extracting the Doppler path.

4.1 Viterbi Algorithm

The Viterbi algorithm is typically used in error correction coding to decode convolutional codes [19, pp. 471]. The basic premise of the algorithm is to find the best path through a system with a discrete number of states. Typically, a trellis diagram, like the one in Figure 4.2, is used to depict the operation of the algorithm.

This algorithm is able to find a connected path through the states of the trellis by

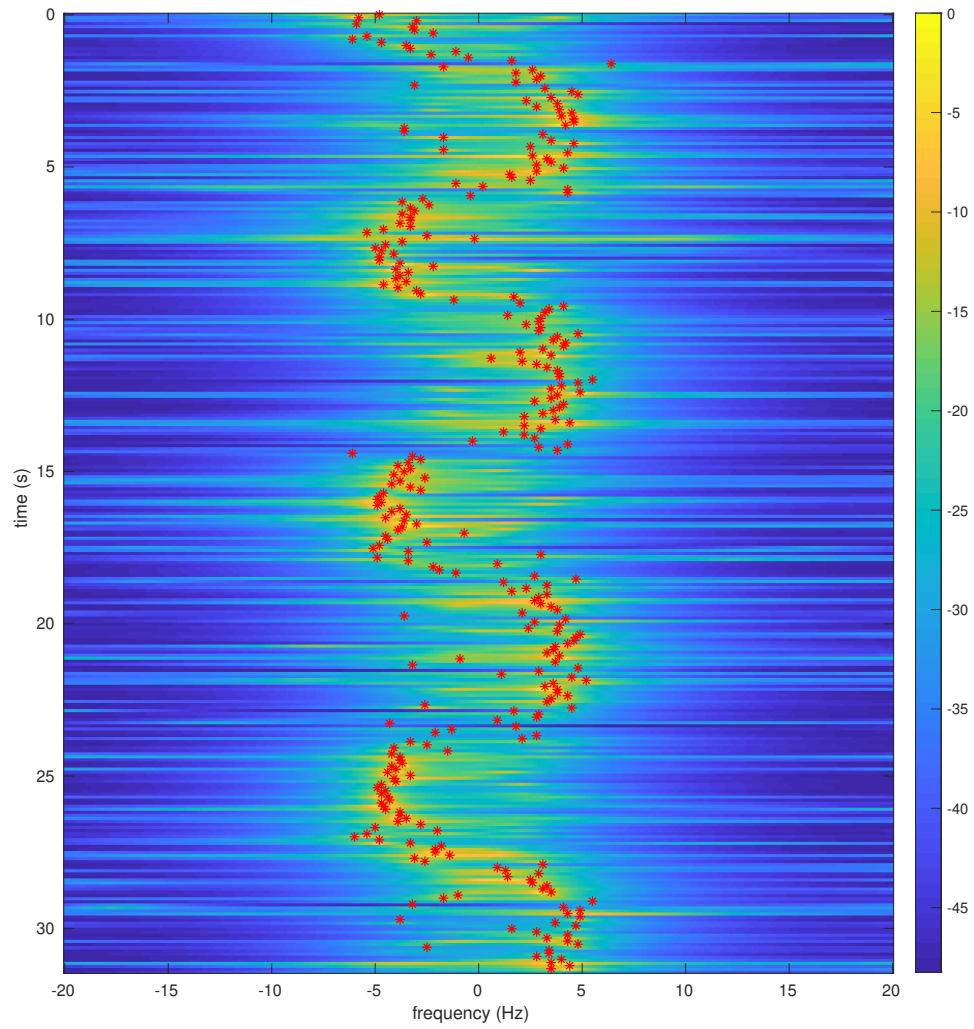


Fig. 4.1: Maximum value path extracted from a MUSIC-generated spectrogram.

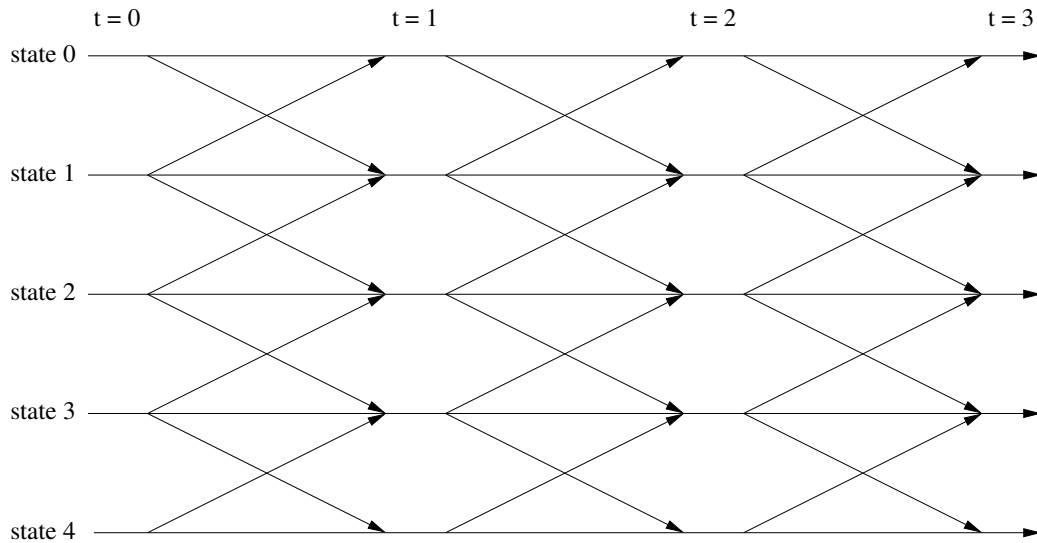


Fig. 4.2: Trellis diagram of a Viterbi algorithm.

putting limits on the state transitions. This makes it easier to ignore sudden jumps to far away states, which makes the system less noise prone. This algorithm follows the best path to each final state and then proceeds to select the best state overall as the most likely path through the trellis.

The Viterbi algorithm finds the best path through the trellis by finding the path with either the lowest path cost or the highest path value. The application determines whether the algorithm uses the lowest path cost or highest path value. For finding the best Doppler frequency path, the highest path value is used for optimizing the path.

To find the best path through the spectrogram, the algorithm uses the frequency bins of the spectrogram as the different states of the trellis. Each state is then limited in the number of states it can transition to. This is shown in the trellis of Figure 4.2 as a limited number of paths from one state to another state. In this case, each state in this trellis can only transition to either itself or one of the neighboring states.

For the frequency bins, this limit is based on how far the Doppler frequency might change from time step to time step. For example, in the human tracking application observations show that Doppler frequency does not change by more than 3 Hz over a 0.1 second period. Therefore, the trellis should not allow any of the states to transition to states more

than 3 Hz away. This limit in frequency shift defines the trellis structure. The limit of how far away in Hz a state may transition to is denoted by δ .

Once the structure is set, the algorithm then iteratively runs through the states of the trellis. The algorithm begins at $t = 0$ by initializing each state's path value. This initialized value is the magnitude of the spectrum at $t = 0$ for each frequency bin. Then the algorithm moves to $t = 1$.

At this next time step, a new path value is generated for each transition to another state. At each new state, the transition with the highest path value to the new state is kept while all other transitions are ignored. This new value becomes the updated score for this state, and the transition is recorded in a transition vector.

This method is then repeated for each additional time step after $t = 1$. At $t = n$ the path value is generated for each transition to state k and then the best value is kept and the corresponding transition is recorded. This means at any time there is a score for each state and a transition vector to each state describing the most optimal path to the state.

The path value for state k at time $t = n$, $M_n(k)$, is calculated using

$$M_n(k) = \max_j [M_{n,n-1}(k, j)], \quad (4.1)$$

where

$$M_{n,n-1}(k, j) = M_{n-1}(j) + v(k) - \mu(k, j) \quad (4.2)$$

refers to the path value of state j of the previous time step that is transitioning to state k of the current time step. Only the best of these transitions are kept.

The path value $M_{n,n-1}(k, j)$ depends on the previous path value of state j , $M_{n-1}(j)$, and the branch metric. The branch metric is composed of the magnitude of the frequency bin of state k ,

$$v(k) = \text{abs}(X[k = f/N]), \quad (4.3)$$

and a transition penalty,

$$\mu(k, j) = K|k - j|. \quad (4.4)$$

The transition penalty is an important parameter for limiting transitions to other frequencies. The further away the frequency the more this parameter lowers the path value. This parameter is controlled by a variable K .

The K value is a weight useful for adjusting the algorithm. The higher K is, the more expensive it is to change frequencies in the algorithm. This discourages the Viterbi algorithm from tracking large shifts in frequency due to a high spike of noise, but it will still track a change in Doppler.

In addition to K , δ is the other variable for controlling the maximum allowed frequency transition. This value controls how quickly the path is able to change, as mentioned previously. Together, these variables ensure the Viterbi algorithm is properly tuned to avoid noise. Figure 4.3 shows the Viterbi algorithm applied to the same spectrogram as in Figure 4.1. The path in Figure 4.3 is less noisy than the path in the maximum value case. This is due to the smoothing ability of the Viterbi algorithm. However, the Viterbi algorithm can smooth the data too much if incorrectly adjusted.

4.2 BCJR Algorithm

The Viterbi algorithm improves the Doppler path by using previous data to better predict more recent data. However, it only uses past and present data at each time step to determine the most likely path. Using future data of the state of the system may improve the results. BCJR is an algorithm capable of determining the best path using past, present, and future data.

BCJR generates a matrix of probabilities by assigning a probability to each state at every time step based on past, present, and future information. This limits the real-time application of this algorithm as it requires future data. However, this algorithm can be implemented in real-time if the application allows some latency in extracting the Doppler path. BCJR comes from the field of error correction coding where it is used in decoding turbo codes [19, pp. 588].

The equations for BCJR are listed as (4.5) – (4.7) [19, pp. 596]. In these equations, p refers to the state at time t , while q refers to the state at time $t + 1$.

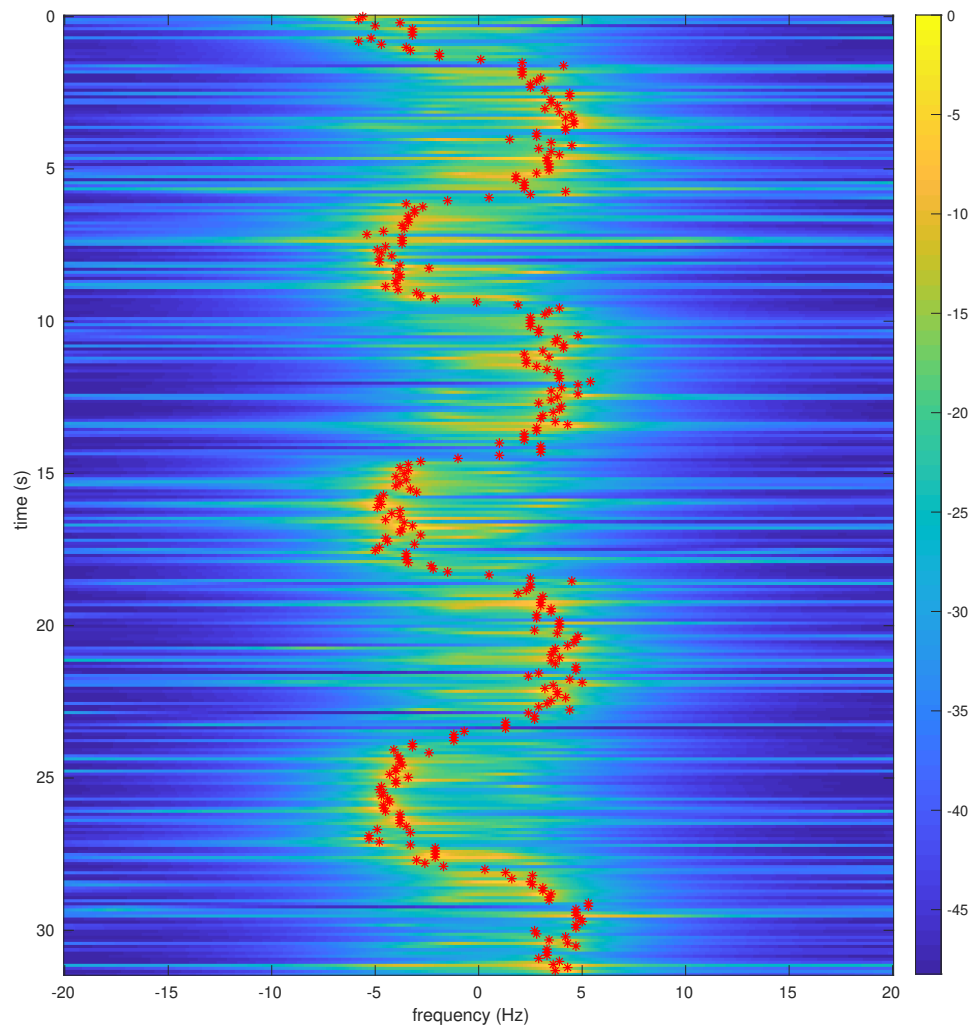


Fig. 4.3: Path extraction using the Viterbi algorithm through a MUSIC generated spectrogram.

$$\alpha_{t+1}(q) = N_q \sum_{p=0}^{Q-1} \alpha_t(p) \gamma_t(p, q) \quad (4.5)$$

$$\beta_t(p) = N_p \sum_{q=0}^{Q-1} \gamma_t(p, q) \beta_{t+1}(q) \quad (4.6)$$

$$P(x|\mathbf{r}) = \alpha_t(p) \sum_{q=0}^{Q-1} \gamma_t(p, q) \beta_{t+1}(q) \quad (4.7)$$

Equation (4.5) is referred to as the forward pass equation. This recursive equation analyzes how the previous states and current state affect future states. The $\alpha_0(p)$ values are initialized to $1/Q$ where Q is the number of states (it is equally likely to start in any of the states). Equation (4.6) is referred to as the backward pass equation. This equation analyzes how present and future states affect the past states. The $\beta_{N-1}(q)$ values, with N being the total number of time steps, are initialized to $1/Q$ (it is equally likely to end in any state). N_q and N_p are normalization constants and are defined as

$$N_q = \frac{1}{\sum_{q=0}^{Q-1} \sum_{p=0}^{Q-1} \alpha_t(p) \gamma_t(p, q)} \quad (4.8)$$

$$N_p = \frac{1}{\sum_{p=0}^{Q-1} \sum_{q=0}^{Q-1} \gamma_t(p, q) \beta_t + 1(q)}. \quad (4.9)$$

Equation (4.7) takes the results from the forward and backward passes to calculate the probability of the state x given all of the observations $\mathbf{r}$.

$\gamma_t(p, q)$ describes the transition between states. This value takes into account the magnitude of the frequency spectrum, $v(f)$, and the shift in frequency from state p to q . For the purpose of extracting the Doppler signal, $\gamma_t(p, q)$ is

$$\gamma_t(p, q) = \begin{cases} e^{K_2 v(p)} e^{-K_1 |p-q|}, & \text{if } |p-q| < \delta \\ 0, & \text{otherwise.} \end{cases} \quad (4.10)$$

This equation has three parameters for adjusting the algorithms, K_1 , K_2 , and δ . K_1 is the

weight given to the branch metric, which is the same as the branch metric in the Viterbi algorithm. K_2 is the weight given to the magnitude of the frequency spectrum for the current state. δ limits how many states a single state can transition to, which is the same as δ in the Viterbi algorithm.

As mentioned, the BCJR algorithm outputs a matrix of probabilities the same size as the input spectrogram matrix. The maximum value path extraction method can extract the best frequency path, but it is typically better to use the Viterbi algorithm.

The Viterbi algorithm is preferred for finding a path in the BCJR probability matrix for the same reasons it is preferred to find a path in the spectrograms: it finds a more continuous path. The BCJR algorithm is not able to smooth the path as much as the Viterbi algorithm does, as it does not look for a path. However, the BCJR algorithm does do a good job improving the values for the Viterbi algorithm to work on.

Figure 4.4 shows the results of using the BCJR algorithm with the Viterbi algorithm. This figure displays the path on top of a color map of the probability matrix generated by BCJR from the MUSIC spectrum of Figure 4.1.

4.3 Balance Factor

The Viterbi and BCJR algorithms are able to improve the path extraction in most cases. However, they fail when the Doppler frequency, $f_d(t)$, is at 0 Hz. When the Doppler frequency is at this DC term, the path extraction is unable to find it.

This occurrence is not uncommon, as it occurs anytime there is no movement or when a moving target moves directly between the transmitter and receiver, as indicated by the bistatic radar equation of (1.10). The derivative terms in (1.10) cancel out when they are equal in magnitude but opposite in sign. This occurs anytime the target is moving directly towards the receiver and directly away from the transmitter or directly away from the transmitter and directly towards the receiver. At this moment, $f_d(t) = 0$, the same as the direct path signal. The processing required to remove the direct path signal (DC removal of Section 3.4) also removes the Doppler frequency in this case.

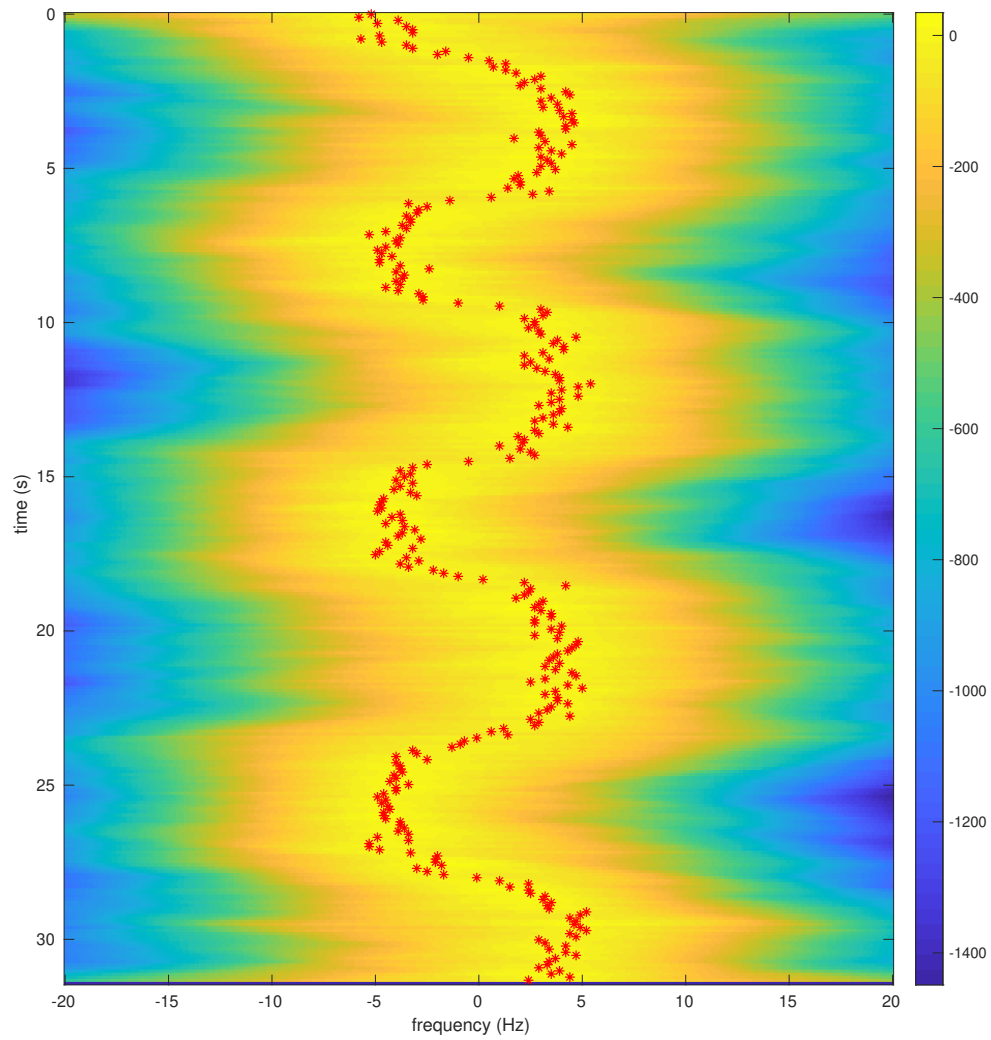


Fig. 4.4: Path extraction using a Viterbi algorithm on a BCJR probability matrix formed using a MUSIC generated spectrogram.

If $f_d(t)$ merely passes through 0 Hz, then the path extraction algorithms are able to easily correct for the lost data. However, when $f_d(t)$ stays at 0 Hz for an extended period of time, the algorithms will try to find a path in the noise. This leads to erroneous Doppler frequency data.

A solution to this is to introduce the idea of a balance factor. A balance factor is a ratio describing how the energy in a spectrum is balanced between the positive and negative frequency bins. It is calculated simply by computing the sum of the magnitudes of all the positive frequencies of the spectrum and then subtracting the sum of the magnitudes of all the negative frequencies. Then the result is normalized by the sum of the magnitude of all frequencies.

$$BF(t) = \frac{\sum_0^{\max f} |v_t(f)| - \sum_{\min f}^0 |v_t(f)|}{\sum_{\min f}^{\max f} |v_t(f)|} \quad (4.11)$$

The resulting balance factor is between -1 and 1 . This value shows how balanced the spectrum is. When calculated for each spectrum in the spectrogram, the result is a vector of these balance factors. Plotting these factors and scaling them shows how they follow the Doppler frequency to either side and hover around 0 when the Doppler frequency is around 0 Hz, as shown in Figure 4.5.

The balance factor contains the information needed to make sure the path extraction knows when the Doppler is at or near 0 Hz. The Viterbi and BCJR algorithms can use this factor to influence their decisions. The easiest way to implement this factor in these algorithms is to force the path to 0 Hz anytime $|BF(t)| < \epsilon$, where ϵ is a threshold parameter. This approach is error prone, as noise heavily influences the balance factor. Therefore, if the noise pushes the balance factor below ϵ while the Doppler is actually at 10 Hz, the results are a sudden jump to 0 Hz and then back to 10 Hz.

A better approach is to pull the spectrum to 0 Hz when $|BF(t)| < \epsilon$. This was implemented in the BCJR algorithm by adding an additional term to $\gamma(t)$ in (4.10), although a similar adjustment could also have been made to the Viterbi algorithm in (4.2). This

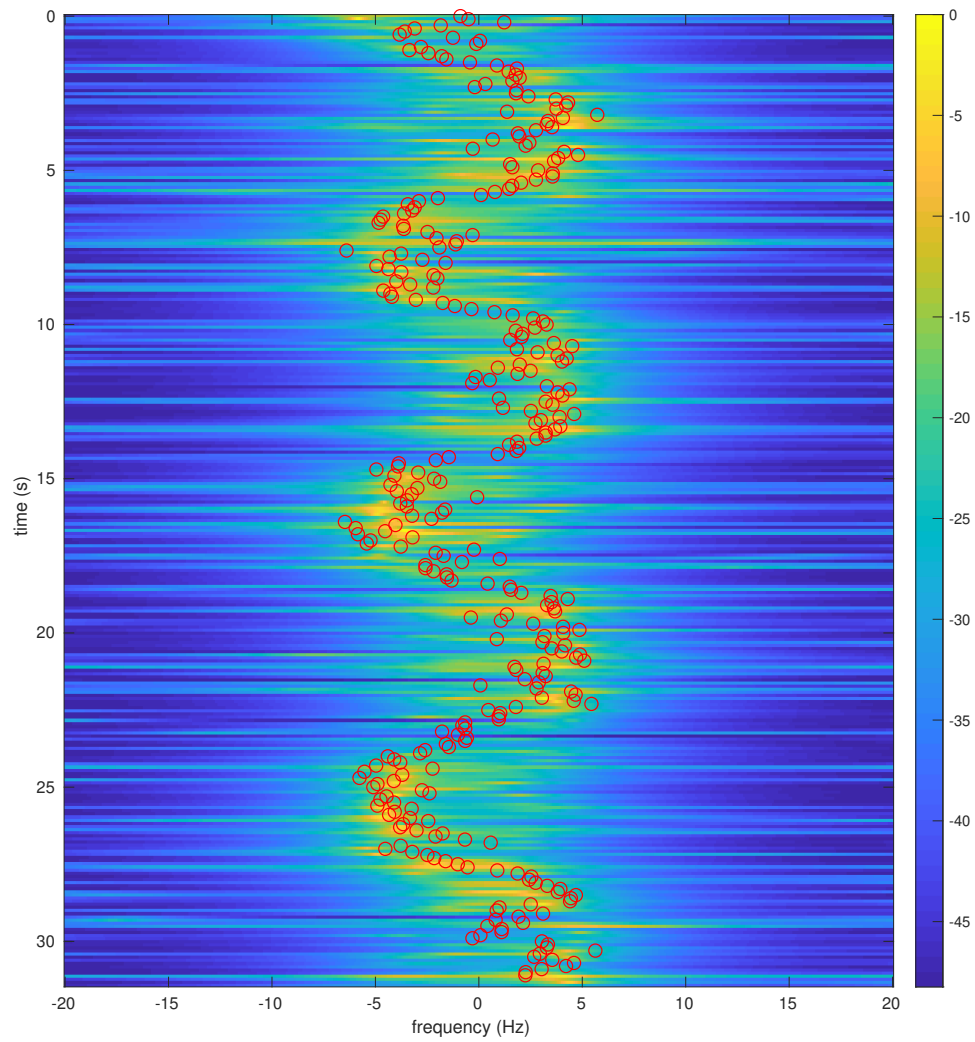


Fig. 4.5: A scaled balance factor vector plotted on top of the MUSIC spectrum the balance factor represents.

adjustment results in

$$\gamma_t(p, q) = \begin{cases} e^{K_2 v(p)} e^{-K_1 |p-q|}, & \text{if } |p-q| < \delta \text{ and } |BF(t)| > \epsilon \\ e^{K_2 (\tilde{B}F(t)v(p) + (1-\tilde{B}F(t))v_0(p))} e^{-K_1 |p-q|}, & \text{if } |p-q| < \delta \text{ and } |BF(t)| \leq \epsilon \\ 0, & \text{otherwise,} \end{cases} \quad (4.12)$$

where

$$\tilde{B}F(t) = K_0 |BF(t)|$$

and

$$v_0(p) = \begin{cases} \Lambda - |K_3 p|, & \text{if } |p| < \sigma \\ 0, & \text{otherwise.} \end{cases}$$

This adjustment uses the $BF(t)$ term as a weight to adjust the shape of the magnitude spectrum when $BF(t)$ is below the threshold value ϵ . This adjustment is a balance between the magnitude spectrum and an adjustment spectrum, such as the one depicted in Figure 4.6. This adjustment spectrum gives more weight to the lower frequencies when the balance factor is closer to 0.

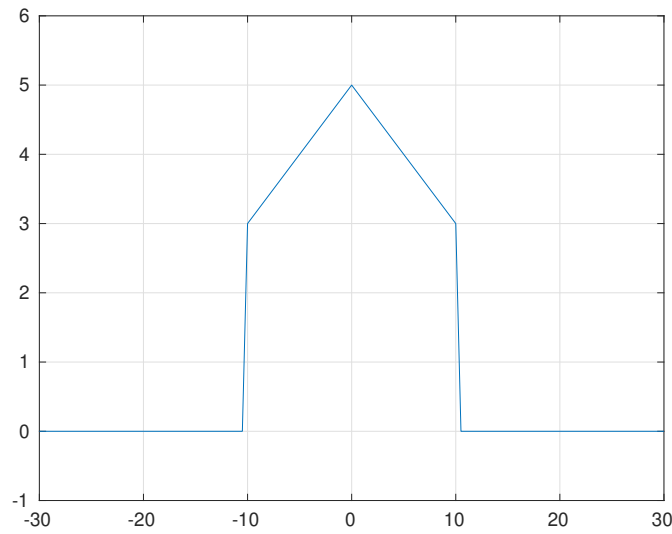


Fig. 4.6: Adjustment frequency spectrum used to pull the Doppler frequency closer to 0 Hz.

This new equation adds several new parameters. K_0 is a parameter used to control the weight given to $BF(t)$. This weight is necessary, as, in practice, $BF(t)$ is generally small and can give too much weight to the adjustment spectrum. The parameters Λ , ϵ , and K_3 all control the shape of the adjustment spectrum, $v_0(p)$: Λ controls the height of the spectrum, ϵ controls the width of the spectrum, and K_3 controls the rate at which the magnitude decreases linearly away from the maximum at 0 Hz. This allows the spectrum to either form a triangle or an irregular pentagon.

Applying this adjustment allows the spectrum to account for Doppler at 0 Hz. Figure 4.7 shows the BCJR probability plot and optimal path when trying to identify the optimal path from a spectrogram with portions of the signal at 0 Hz. This figure demonstrates that the optimal path is not able to sufficiently transition from side to side with the Doppler frequency. Figure 4.8 shows the results after applying the balance factor adjustments to the algorithm. The adjustments allow the BCJR to give higher probabilities to frequencies closer to 0 Hz, as seen around the 10-second mark in Figure 4.8. This also makes it less likely to be on the wrong side as the balance factor makes transitions from side to side more probable.

4.4 Correcting Symmetry

This section applies only to the First DEM of (2.14). Section 2.1.1 discussed how this method breaks down in the carrier phase PLL, as the PLL tries to track the Doppler frequency and remove it. This results in the generation of a new frequency term at the negative Doppler frequency (shown in (2.15)). This symmetric frequency problem renders the First DEM useless when the carrier phase PLL is used unless additional processing is done to recover the sign of the Doppler frequency. Therefore, a method to extract the sign of the Doppler frequency was devised. This method takes advantage of the rotations of the symbols around the constellation points to determine the direction of rotation. Figure 4.9 shows a diagram of how the Doppler frequency modifies the signal constellation by rotating the symbols.

There are two important parameters to keep track of in relation to these rotations: the

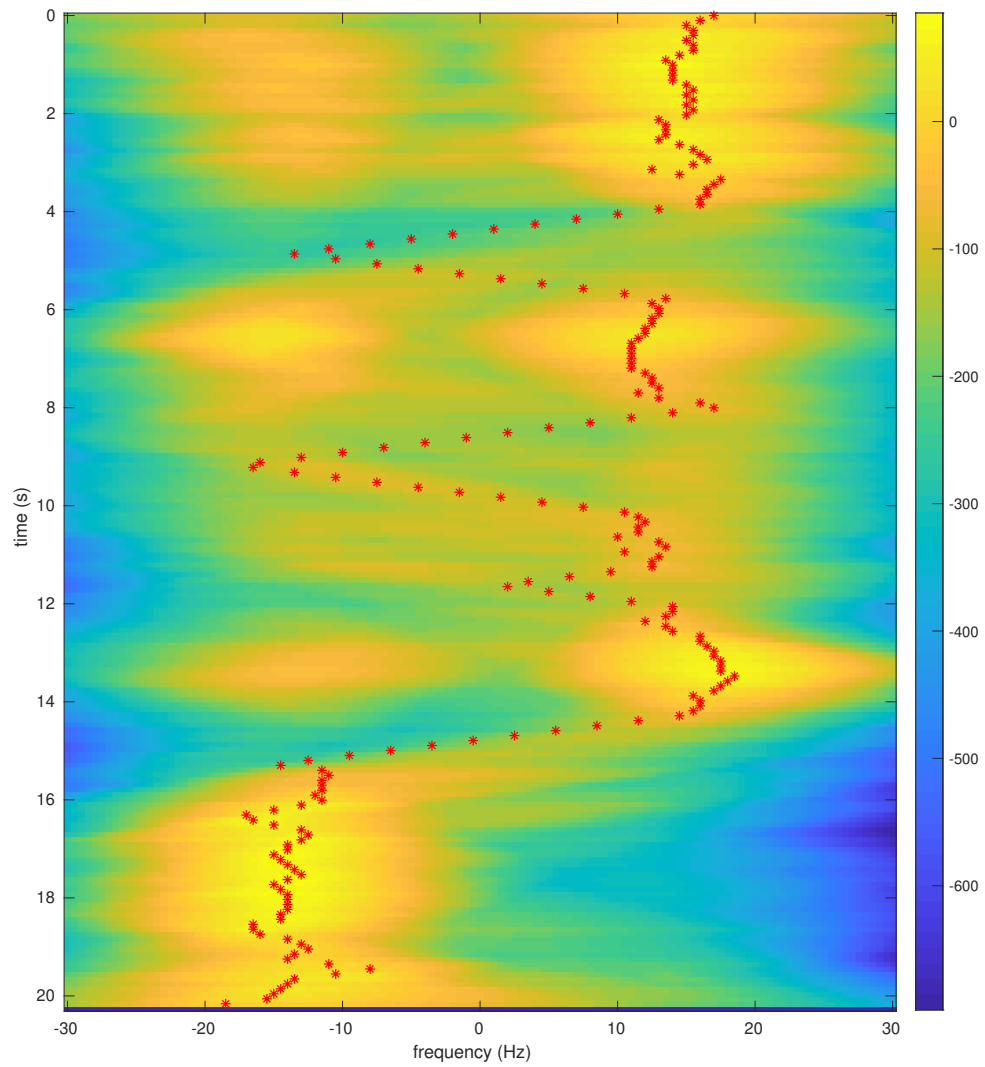


Fig. 4.7: Trying to apply the BCJR algorithm to a spectrogram without the balance factor adjustment when there are portions of the signal at 0 Hz.

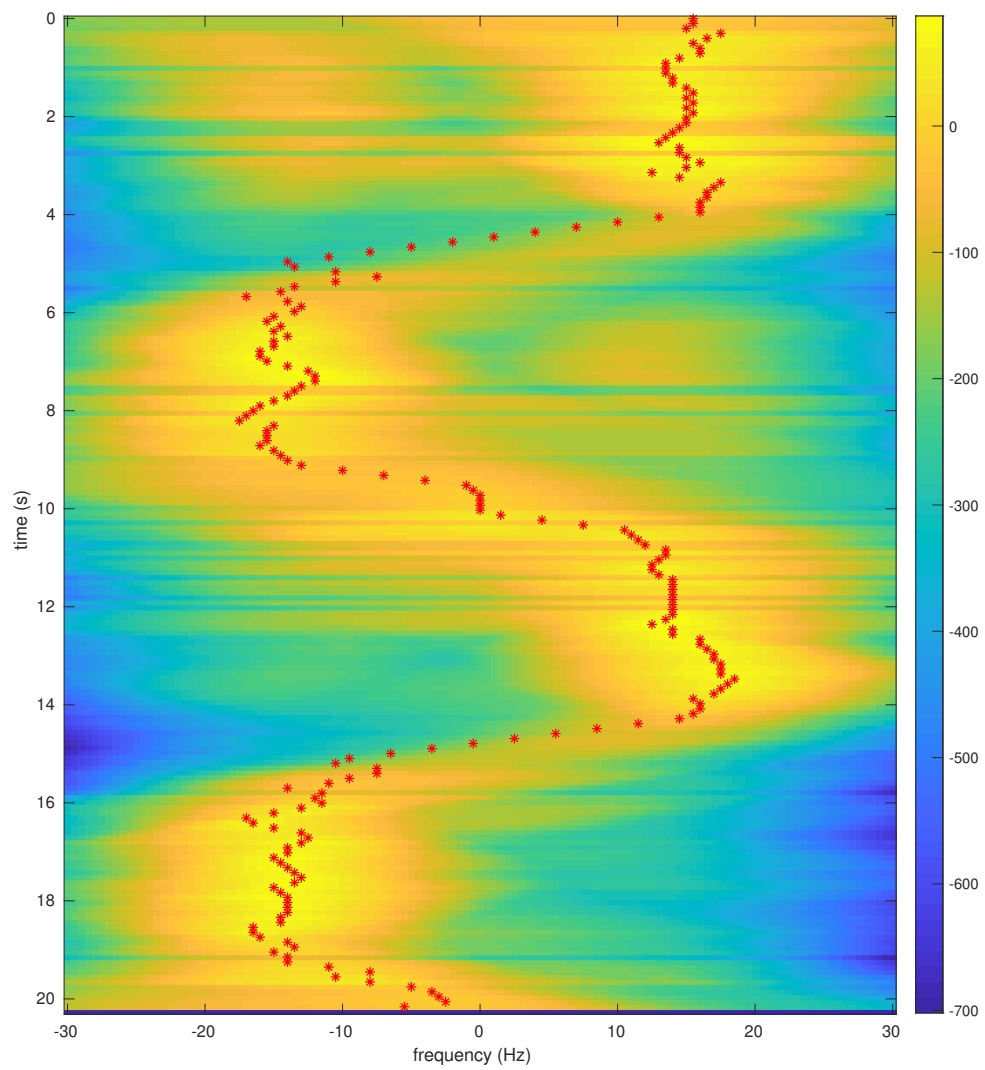


Fig. 4.8: Trying to apply the BCJR algorithm to a spectrogram with the balance factor adjustment when there are portions of the signal at 0 Hz.

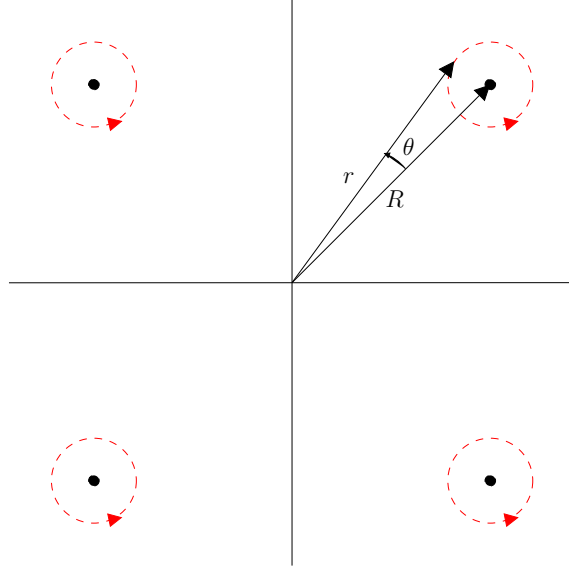


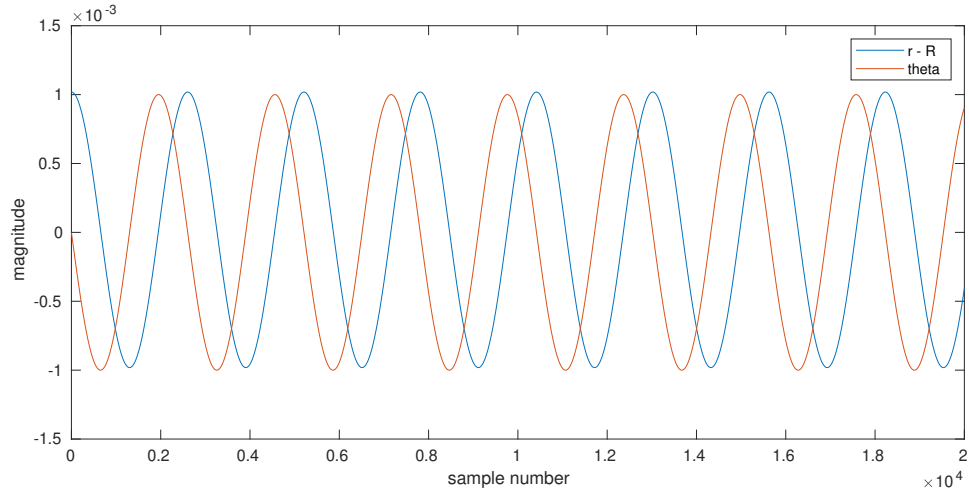
Fig. 4.9: Diagram of how Doppler echo symbols rotate on a non-rotating signal constellation.

angle of error, θ , and the radius of error, r . These two quantities define how the Doppler frequency modifies the signal point. If the symbols are all a single constellation point, a plot of θ compared to the value of $r - R$, with R being the average radius, would result in the plots of Figure 4.10. If the symbols are from multiple constellation points the plots would be the same as long as θ is measured in comparison to the corresponding constellation point, which would be obtained from the estimated symbol data.

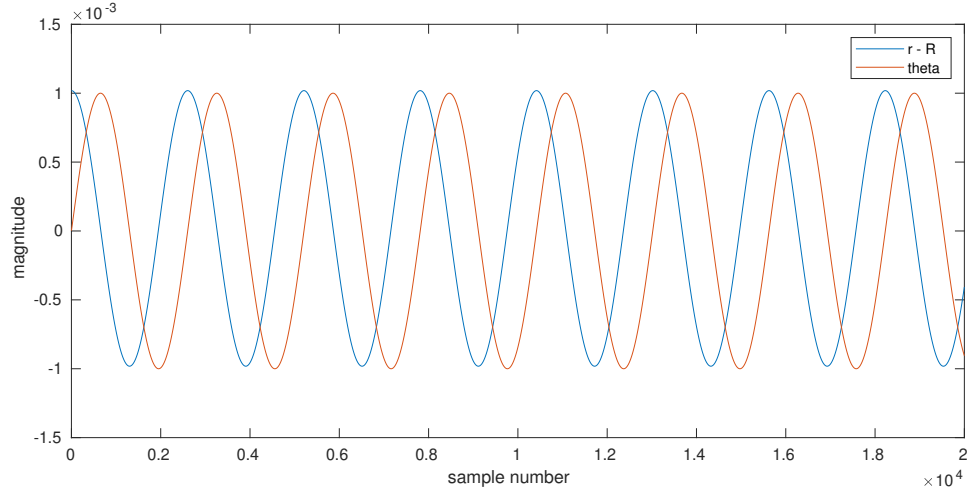
Both $r - R$ and θ move in a sinusoidal manner, approximately 90° apart from each other. When the Doppler frequency is positive, θ leads $r - R$ by 90° . When the Doppler frequency is negative, θ lags $r - R$ by 90° . This angle of separation is a function of the amplitude of the Doppler component of the receiver signal, but as the amplitude of the Doppler component approaches 0, the angle approaches 90° . Since the amplitude of the Doppler frequency is small, this is a good approximation for the system.

With this information, the sign is extracted by sending θ through a 90° delay filter and through a 90° advance filter then correlating the output of each filter with $r - R$. If the delay filter correlation is higher than the advance filter, then the sign is positive. If the advance filter correlation is higher, then the sign is negative.

Figure 4.11 shows the case where there is a carrier frequency offset on the signal of



(a) Positive Doppler



(b) Negative Doppler

Fig. 4.10: Comparison of $r - R$ and θ for both the positive and negative Doppler cases.

interest, which is the case for the constellation of $s(k)$, the signal coming out of the symbol timing synchronization block. In this case, $r - R$ is the same value, but θ now continuously increases or decreases as the entire constellation rotates. The Doppler echo symbols rotate as helices.

In this case, trying to correlate will not work very well, since θ now increases or decreases linearly as it oscillates, as shown in Figure 4.12. It is harder to distinguish θ lagging and leading $r - R$. Trying to apply the correlation does not work because of how different the

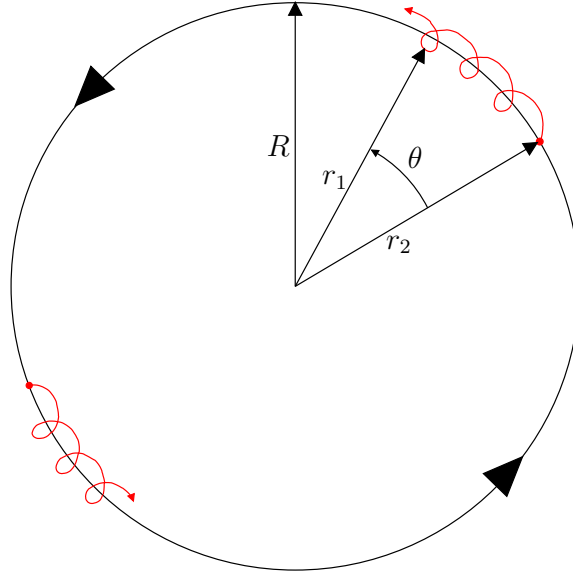


Fig. 4.11: Diagram of how Doppler frequencies rotate on a rotating signal constellation.

two values are. However, if the carrier frequency offset is removed from the signal, then θ will not increase or decrease linearly.

Removing the carrier frequency offset can either be done using the method described in Section 3.3 or by fitting a line to the plot of θ and subtracting the line. Once θ is no longer increasing or decreasing, then the correlation described previously in this section will work.

This correlation information can now correct the symmetry problem. Figure 4.13 shows a symmetric DFT spectrum of the First DEM with a Viterbi algorithm applied to only the negative frequency bins. This allows the algorithm to only follow the magnitude of the frequencies. The sign is corrected for by multiplying each frequency by either a 1 or a -1 depending on which of the correlation values are higher. This results in Figure 4.14.

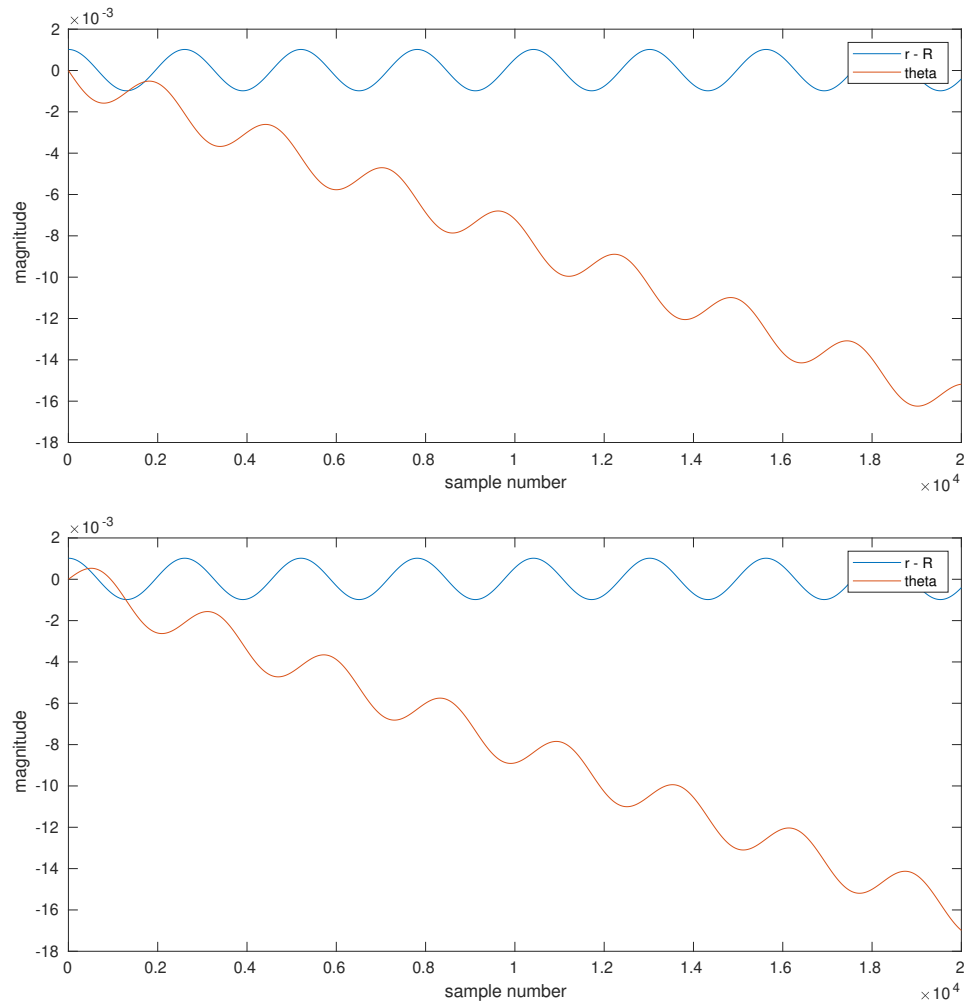


Fig. 4.12: Comparison of $r - R$ and θ with a carrier phase offset, for both the positive and negative Doppler cases.

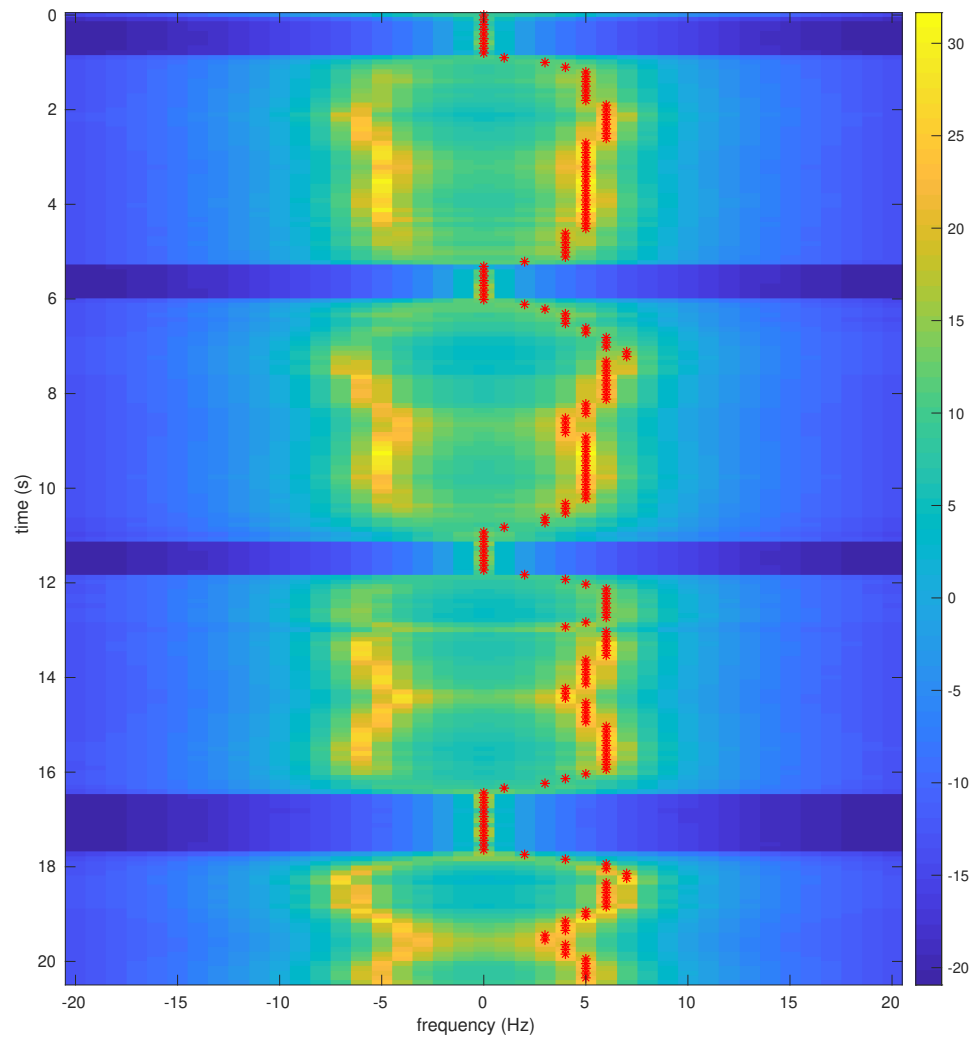


Fig. 4.13: A symmetric MUSIC spectrum of the Doppler echo signal.

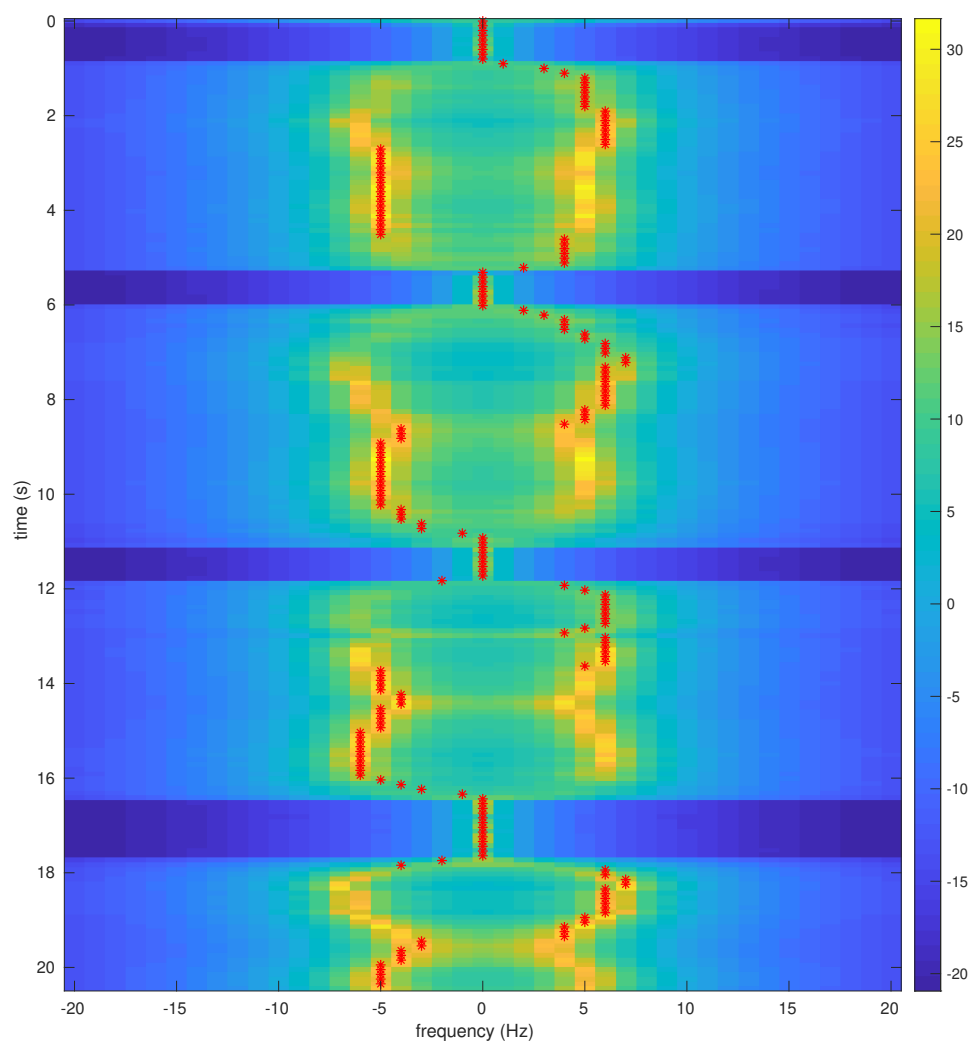


Fig. 4.14: Result of applying symmetry correction.

CHAPTER 5

RESULTS

This chapter describes the results of extracting the Doppler frequency path using the system of Figure 1.3. This chapter will go over the various test setups and the produced spectrograms and Doppler frequency paths. The parameters for generating the figures in this chapter are all recorded in Appendix A. Other parameters related to how the data was collected are mentioned in each specific test set section.

5.1 Simulation Results

The Doppler system simulation was done in GNU Radio. The same flowchart used in the full system, as described in Chapter 2, was used for the simulation, with one difference. Instead of the flowchart sending data into a transmitter block and then receiving the data using the receiver block, the transmitted data is sent through a simulated channel (Figure 5.1).

This simulated channel uses a Channel Model block, which models the noise, frequency offset, and timing offset of a typical channel. In addition, the simulated channel also has a Multiply Constant block to model a phase offset. Finally, the channel models the Doppler

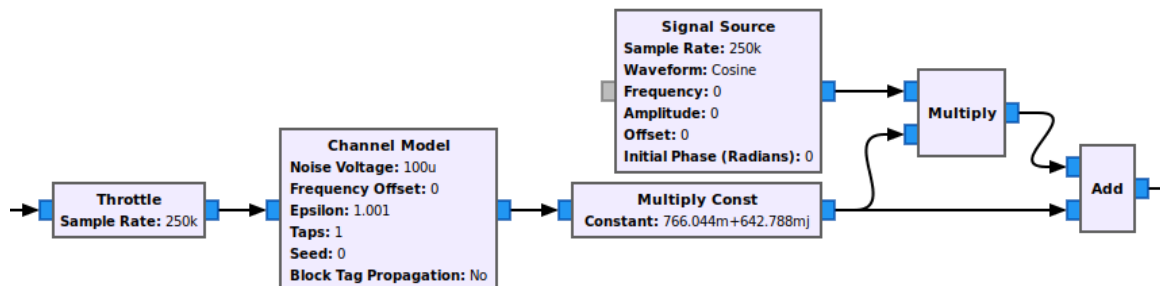


Fig. 5.1: GNU Radio simulated channel with a simulated Doppler echo.

echo by modulating the signal using a signal source and then adding the resulting signal to the original signal.

This simulated channel allowed the First DEM to be tested and led to the discovery of the double frequency problem and an understanding of the limitations of the system, such as the power of the DC component compared to the power of the Doppler echo component.

Figure 5.2 shows the simulation of the double frequency problem discussed in Section 2.1.1. The two symmetric peaks in this simulation are exaggerated in height in order to see

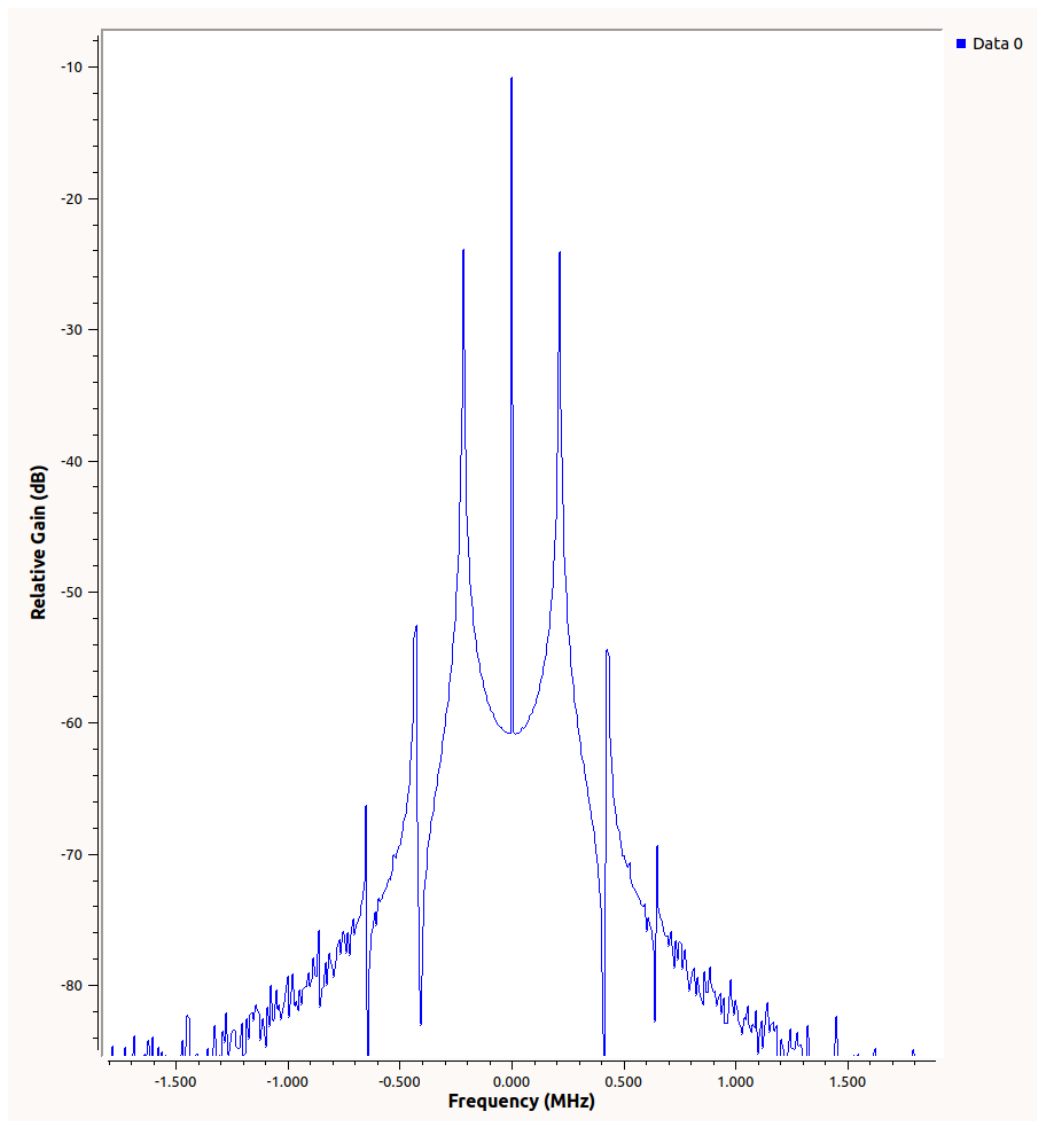


Fig. 5.2: Simulation of the double frequency issue caused by the carrier phase PLL.

the issue more clearly. The two peaks would be closer to the noise floor in a real situation.

This simulation shows how hard it is to differentiate the false peak from the real peak of the Doppler frequency. The simulation also reveals that the system creates harmonics of the Doppler frequency with lower amplitudes. These harmonics do not interfere with the Doppler extraction, but it does mean the PLL creates additional noise as it tries to track the Doppler.

5.2 Implementation Results

The Doppler extraction system was tested over a period of seven months. These tests all used the communication system described in Chapter 2, with various different configurations of transmitters, receivers, antenna placement, and number of amplifiers. The GNU Radio parameters also varied from test set to test set.

The following sections describe the setup for collecting the data for each test set. Various Doppler extraction plots are also provided to show the path the MATLAB code was able to extract from the data, with the parameters for these plots being recorded in Appendix A.

Each test set varied different parameters of the setup. Therefore, the corresponding section for each set of tests records the specific parameters of the test set. Each section describes the placement of the antennas and contains four tables. The first table describes the device parameters:

- Transmitter information
- Transmitter gain in dB
- Receiver information
- Receiver gain in dB
- Number of amplifiers used
- Sample rate of the communication system

- Interpolation and decimation rate
- Carrier frequency
- Master clock rate of the device, if configurable

The second table describes the symbol timing synchronization and pulse shape information:

- Symbol rate
- Excess bandwidth
- Matched filter length
- Timing error detector (TED)
- Expected TED gain
- Loop bandwidth
- Damping factor
- Maximum deviation
- Interpolating resampler

The third table contains the information for the carrier phase synchronization:

- $B_n T_s$, the loop bandwidth
- ζ , the damping factor
- K_0
- K_p

The values in the third table are used to calculate the K_1 and K_2 values using [15, pp. 684]

$$K_1 = \frac{1}{K_0 K_p} \frac{\frac{4\zeta}{N} \left(\frac{B_n T_s}{\zeta + \frac{1}{4\zeta}} \right)}{1 + \frac{2\zeta}{N} \left(\frac{B_n T_s}{\zeta + \frac{1}{4\zeta}} \right) + \left(\frac{B_n T_s}{N(\zeta + \frac{1}{4\zeta})} \right)^2} \quad (5.1)$$

$$K_2 = \frac{1}{K_0 K_p} \frac{\frac{4}{N^2} \left(\frac{B_n T_s}{\zeta + \frac{1}{4\zeta}} \right)}{1 + \frac{2\zeta}{N} \left(\frac{B_n T_s}{\zeta + \frac{1}{4\zeta}} \right) + \left(\frac{B_n T_s}{N(\zeta + \frac{1}{4\zeta})} \right)^2}. \quad (5.2)$$

The final table describes the different tests conducted in the particular test set. Each test in a test set differs in length and in the path of the moving target. Some tests had the moving target carrying a sheet of aluminum to improve the strength of the Doppler echo. Not all tests taken are provided as duplicate tests are not listed in these tables.

5.2.1 June 26, 2020

The first test set collected used two Heyrtz retractable antennas, one for transmitting and the other for receiving. These antennas were affixed on top of a cubicle wall and positioned a foot apart. They were both connected to a single E320 device, which acted as a transmitter and a receiver. The tests consisted of the moving target suddenly moving a short distance towards the array. The setup for these tests is recorded in Tables 5.1 – 5.3 and the tests are recorded in Table 5.4.

This first test set was an initial inquiry into how strong the Doppler echo would be. Most of the tests conducted were short and were mainly focused on a sudden movement forward. Other tests were conducted but were less useful than these. As these tests were some of the first conducted, the tests only collected data from the First DEM.

Table 5.1: Device Parameters, June 26, 2020

Transmitter	E320 #41, RF A, Tx/Rx
Transmitter gain	89.75 dB
Receiver	E320 #41, RF B, Tx/Rx
Receiver gain	36 dB
Number of amplifiers	0
Sample rate	31250 Samples/s
Interpolation/Decimation rate	1
Carrier frequency	905 MHz
Master clock rate	500 kHz

Table 5.2: Symbol Timing Synchronization Parameters, June 26, 2020

Samples per symbol	4
Excess bandwidth	0.5
Matched filter length	161
TED	$y[n]y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.1
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.3: Carrier phase synchronization parameters, June 26, 2020

$B_n T_s$	90μ
ζ	0.707
K_p	0.5
K_0	1

The PLL problem did not affect this data because there was no need to perform carrier phase synchronization. Only one device was used for both transmitting and receiving data, which means the same clock was used for modulating and demodulating the communication signal. The $B_n T_s$ value was turned down to only adjust for a phase offset and not a frequency offset, which means the Doppler frequency was filtered out by the loop filter.

Figures 5.3 – 5.5 show the resulting DFT spectrograms with a path extracted by the

Table 5.4: Tests, June 26, 2020

Test	Metal	Description
1	Yes	Moving towards the antennas at an angle perpendicular to the antennas, slower pace
2	Yes	Moving towards the antennas at an angle perpendicular to the antennas, normal pace
3	Yes	Moving towards the antennas at an angle perpendicular to the antennas, faster pace
4	No	Moving towards the antennas at an angle perpendicular to the antennas, slower pace
5	No	Moving towards the antennas at an angle perpendicular to the antennas, normal pace
6	No	Moving towards the antennas at an angle perpendicular to the antennas, faster pace

Viterbi algorithm for tests 1, 5, and 6. These tests had the best results as they all showed a perturbation of the Doppler frequencies due to the sudden movement forward. Test 1 was the only one of these tests to use the sheet metal to improve the reflection strength.

An important observation over all of these spectrograms is the two bands of energy on either side of the DC value. These energy bands are remnants of the strong DC component. The DC was removed here by taking a data frame, estimating the mean of the data frame, and then subtracting the mean from the data frame before computing the spectrum for the frame. Then, the DC for the spectrum is scaled up to the mean of the spectrum. This results in the spectrogram having better contrast.

The energy bands are not as strong as the Doppler, but they do make it hard to track the Doppler path when the magnitude of the Doppler frequency drops below 4 Hz. This makes subtle changes in Doppler at lower frequencies harder to track.

It is important to note how spread out the Doppler is. This is a result of the DFT, as the Doppler frequency could change by 1 – 2 Hz over the data frame length, which is 0.2 seconds. This means the resolution of the spectrum is lower, as mentioned in Section [3.1](#).

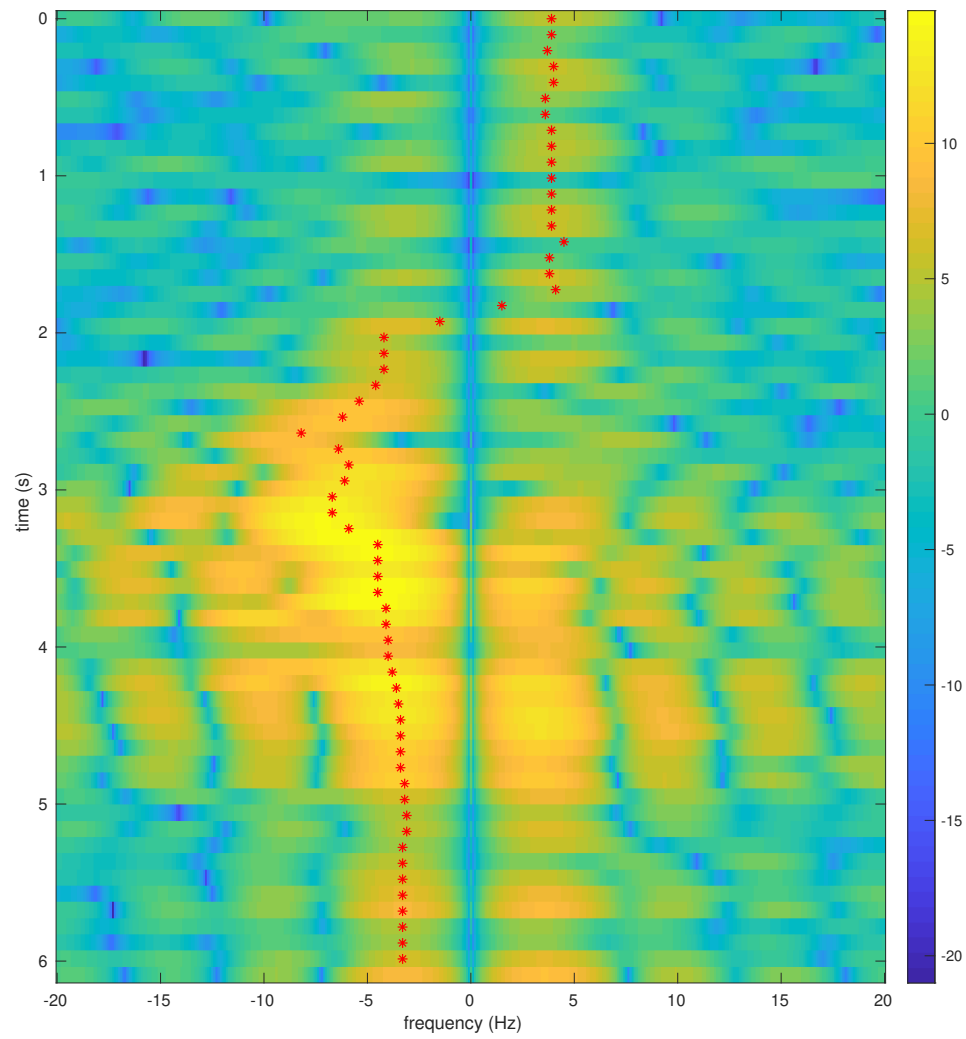


Fig. 5.3: Test 1 extracted path over a DFT spectrogram.

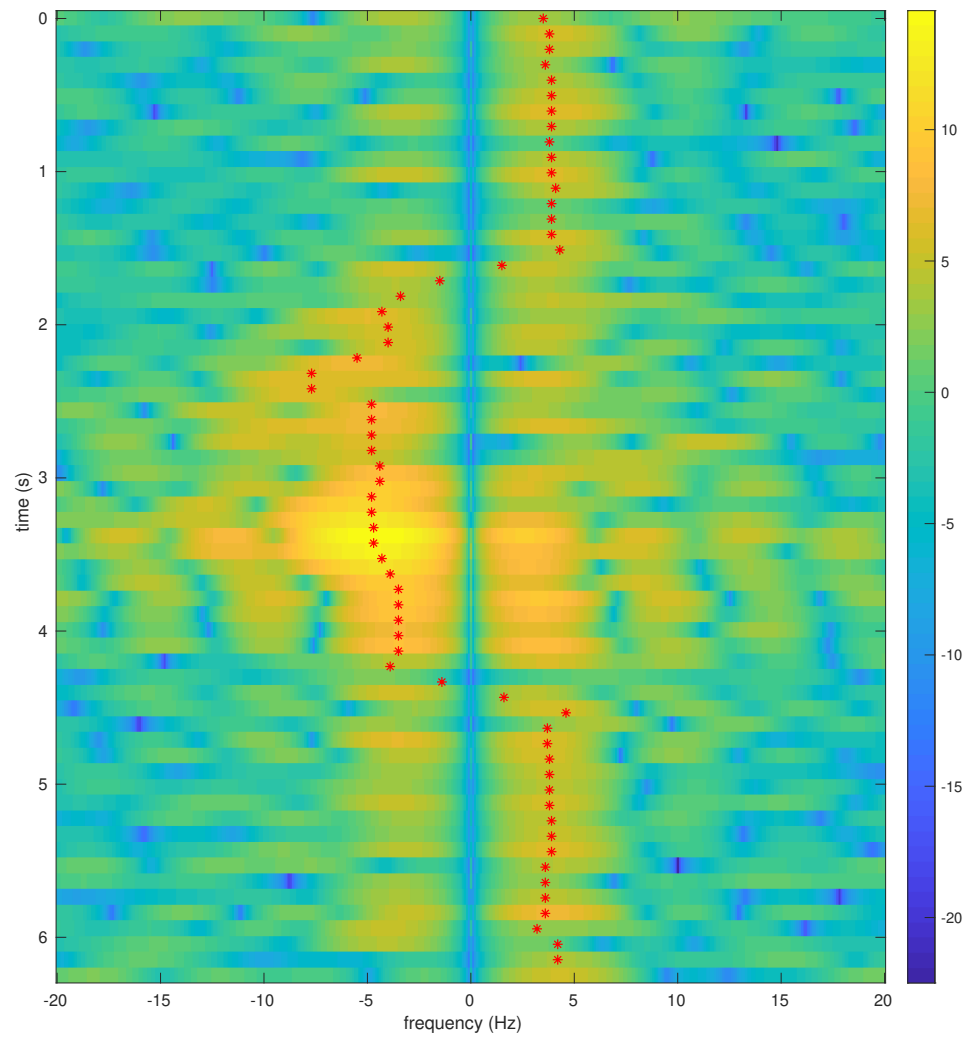


Fig. 5.4: Test 5 extracted path over a DFT spectrogram.

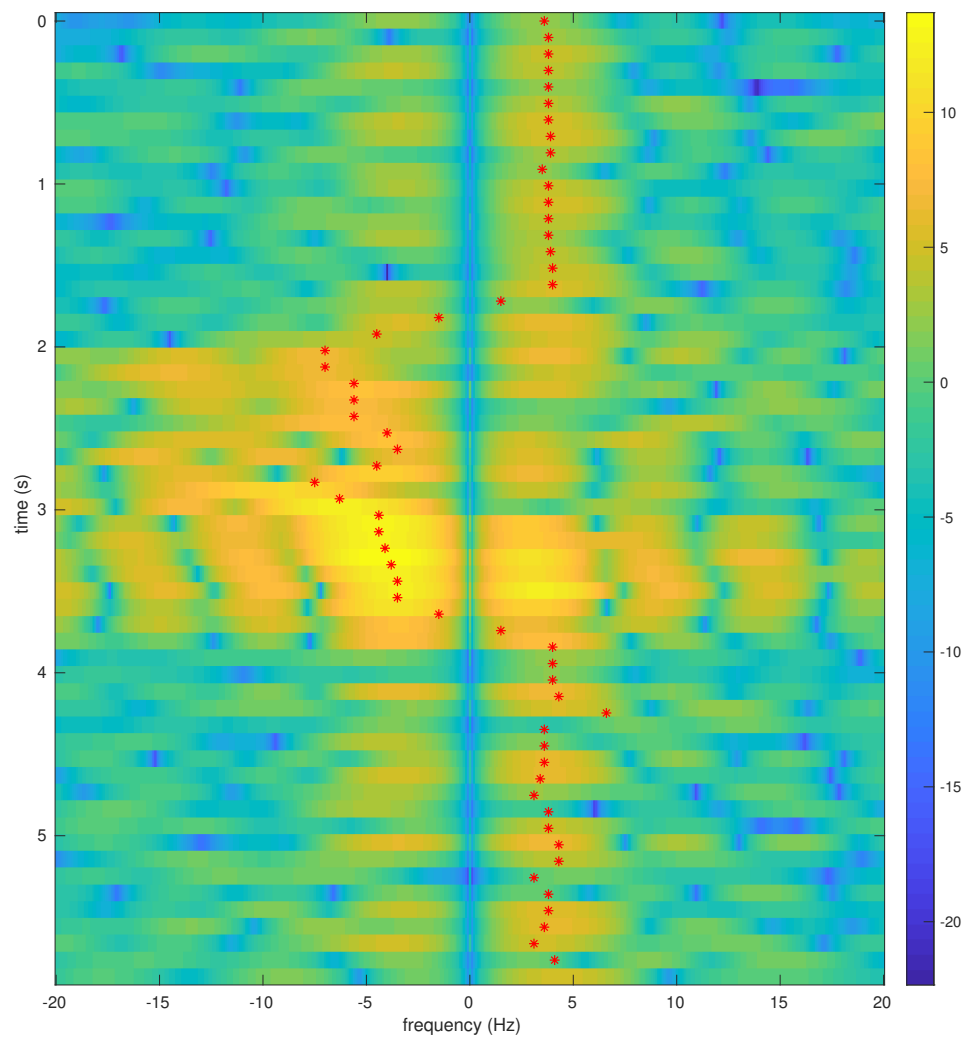


Fig. 5.5: Test 6 extracted path over a DFT spectrogram.

5.2.2 July 13, 2020

This test set has the same setup as the previous one. Two Heyrtz antennas were attached on top of a cubicle wall, a foot apart from each other, and were connected to a single E320 device. This test set introduced a new type of test, which allowed the Doppler echo path to become more apparent. The new test type involved having the moving target continuously move towards and away from the antenna array. The setup for these tests is recorded in Tables 5.5 – 5.7 and the tests are recorded in Table 5.8.

The objective of these tests was to have a continuous path of Doppler frequencies to avoid the problem of the Doppler sitting at DC. Figures 5.6 to 5.9 show the results of the

Table 5.5: Device Parameters, July 13, 2020

Transmitter	E320 #41, RFA, Tx/Rx
Transmitter gain	89.75 dB
Receiver	E320 #41, RFB, Tx/Rx
Receiver gain	36 dB
Number of amplifiers	0
Sample rate	31250 Samples/s
Interpolation/Decimation rate	1
Carrier frequency	905 MHz
Master clock rate	500 kHz

Table 5.6: Symbol Timing Synchronization Parameters, July 13, 2020

Samples per symbol	4
Excess bandwidth	0.5
Matched filter length	161
TED	$y[n]y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.1
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.7: Carrier phase synchronization parameters, July 13, 2020

$B_n T_s$	90μ
ζ	0.707
K_p	0.5
K_0	1

Table 5.8: Tests, July 13, 2020

Test	Metal	Description
1	Yes	Moving towards and away from the antenna array repeatedly
2	Yes	Moving towards and away from the antenna array repeatedly
3	No	Moving towards and away from the antenna array repeatedly
4	No	Moving towards and away from the antenna array repeatedly

in and out motion on the Doppler frequency path. The Doppler frequency path oscillates in all of these figures and shows the ability of the Viterbi algorithm to find this path in the noise more easily than the previous test sets.

These tests were the first to compare the DFT and the MUSIC algorithm. The MUSIC algorithm overall does a better job at extracting the signal than the DFT. The spectrogram created by MUSIC has less spread due to the increase in resolution. However, MUSIC is computationally more complex, especially since $P = 50$ and $M = 200$ for both of the MUSIC spectrograms.

Once again, the tests with the metal are easier to extract the Doppler path from than the tests without the metal. This is more clear in these figures than those in the last test set as the MUSIC spectrogram becomes noisier in Figure 5.9 than in Figure 5.8.

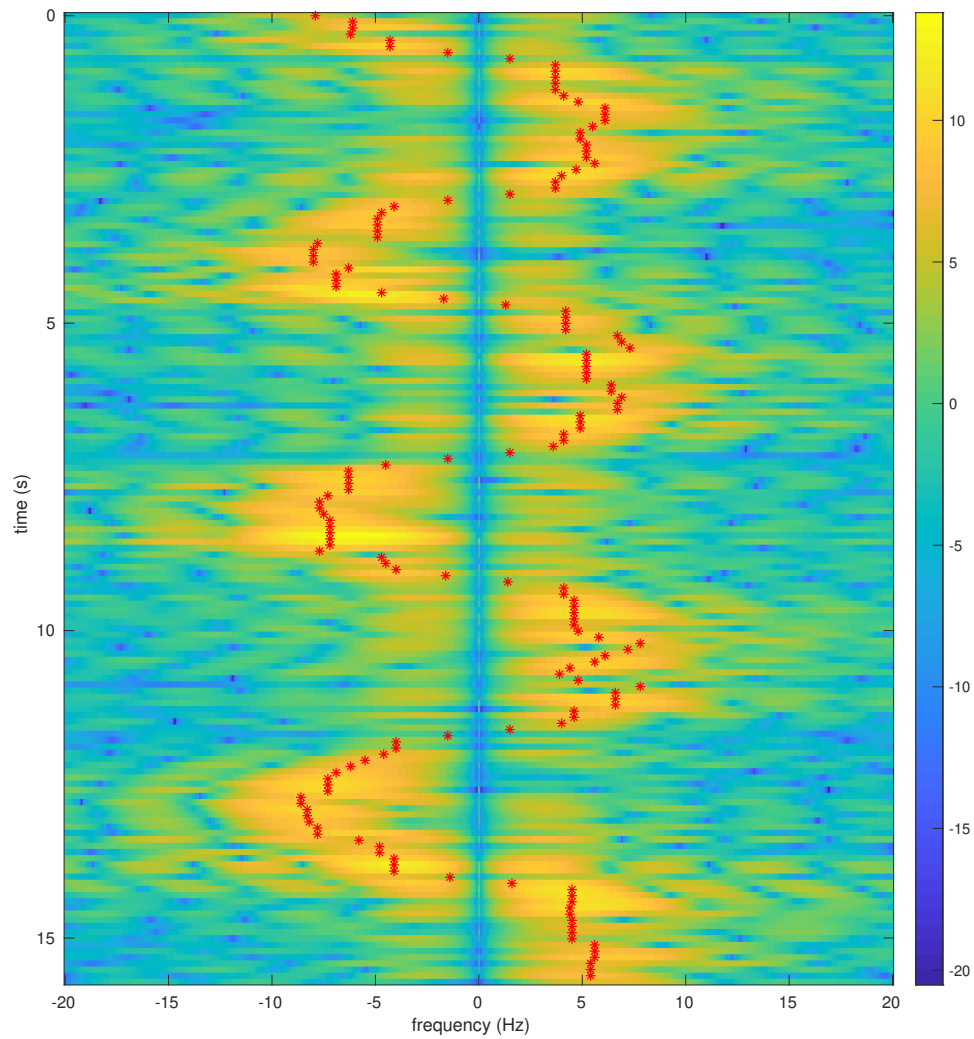


Fig. 5.6: Test 1 extracted path over a DFT spectrogram.

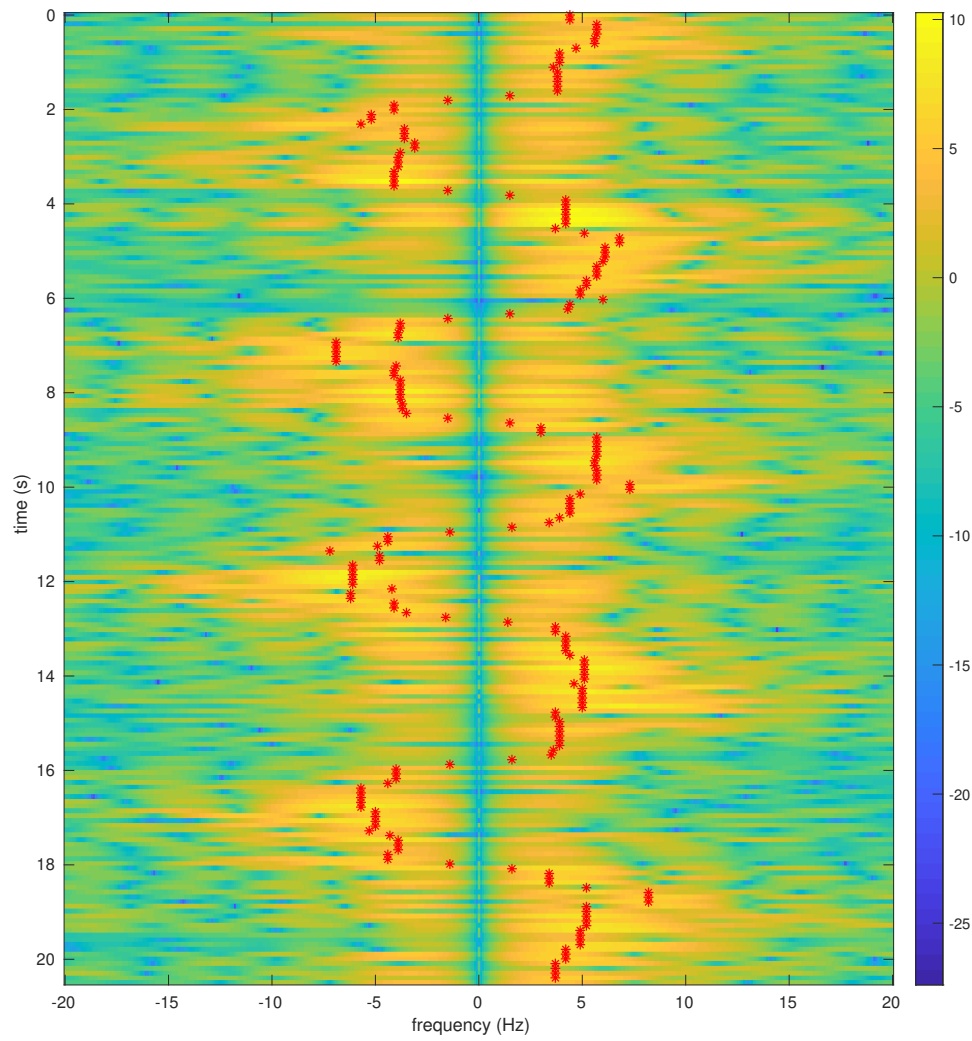


Fig. 5.7: Test 3 extracted path over a DFT spectrogram.

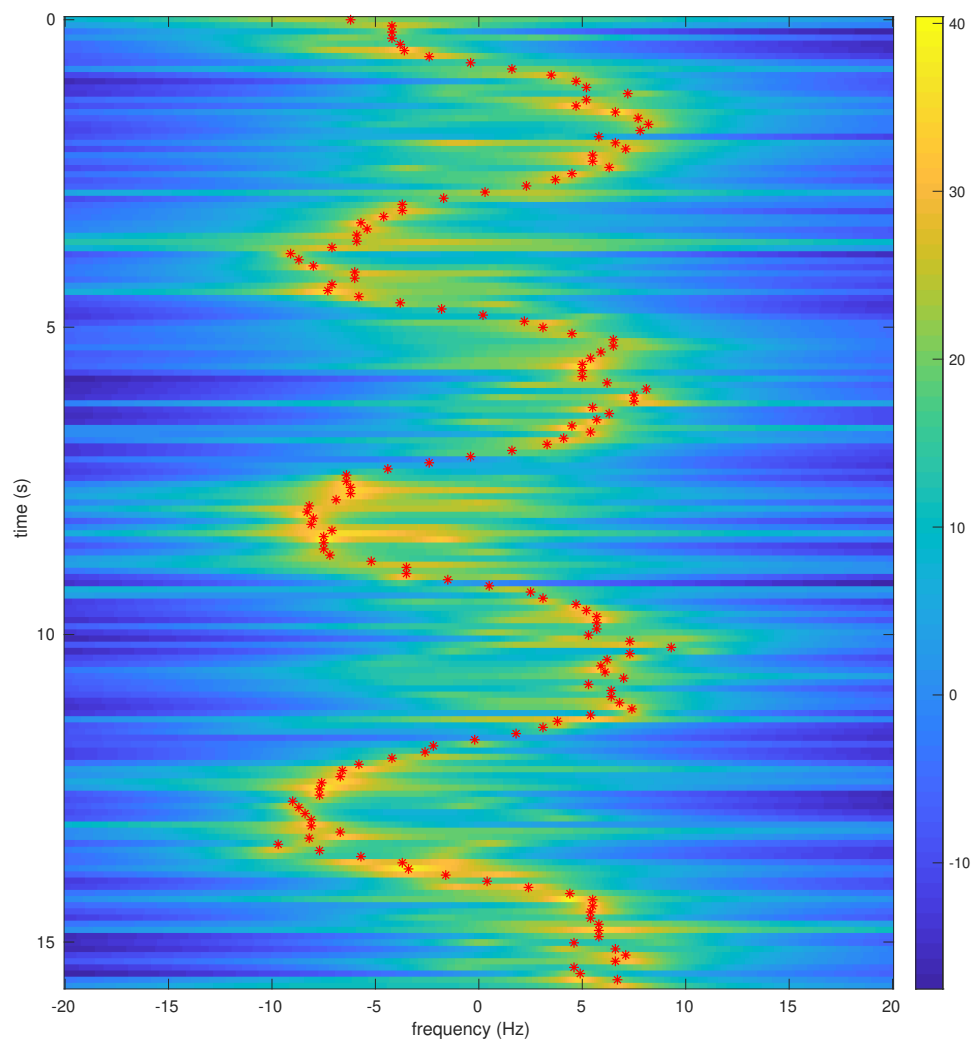


Fig. 5.8: Test 1 extracted path over a MUSIC spectrogram.

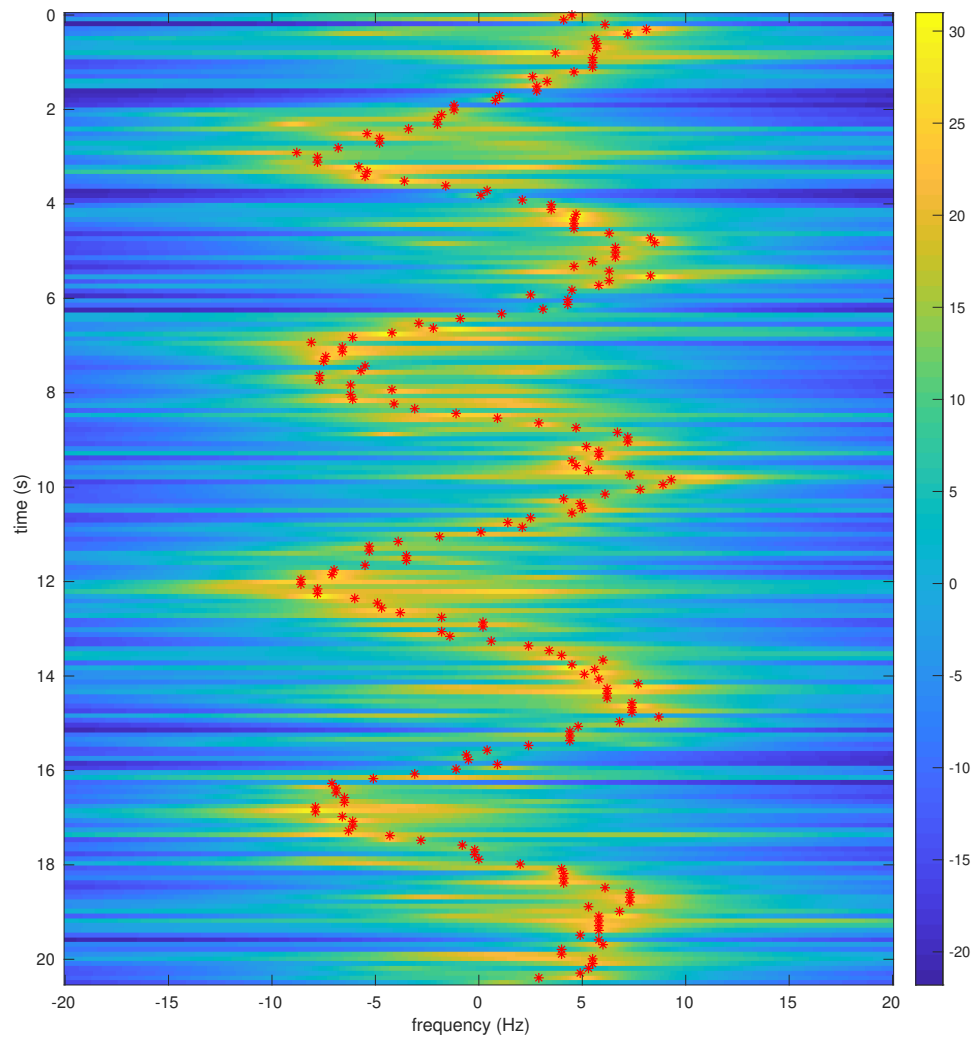


Fig. 5.9: Test 3 extracted path over a MUSIC spectrogram.

5.2.3 July 15, 2020

These tests once again used two Heyrtz antennas attached to the top of a cubicle a foot apart from each other. The setup for these tests was nearly identical to the previous tests, except for the loop bandwidth being higher and the use of two E320s. The setup for these tests is recorded in Tables 5.9 – 5.11 and the tests are recorded in Table 5.12.

These tests were the first tests using both Doppler extraction methods. The purpose of these tests was to see if the Second DEM was able to more accurately extract the Doppler than the First DEM when there was a carrier frequency offset.

Figure 5.10 shows how the First DEM fails when there is a carrier frequency offset. The spectrogram becomes symmetric and makes it difficult for the Viterbi algorithm to find the correct path through the spectrogram. There is also additional noise besides the symmetric Doppler peak. This suggests the harmonics seen in the simulation have more effects than the simulation suggested.

Table 5.9: Device Parameters, July 15, 2020

Transmitter	E320 #41, RFA, Tx/Rx
Transmitter gain	89.75 dB
Receiver	E320 #40, RFB, Tx/Rx
Receiver gain	62 dB
Number of amplifiers	0
Sample rate	31250 Samples/s
Interpolation/Decimation rate	1
Carrier frequency	905 MHz
Master clock rate	500 kHz

Table 5.10: Symbol Timing Synchronization Parameters, July 15, 2020

Samples per symbol	4
Excess bandwidth	0.5
Matched filter length	161
TED	$y[n]y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.1
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.11: Carrier phase synchronization parameters, July 15, 2020

$B_n T_s$	90 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

Table 5.12: Tests, July 15, 2020

Test	Metal	Description
1	Yes	Moving towards and away from the antenna array repeatedly, transmitter and receiver tied to the same clock
2	No	Moving towards and away from the antenna array repeatedly, transmitter and receiver tied to the same clock
3	Yes	Moving towards and away from the antenna array repeatedly, transmitter and receiver on different clocks
4	No	Moving towards and away from the antenna array repeatedly, transmitter and receiver on different clocks

The Second DEM is able to clean up this spectrogram and find an appropriate Doppler path as seen in Figure 5.11. The estimation of the carrier frequency and subsequent demodulation of the signal are able to remove the carrier frequency offset well enough to extract the path from the spectrogram with the Viterbi algorithm.

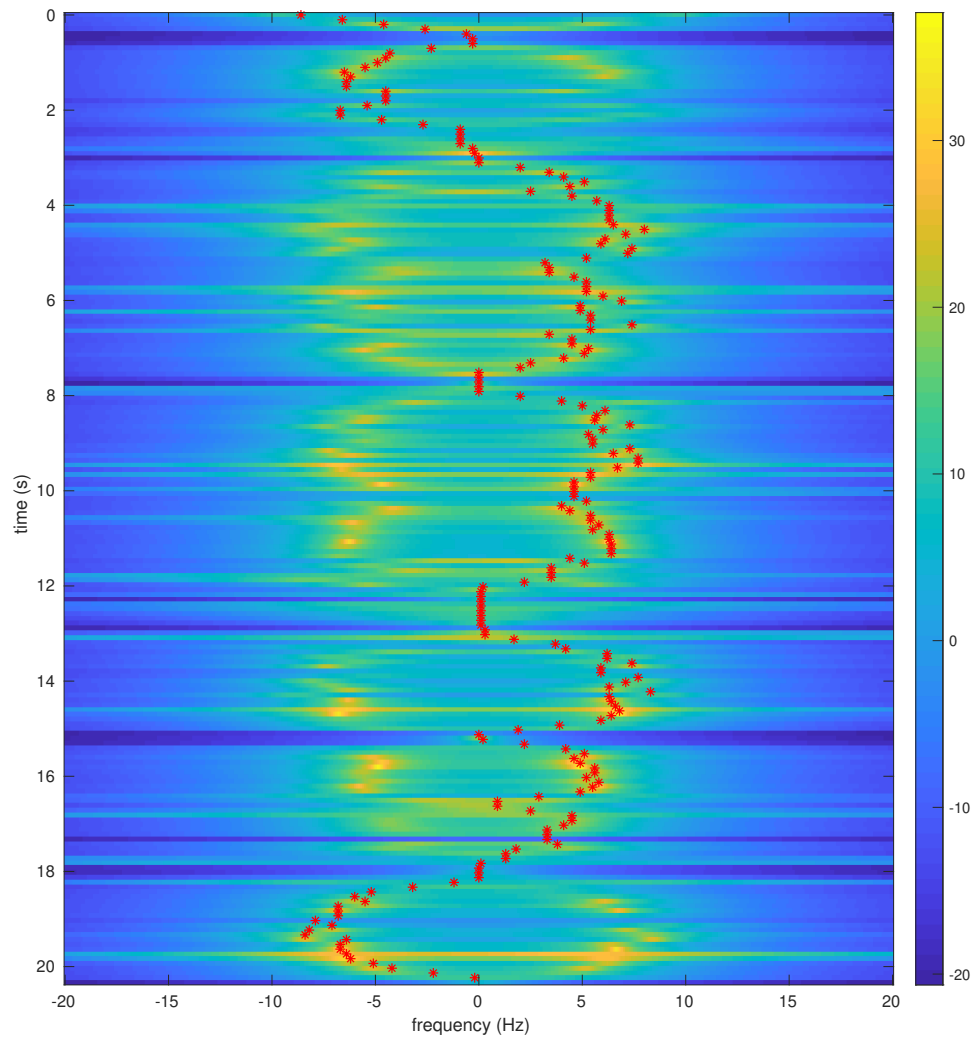


Fig. 5.10: Test 3 using the First DEM, with the optimal path plotted on the MUSIC spectrum.

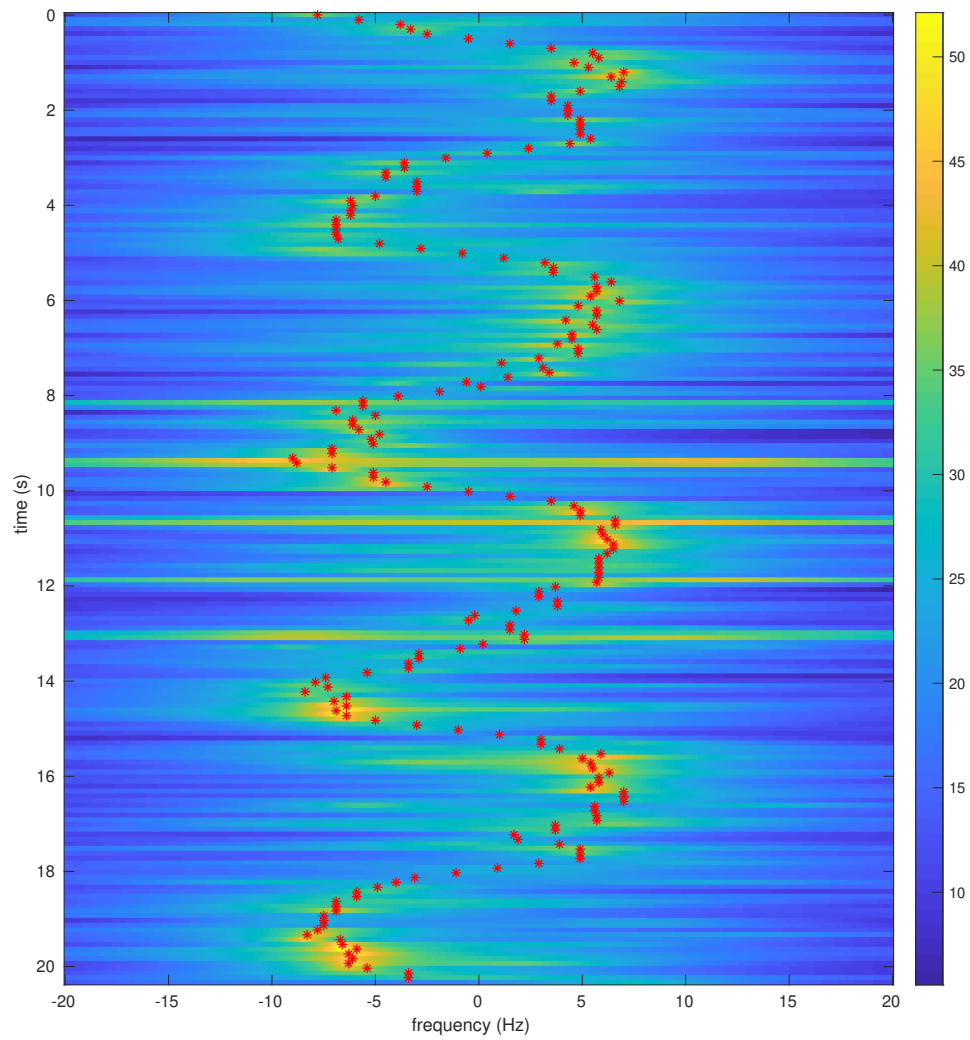


Fig. 5.11: Test 3 using the Second DEM, with the optimal path plotted on the MUSIC spectrum.

5.2.4 July 21, 2020

These tests used two Heyrtz antennas attached to the top of a cubicle a foot apart from each other. These tests were similar to the previous test sets but used different devices for the transmitter and receiver. A glitch in E320 devices caused extra phase noise. This extra phase noise made it difficult to extract the Doppler without the metal, so other devices were explored as alternatives. This test set investigated using the B200 mini and RTL-SDR devices. The setup for these tests is recorded in Tables 5.13 – 5.15 and the tests are recorded in Table 5.16.

The RTL-SDR dongle is not an optimal device for collecting the Doppler data, as it has a poor clock. A poor clock results in the carrier frequency offset becoming noisier and harder to account for in the carrier phase removal. Figure 5.12 shows the system was still able to extract a path for the Doppler despite the extra noise in Test 1, the test with metal. However, without the metal, the Viterbi algorithm lost the path partway through the test

Table 5.13: Device Parameters, July 21, 2020

Transmitter	B200 mini
Transmitter gain	75 dB
Receiver	RTL-SDR Dongle
Receiver gain	25 dB
Number of amplifiers	0
Sample rate	62500 Samples/s
Interpolation/Decimation rate	1
Carrier frequency	905 MHz
Master clock rate	n/a

Table 5.14: Symbol Timing Synchronization Parameters, July 21, 2020

Samples per symbol	8
Excess bandwidth	0.5
Matched filter length	161
TED	$y[n]y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.1
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.15: Carrier phase synchronization parameters, July 21, 2020

$B_n T_s$	90 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

Table 5.16: Tests, July 21, 2020

Test	Metal	Description
1	Yes	Moving towards and away from the antenna array repeatedly
2	Yes	Moving towards and away from the antenna array repeatedly
3	No	Moving towards and away from the antenna array repeatedly
4	No	Moving towards and away from the antenna array repeatedly

as Figure 5.13 shows.

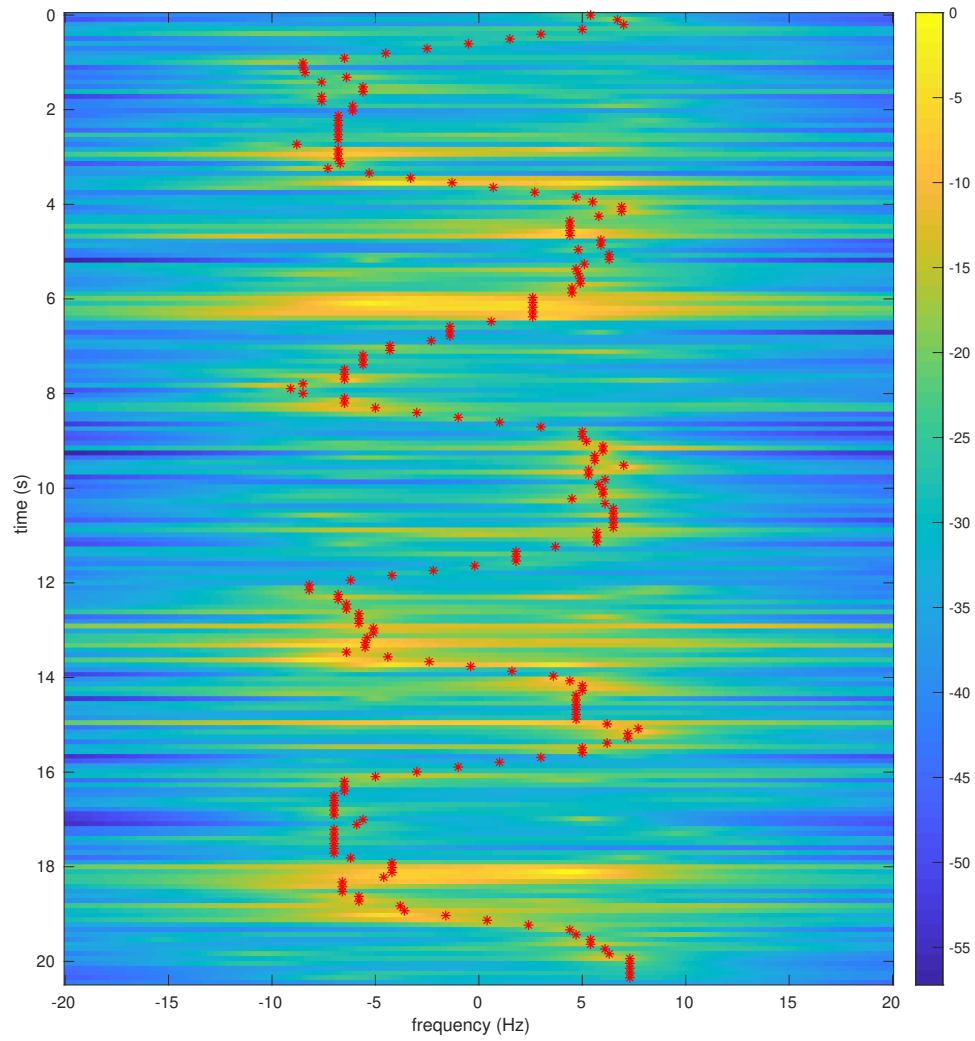


Fig. 5.12: Test 1 with optimal path plotted on MUSIC spectrum.

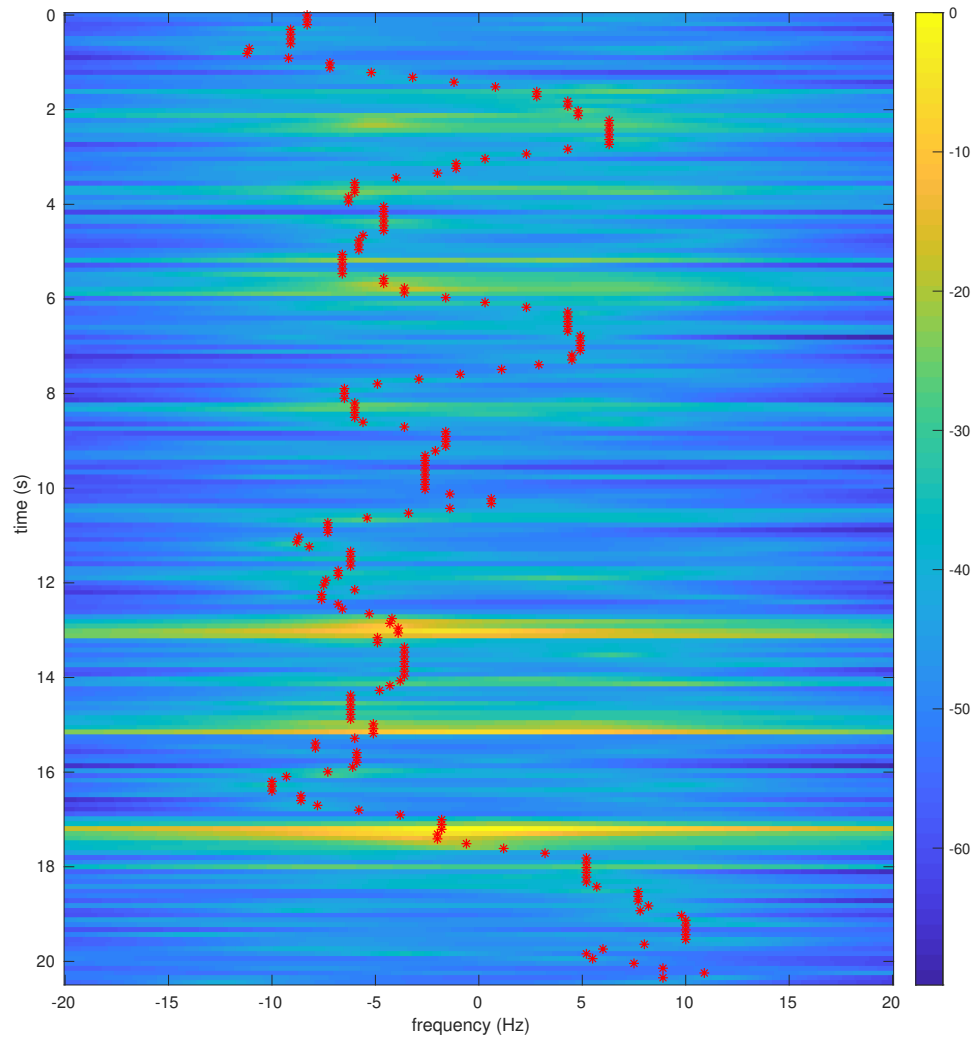


Fig. 5.13: Test 3 with optimal path plotted on MUSIC spectrum.

5.2.5 August 10, 2020

These tests used two Heyrtz antennas attached to the top of a cubicle a foot apart from each other. This test set investigated using the B200 mini and B200 devices. The setup for these tests is recorded in Tables 5.17 – 5.19 and the tests are recorded in Table 5.20.

The B200 device has a GPSDO clock on it, causing the carrier frequency offset to be less noisy than the carrier frequency offset for the RTL-SDR devices in the previous test set. This led to the ability to extract the Doppler frequency from the signal both with and without the metal. Figure 5.14 shows the Doppler extraction with the metal and Figure 5.15 shows the Doppler extraction without the metal.

Table 5.17: Device Parameters, August 10, 2020

Transmitter	B200 mini
Transmitter gain	80 dB
Receiver	B200
Receiver gain	75 dB
Number of amplifiers	0
Sample rate	62500 Samples/s
Interpolation/Decimation rate	8
Carrier frequency	905 MHz
Master clock rate	n/a

Table 5.18: Symbol Timing Synchronization Parameters, August 10, 2020

Samples per symbol	8
Excess bandwidth	0.5
Matched filter length	161
TED	$y[n]y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.02
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.19: Carrier phase synchronization parameters, August 10, 2020

$B_n T_s$	9 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

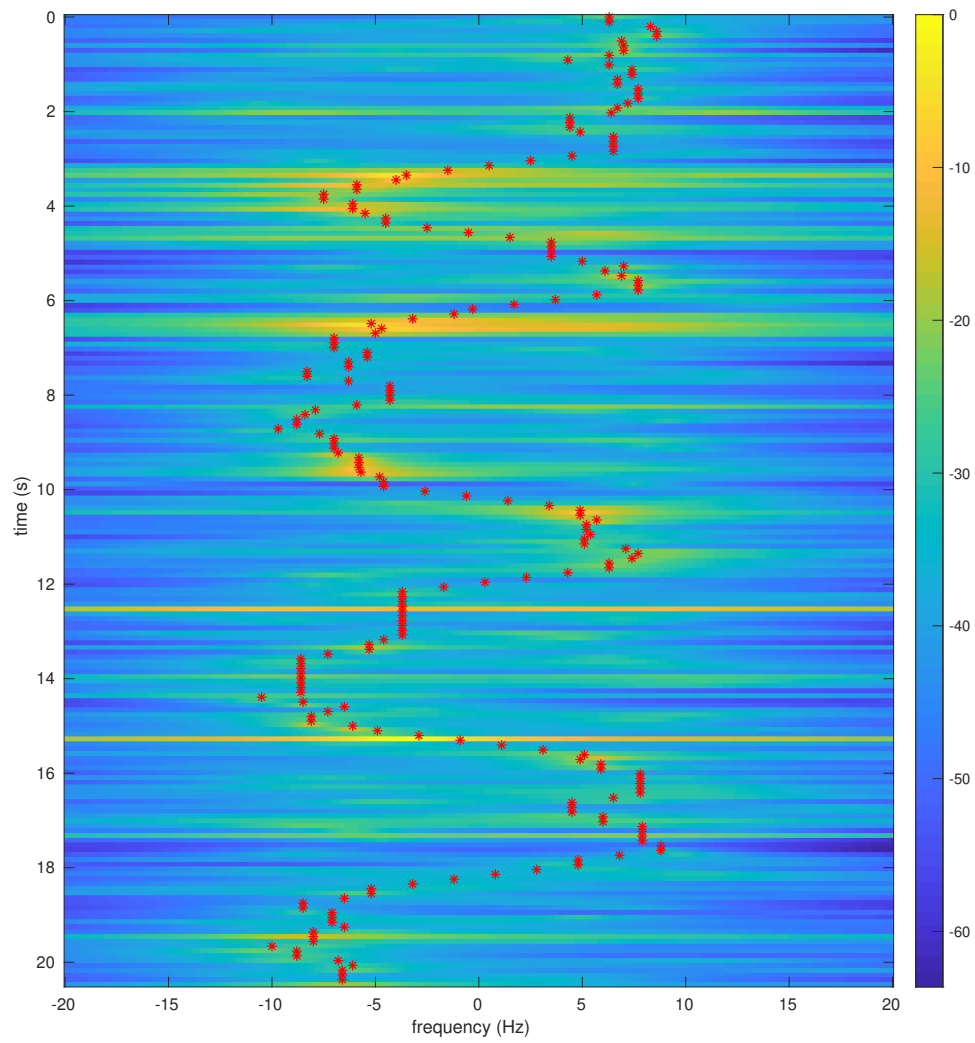


Fig. 5.14: Test 1 with the optimal path plotted on MUSIC spectrum.

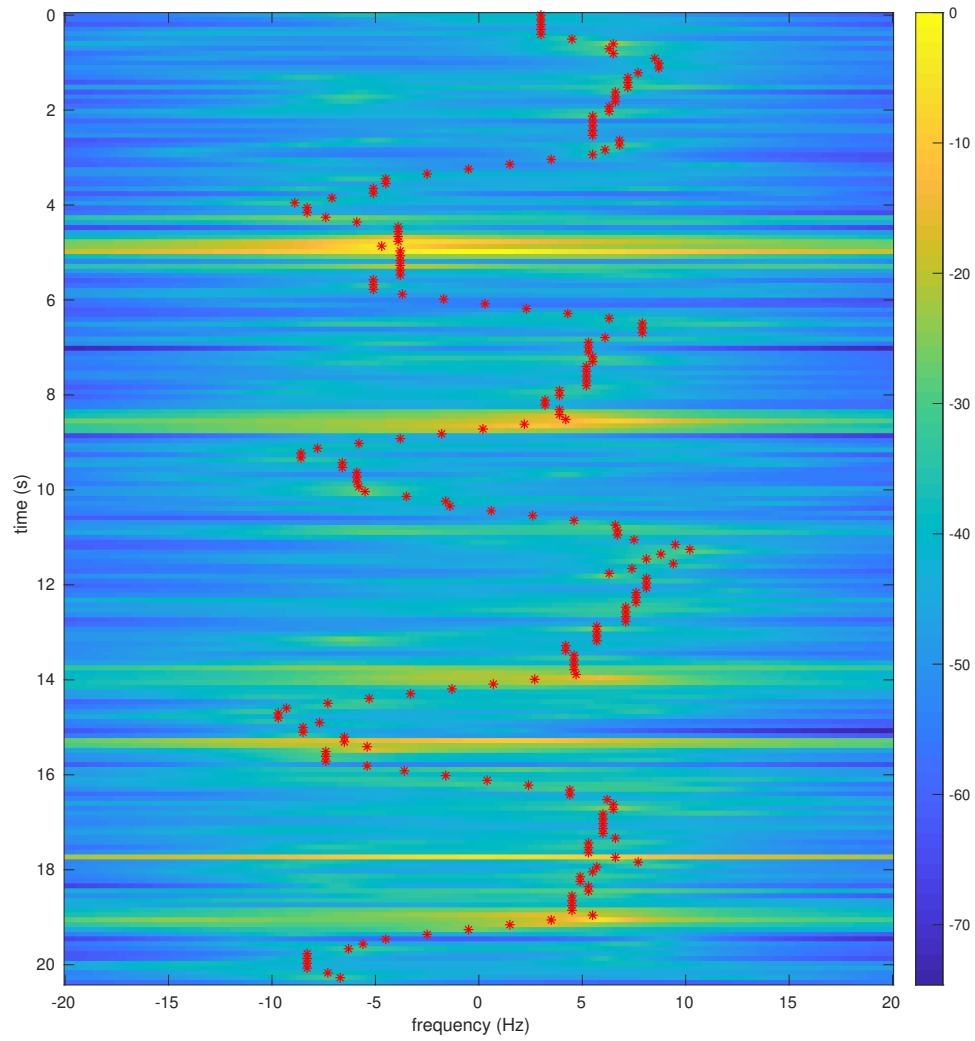


Fig. 5.15: Test 2 with the optimal path plotted on MUSIC spectrum.

Table 5.20: Tests, August 10, 2020

Test	Metal	Description
1	Yes	Moving towards and away from the antenna array repeatedly
2	No	Moving towards and away from the antenna array repeatedly
3	Yes	Moving toward array suddenly
4	No	Moving toward array suddenly
5	Yes	Walking in a clockwise circle
6	No	Walking in a clockwise circle
7	Yes	Walking in and out of the room in a counter clockwise direction
8	No	Walking in and out of the room in a counter clockwise direction

5.2.6 Simulated Doppler Paths

The rest of the test sets recorded data using the SW, NW, SC, and NC hall antennas, which were positioned as shown in Figure 2.6. These antennas are HyperLink antennas, which were better suited for the transmission frequency of the communication signal.

The correct path for these tests is harder to determine due to the geometry of the antennas being more complex than the previous setup. Therefore, a simulation program was written in order to simulate what the Doppler path might look like.

All of the following test sets have one test in common. This test is when the moving target moves in the southern hall between the SC and SW antennas. Therefore, this test was simulated to compare with the actual Doppler frequency paths.

The simulated data was merely used to get a general idea of what the path should look like. It is not a direct comparison. Human movement is hard to simulate precisely in a computer model, and, therefore, the simulated Doppler path is somewhat unrepresentative of the real Doppler path. However, the general path of these simulations should be close to the actual recorded paths. Figures 5.16 – 5.19 show the simulated paths.

Some of the tests move the position of the transmitter. These tests can still be compared to this simulated data, but caution should be used when using such comparisons as the placement of the transmitter can easily change the Doppler path at each antenna.

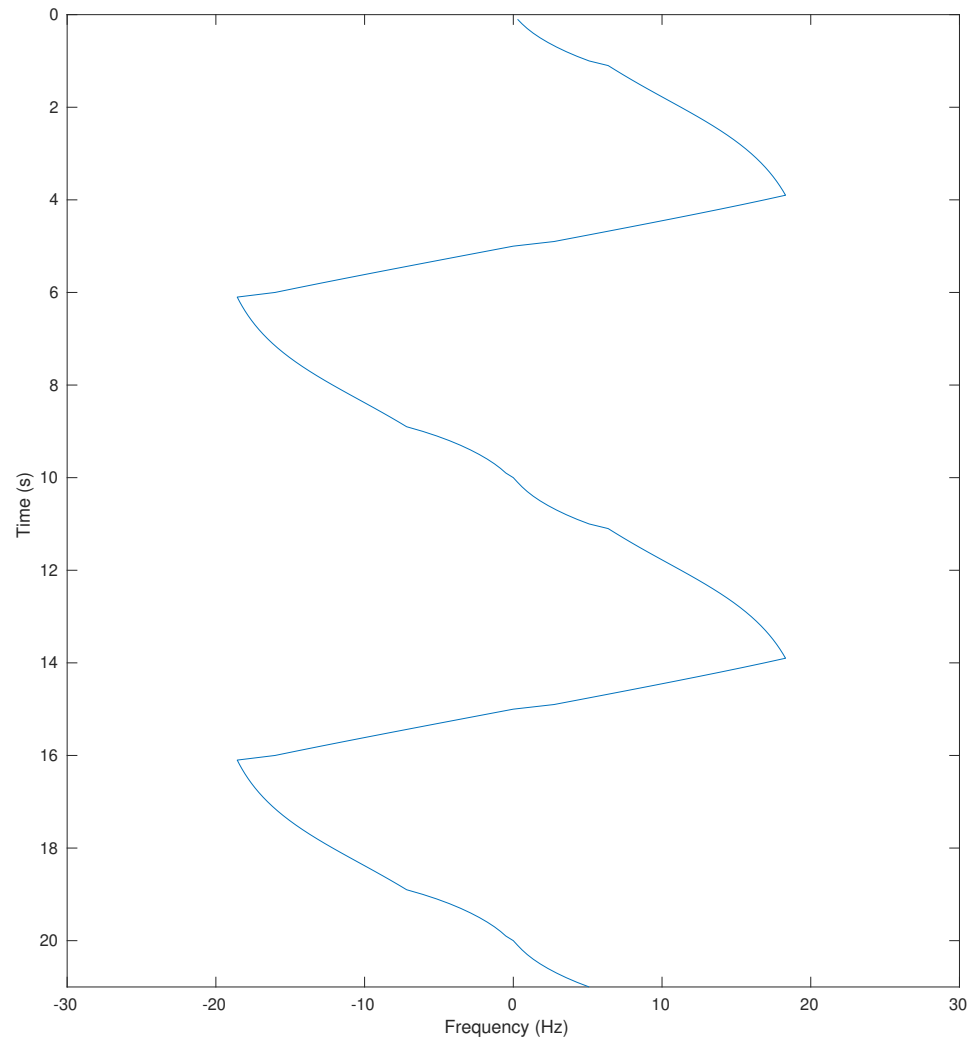


Fig. 5.16: Doppler frequency simulated at the SC antenna.

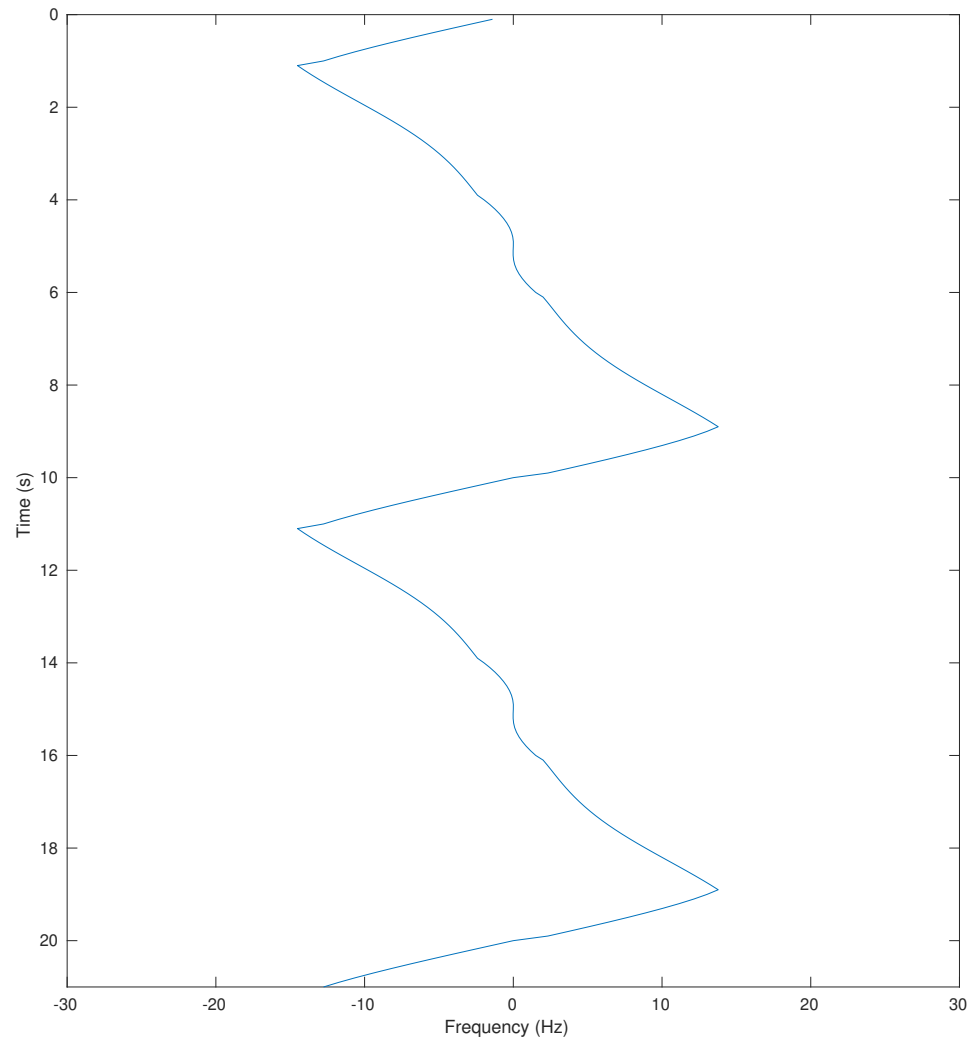


Fig. 5.17: Doppler frequency simulated at the SW antenna.

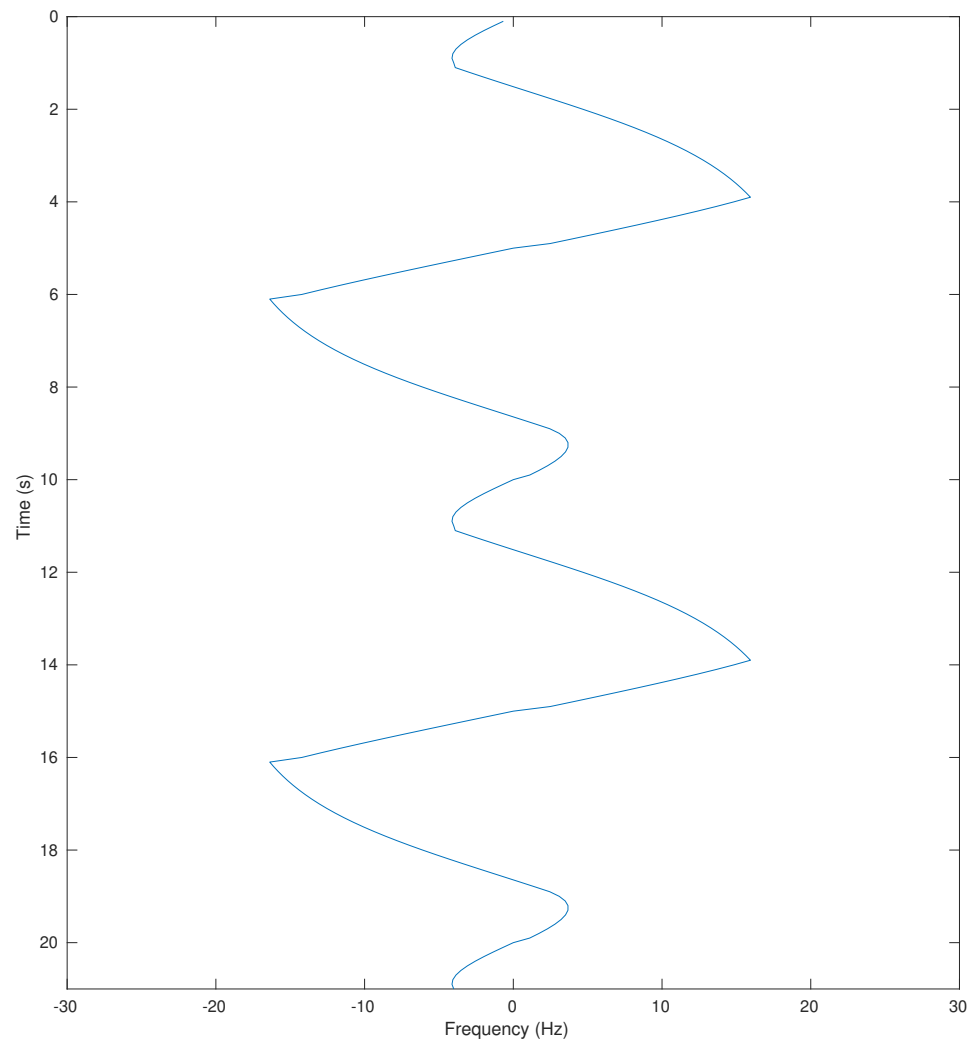


Fig. 5.18: Doppler frequency simulated at the NC antenna.

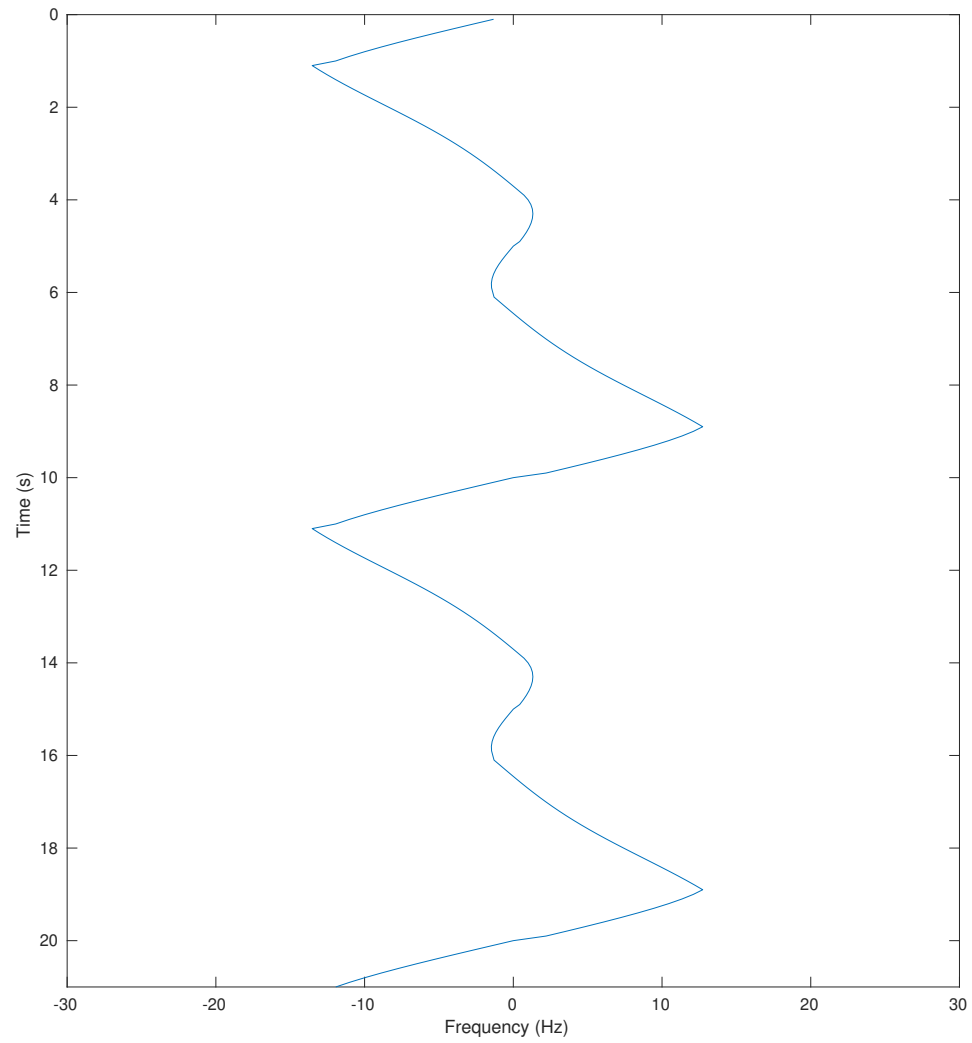


Fig. 5.19: Doppler frequency simulated at the NW antenna.

5.2.7 October 2, 2020

For these tests, the transmitter was placed right by the door to the hall as depicted in Figure 2.6. The receive antennas were set up as mentioned in Section 5.2.6. The position of the transmitter relative to the SW antenna was 10.7 m to the east and 0.9 m to the south. These tests were the first successful hall tests. The setup for these tests is recorded in Tables 5.21 – 5.23 and the tests are recorded in Table 5.24. Prior tests had all failed due to the lack of power on the transmitted signal. An additional amplifier was added to the system to increase the transmit power.

These tests were successful in extracting the Doppler path in a more realistic system.

Table 5.21: Device Parameters, October 2, 2020

Transmitter	B200 mini
Transmitter gain	85 dB
Receiver 1	B210-1, RF A, Tx/Rx
Receiver 1 gain	45 dB
Receiver 1 antenna	SC building antenna
Receiver 2	B210-2, RF A, Tx/Rx
Receiver 2 gain	56 dB
Receiver 2 antenna	SW building antenna
Receiver 3	B200, Tx/Rx
Receiver 3 gain	80 dB
Receiver 3 antenna	NW building antenna
Number of amplifiers	2
Sample rate	62500 Samples/s
Interpolation/Decimation rate	4
Carrier frequency	905 MHz
Master clock rate	n/a

Table 5.22: Symbol Timing Synchronization Parameters, October 2, 2020

Samples per symbol	8
Excess bandwidth	0.5
Matched filter length	161
TED	$\text{sgn}(y[n])y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.02
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.23: Carrier phase synchronization parameters, October 2, 2020

$B_n T_s$	90 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

Table 5.24: Tests, October 2, 2020

Test	Metal	Description
1	Yes	Moving between the SW and SC antennas
2	No	Moving between the SW and SC antennas
3	Yes	Moving between the SW antenna and transmit antenna
4	No	Moving between the SW antenna and transmit antenna

However, as Figure 5.20 and Figure 5.21 show, the distance from the transmitter to a receiver makes a difference. The NW antenna was not able to pick up a strong enough Doppler echo to extract the correct Doppler frequency path.

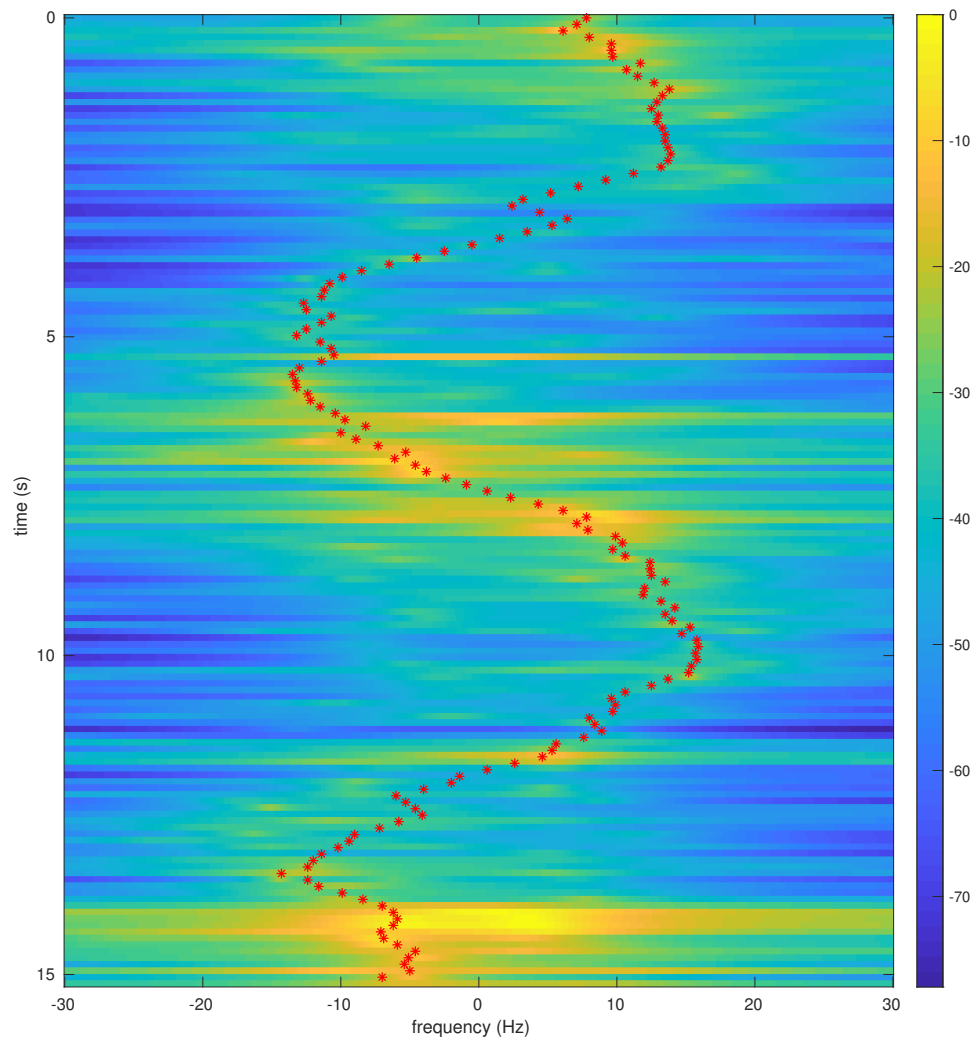


Fig. 5.20: Test 1 with optimal path plotted on MUSIC spectrum, SW antenna.

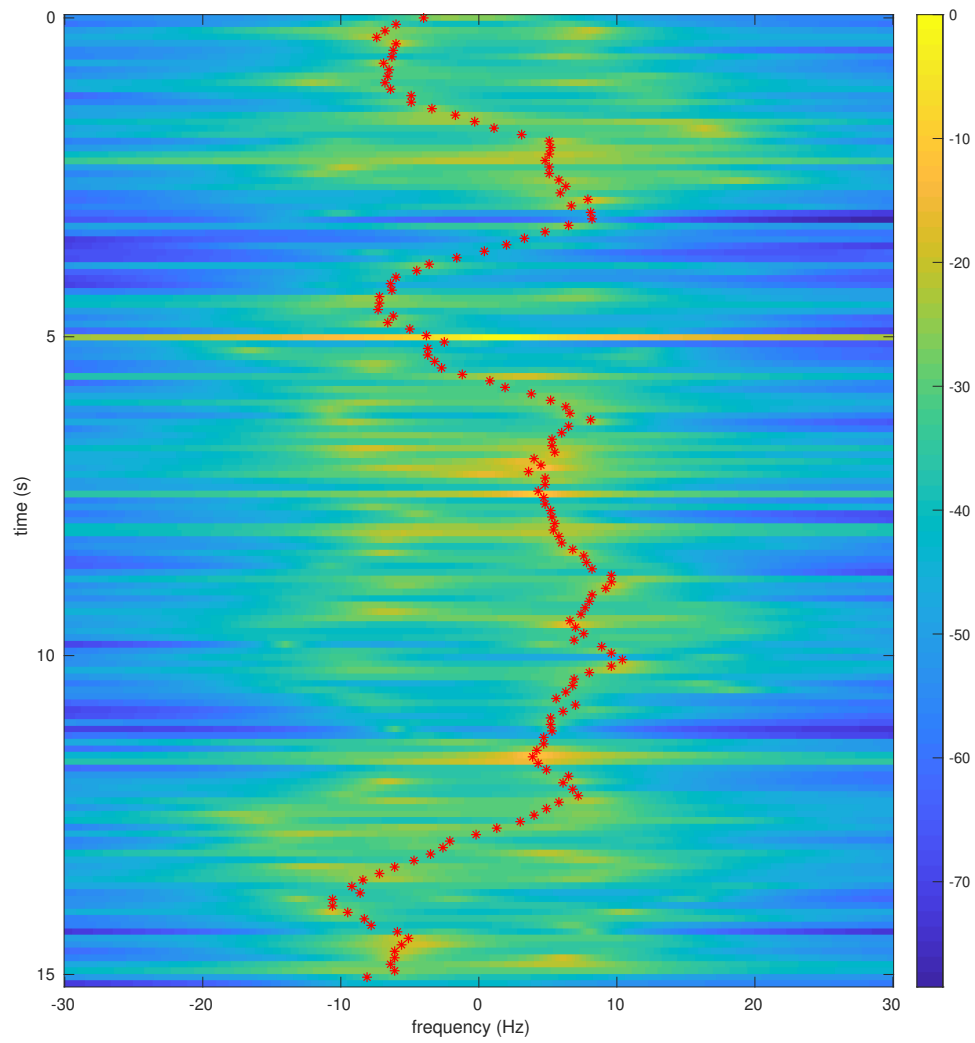


Fig. 5.21: Test 1 with optimal path plotted on MUSIC spectrum, NW antenna.

5.2.8 October 9, 2020

For these tests, the transmitter antenna was placed right by the door as in the previous test set. The receiver antennas were positioned as described in Section 5.2.6. The position of the transmitter relative to the SW antenna was 10.7 m to the east and 0.9 m to the south. The setup for these tests is recorded in Tables 5.25 – 5.27 and the tests are recorded in Table 5.28.

Table 5.25: Device Parameters, October 9, 2020

Transmitter	B210-1, RF A, Tx/Rx
Transmitter gain	85 dB
Receiver 1	B210-1, RF B, Rx2
Receiver 1 gain	43 dB
Receiver 1 antenna	SC building antenna
Receiver 2	B210-2, RF A, R2x
Receiver 2 gain	55 dB
Receiver 2 antenna	SW building antenna
Receiver 3	B200, Rx2
Receiver 3 gain	60 dB
Receiver 3 antenna	NC building antenna
Receiver 4	B200 mini, Rx2
Receiver 4 gain	77 dB
Receiver 4 antenna	NW building antenna
Number of amplifiers	2
Sample rate	62500 Samples/s
Interpolation/Decimation rate	4
Carrier frequency	905 MHz
Master clock rate	n/a

Table 5.26: Symbol Timing Synchronization Parameters, October 9, 2020

Samples per symbol	8
Excess bandwidth	0.5
Matched filter length	161
TED	$\text{sgn}(y[n])y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.02
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.27: Carrier phase synchronization parameters, October 9, 2020

$B_n T_s$	90 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

Table 5.28: Tests, October 9, 2020

Test	Metal	Description
1	Yes	Moving between the SW and SC antennas
2	No	Moving between the SW and SC antennas
3	Yes	Moving between the SW and NW antennas
4	No	Moving between the SW and NW antennas
5	Yes	Moving between the NC and NW antennas
6	No	Moving between the NC and NW antennas
7	Yes	Moving in a clockwise circle in the middle of the three central rooms depicted in figure 2.6
8	No	Moving in a clockwise circle in the middle of the three central rooms depicted in figure 2.6

These tests were the first with the balance factor correction added to the BCJR algorithm. This helped overcome situations where the Doppler was at 0 Hz. Figures 5.22 – 5.25 show the Doppler at all four of the antennas for Test 1. The NC and SC antenna’s extracted paths are similar to the simulated paths. However, the NW and SW paths do not compare as well.

All of these paths were generated using the same parameters. It is usually more appropriate to use different parameters for each antenna when different devices are used at each antenna. This is primarily due to each device having a different carrier frequency offset. It would be more ideal to use a single device or multiple of the same device tied to the same clock.

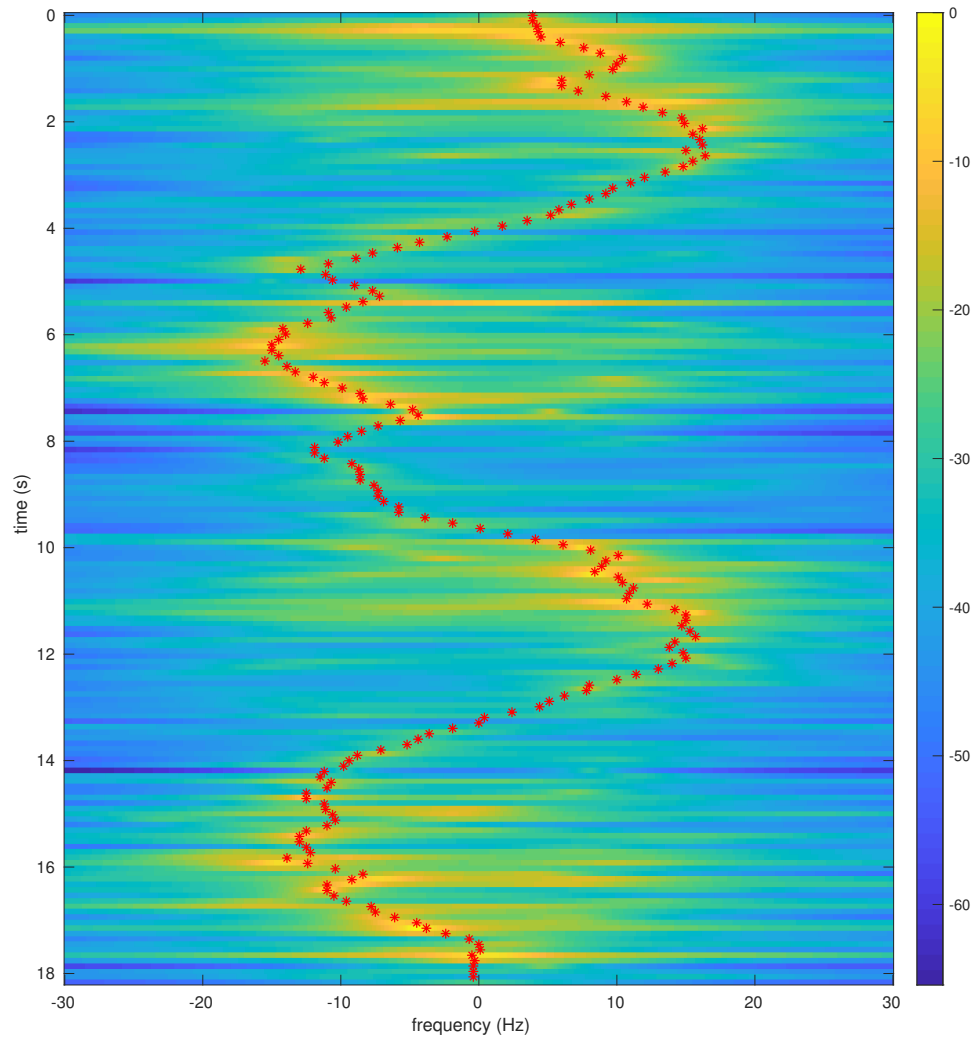


Fig. 5.22: Test 1 with optimal path plotted on MUSIC spectrum, SC antenna.

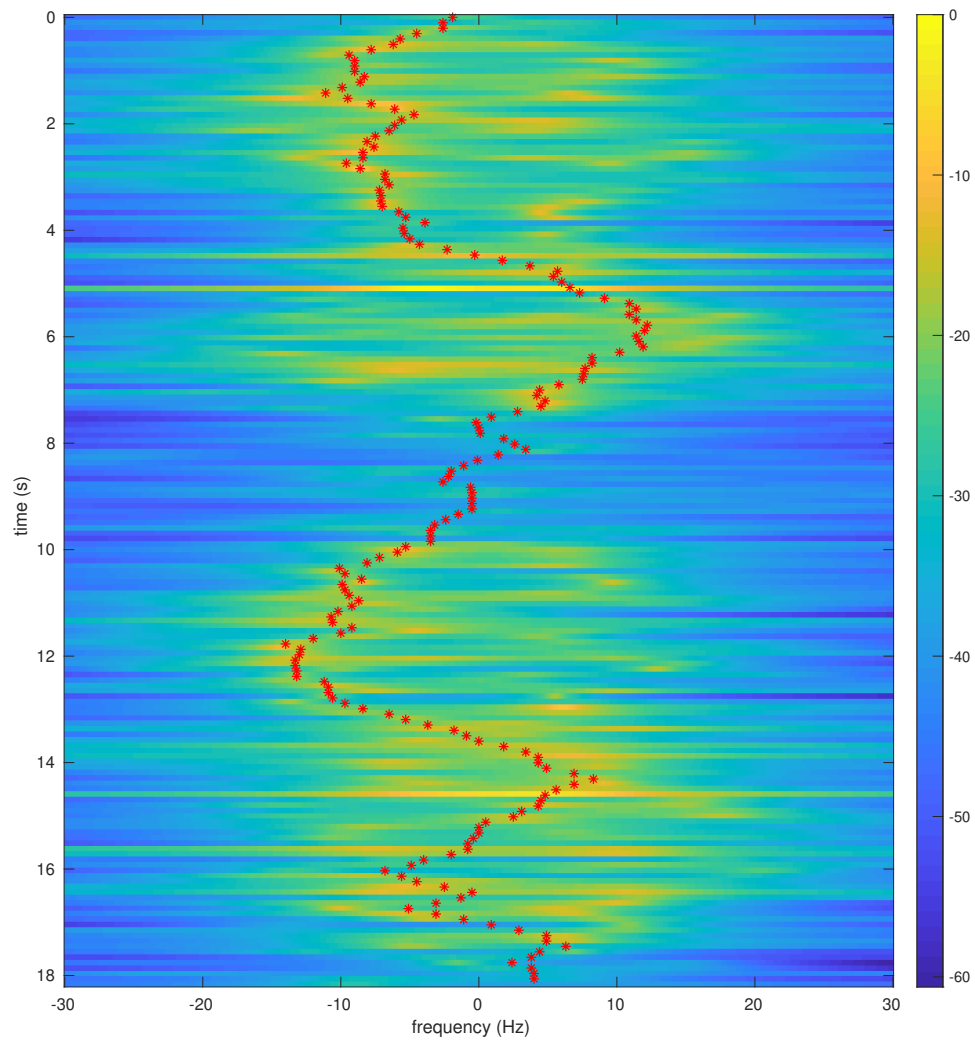


Fig. 5.23: Test 1 with optimal path plotted on MUSIC spectrum, SW antenna.

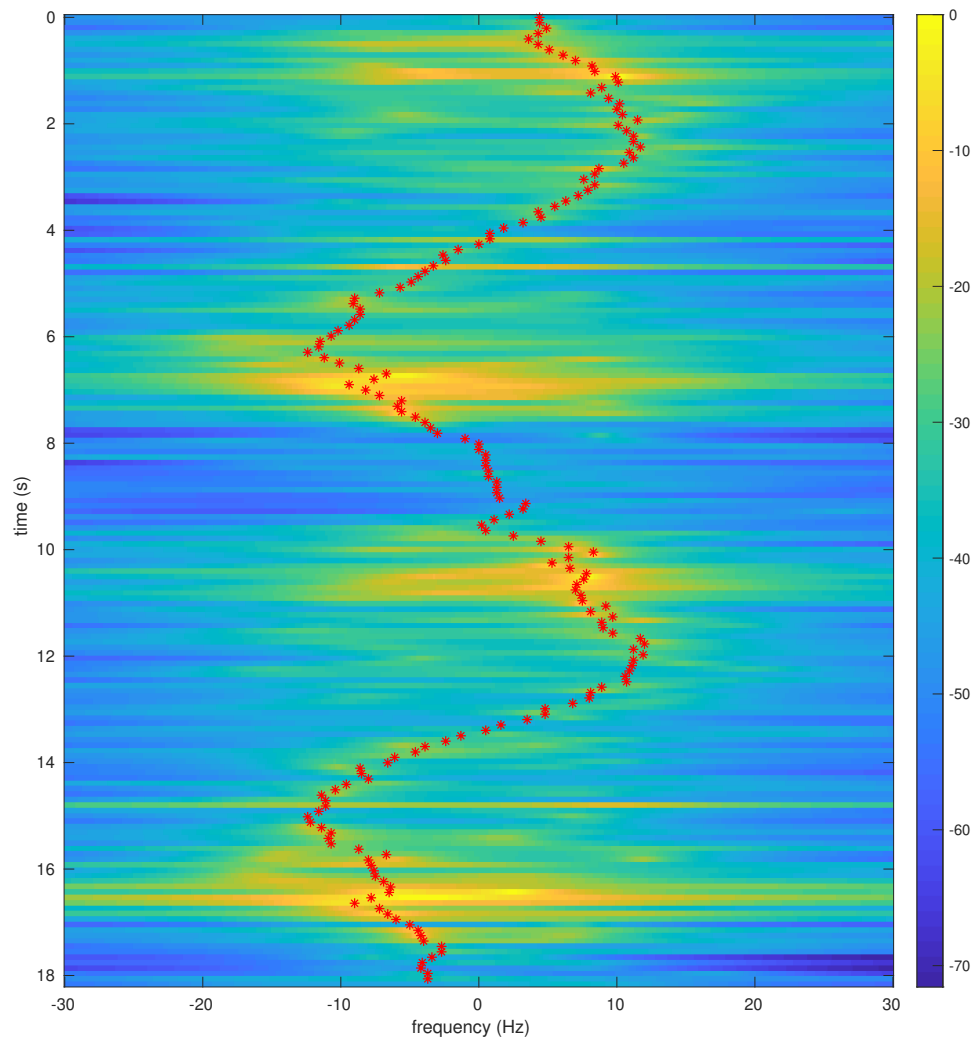


Fig. 5.24: Test 1 with optimal path plotted on MUSIC spectrum, NC antenna.

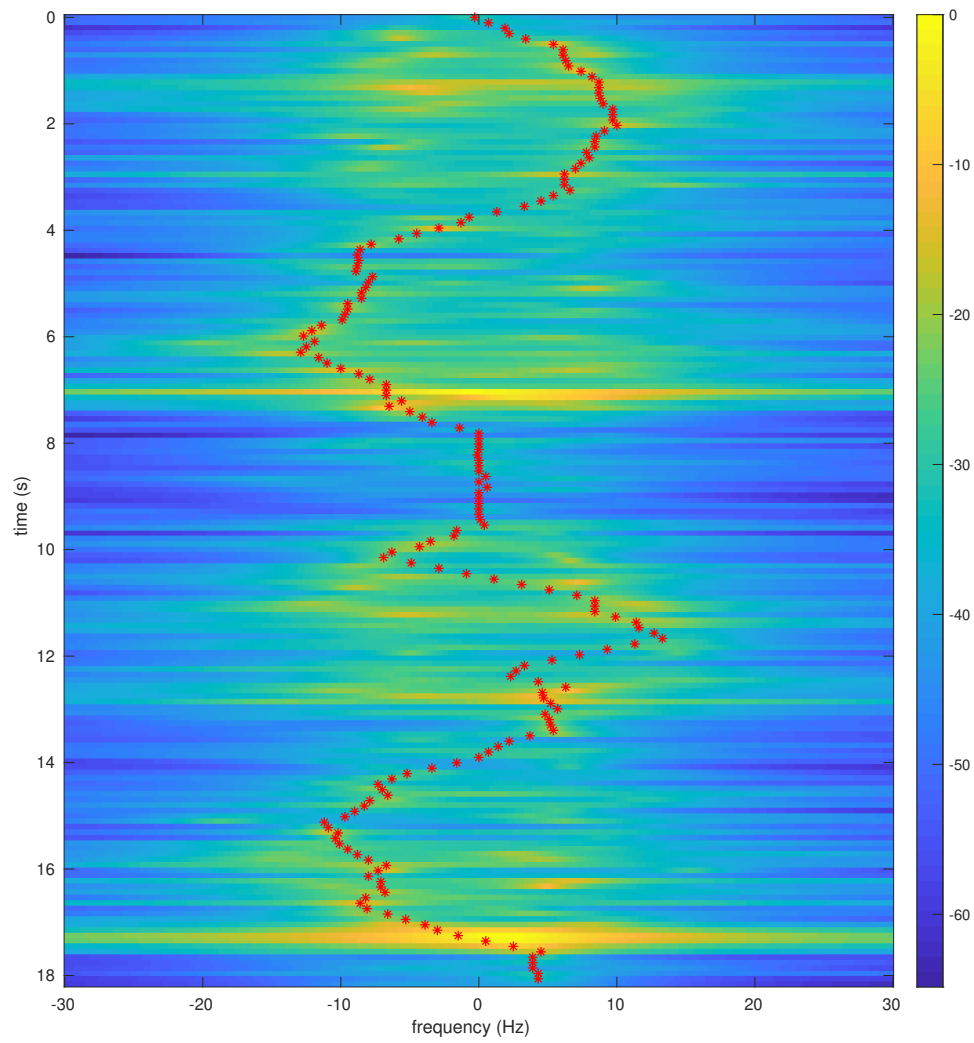


Fig. 5.25: Test 1 with optimal path plotted on MUSIC spectrum, NW antenna.

5.2.9 November 6, 2020

These tests moved the transmitter antenna further away from the door. The receiver antennas were positioned as described in Section 5.2.6. The transmitter was 10.4 m east and 5.8 m south of the SW antenna. The setup for these tests is recorded in Tables 5.29 – 5.31 and the tests are recorded in Table 5.32.

Figures 5.26 – 5.29 are the results of the Doppler extraction for Test 1. These results are encouraging, as both the SW and SC antenna paths are regular back and forth paths.

Table 5.29: Device Parameters, November 6, 2020

Transmitter	B210-1, RF A, Tx/Rx
Transmitter gain	85 dB
Receiver 1	B210-1, RF B, Rx2
Receiver 1 gain	51 dB
Receiver 1 antenna	SC building antenna
Receiver 2	B210-2, RF A, R2x
Receiver 2 gain	76 dB
Receiver 2 antenna	NW building antenna
Receiver 3	B200, Rx2
Receiver 3 gain	57 dB
Receiver 3 antenna	NC building antenna
Receiver 4	B200 mini, Rx2
Receiver 4 gain	75 dB
Receiver 4 antenna	SW building antenna
Number of amplifiers	2
Sample rate	62500 Samples/s
Interpolation/Decimation rate	4
Carrier frequency	905 MHz
Master clock rate	n/a

Table 5.30: Symbol Timing Synchronization Parameters, November 6, 2020

Samples per symbol	8
Excess bandwidth	0.5
Matched filter length	161
TED	$\text{sgn}(y[n])y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.02
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.31: Carrier phase synchronization parameters, November 6, 2020

$B_n T_s$	90 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

Table 5.32: Tests, November 6, 2020

Test	Metal	Description
1	Yes	Moving between the SW and SC antennas
2	No	Moving between the SW and SC antennas
3	Yes	Moving between the SW and NW antennas
4	No	Moving between the SW and NW antennas
5	Yes	Moving between the NC and NW antennas
6	No	Moving between the NC and NW antennas

However, the NW and NC paths are very poor. This is unsurprising as moving the transmitter further away would cause the Doppler echo to be even weaker when it reaches these antennas. The balance factor pulls the Doppler path to 0 Hz at the NC antenna because the path is too weak. This suggests the NC antenna is outside of the maximum range the system can operate in.

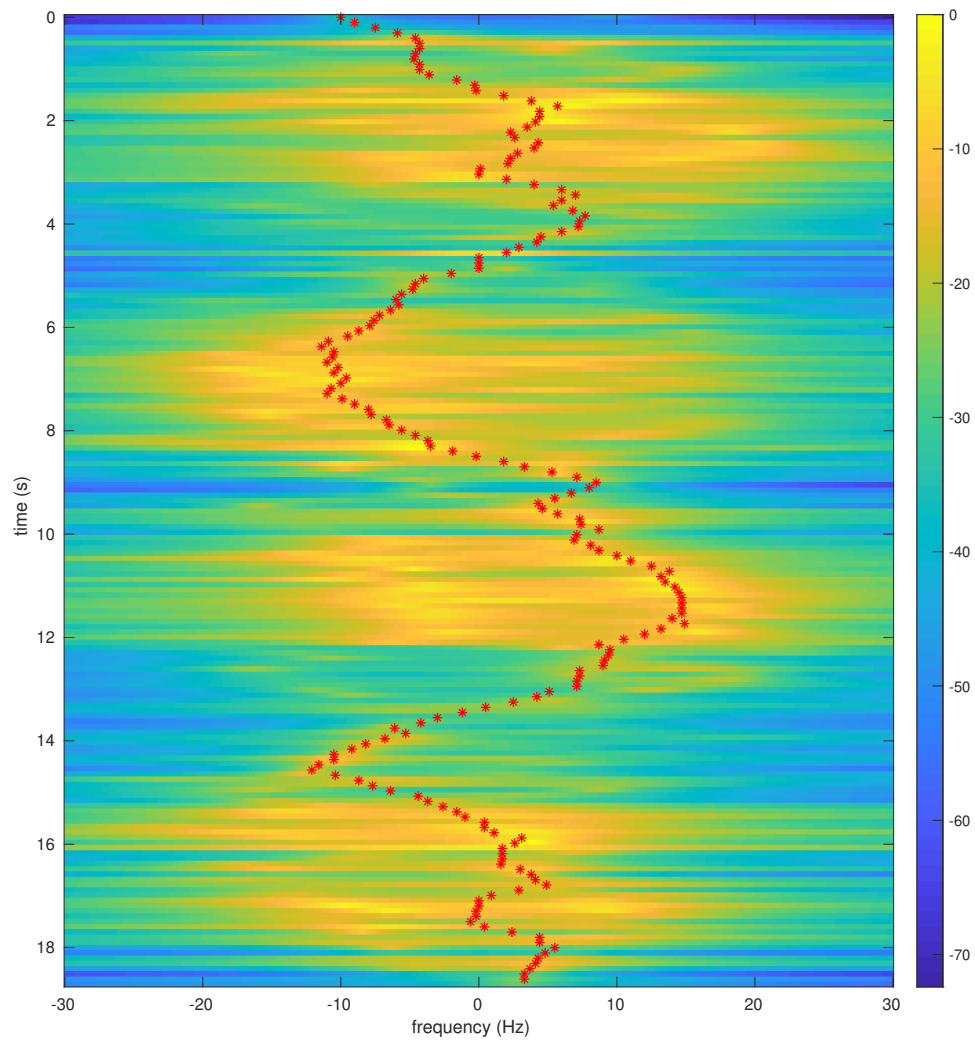


Fig. 5.26: Test 1 with optimal path plotted on MUSIC spectrum, SC antenna.

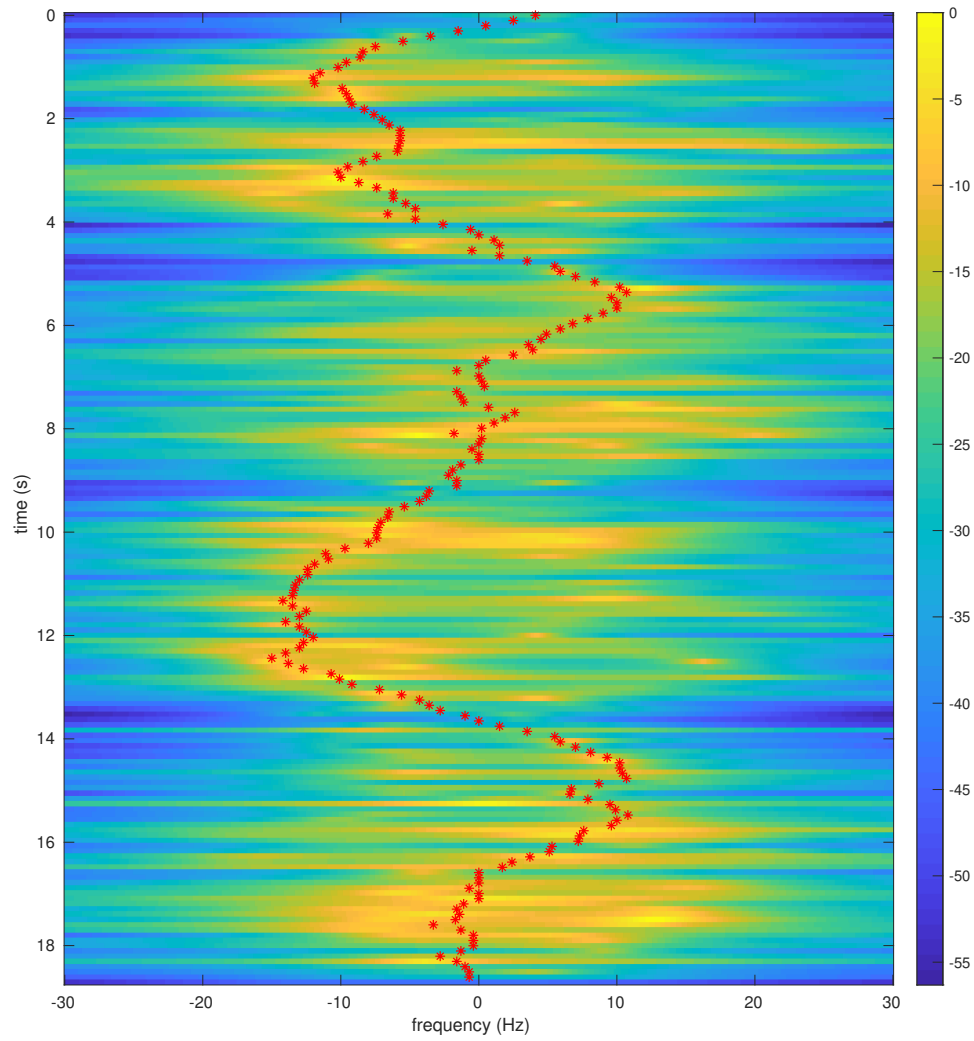


Fig. 5.27: Test 1 with optimal path plotted on MUSIC spectrum, SW antenna.

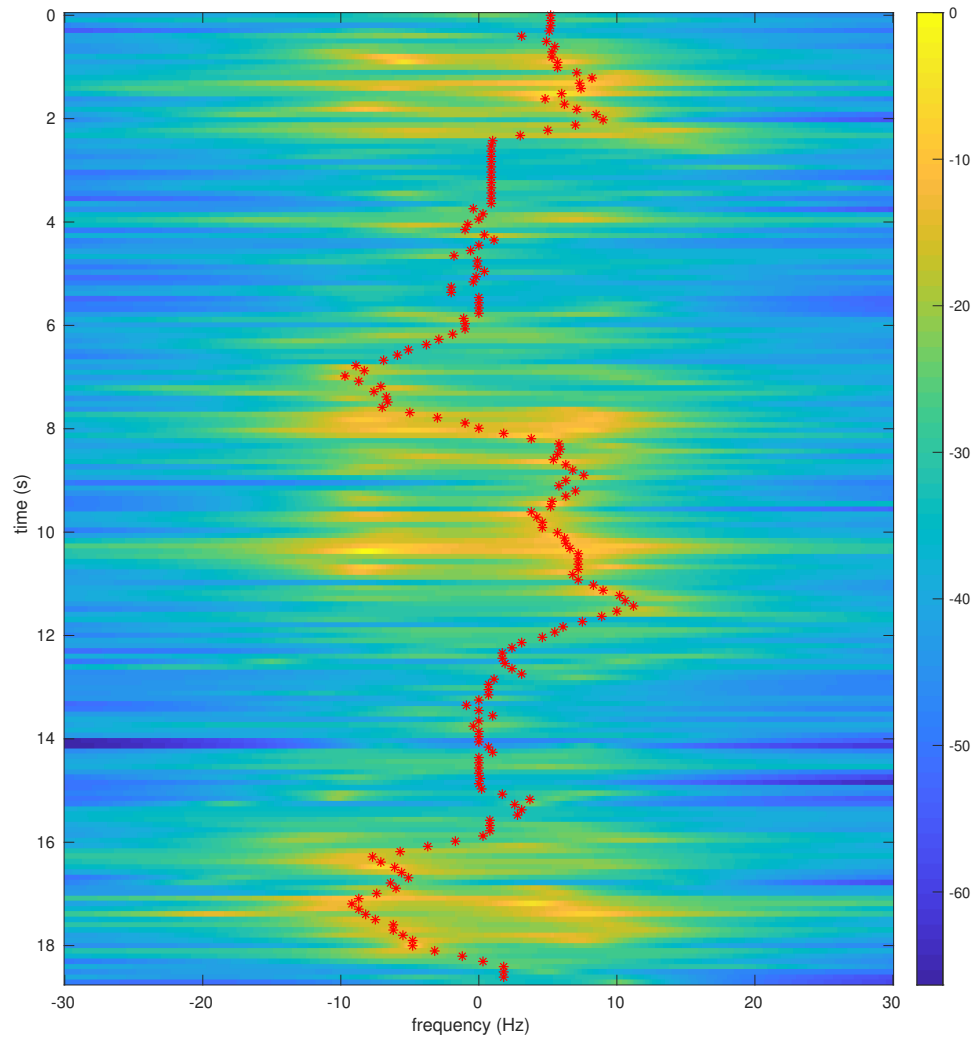


Fig. 5.28: Test 1 with optimal path plotted on MUSIC spectrum, NC antenna.

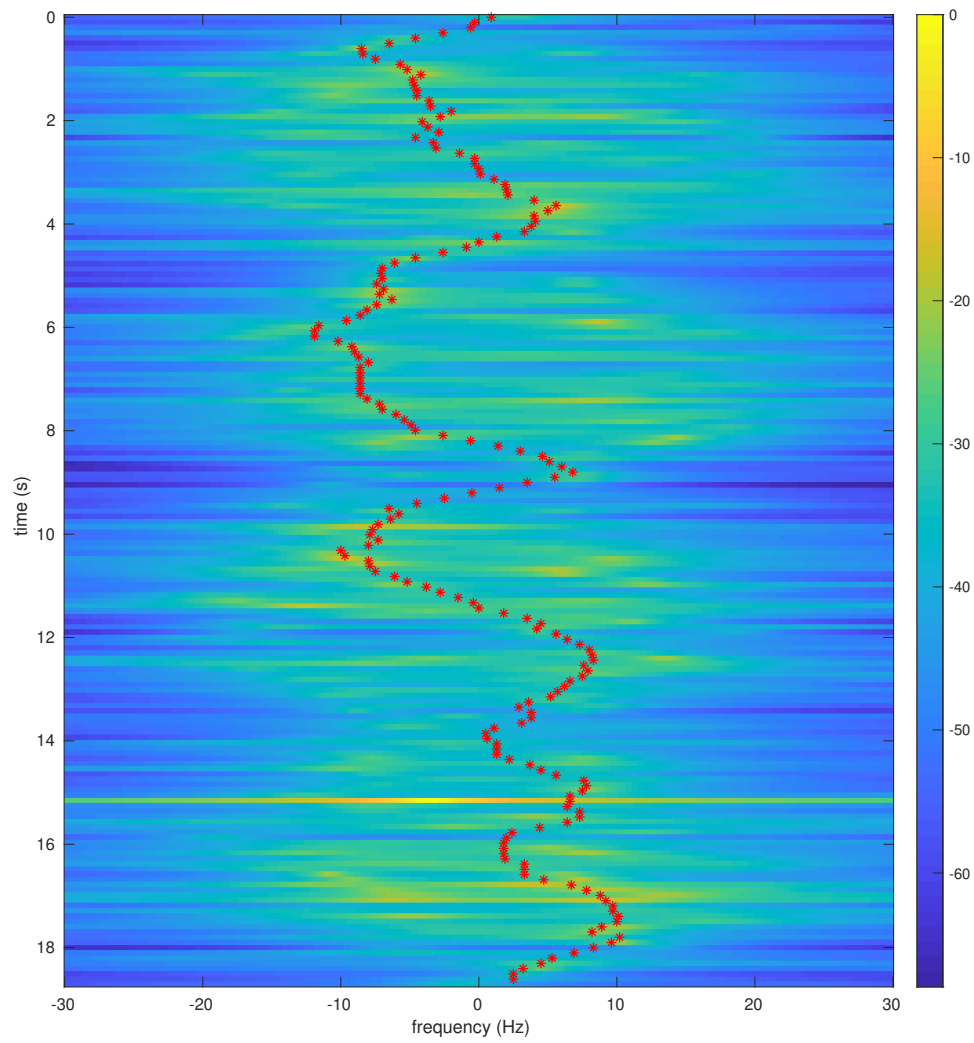


Fig. 5.29: Test 1 with optimal path plotted on MUSIC spectrum, NW antenna.

5.2.10 November 13, 2020

These tests moved the transmitter antenna to a more centralized location. The transmitter was 7.01 m east and 6.4 m north of the SW antenna. This position is in the middle room of the three central rooms of Figure 2.6. The receiver antennas were positioned as described in Section 5.2.6. The setup for these tests is recorded in Tables 5.33 – 5.35 and the tests are recorded in Table 5.36.

There were only two tests in this test set, one with metal and one without. Figures

Table 5.33: Device Parameters, November 13, 2020

Transmitter	B00, Tx/Rx
Transmitter gain	85 dB
Receiver 1	B210-1, RF A, Rx2
Receiver 1 gain	48 dB
Receiver 1 antenna	SC building antenna
Receiver 2	B210-2, RF A, R2x
Receiver 2 gain	64 dB
Receiver 2 antenna	NW building antenna
Receiver 3	B200 mini-1, Rx2
Receiver 3 gain	63 dB
Receiver 3 antenna	SW building antenna
Receiver 4	B200 mini-2, Rx2
Receiver 4 gain	85 dB
Receiver 4 antenna	NC building antenna
Number of amplifiers	2
Sample rate	62500 Samples/s
Interpolation/Decimation rate	4
Carrier frequency	905 MHz
Master clock rate	n/a

Table 5.34: Symbol Timing Synchronization Parameters, November 13, 2020

Samples per symbol	8
Excess bandwidth	0.5
Matched filter length	161
TED	$\text{sgn}(y[n])y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.02
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.35: Carrier phase synchronization parameters, November 13, 2020

$B_n T_s$	90 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

Table 5.36: Tests, November 13, 2020

Test	Metal	Description
1	Yes	Moving between the SW and SC antennas
2	No	Moving between the SW and SC antennas

5.30 – 5.37 compare Test 1 to Test 2 at each antenna. For the SC, SW, and NC antennas, the cases with and without metal are very similar to each other. Some data appears to be lost in the test without metal, Test 2, but overall the general path seems to follow the assumed Doppler path.

However, the NW antenna appears to receive the weakest signal; the algorithm seems to follow an incorrect path at this antenna. In the test with metal, this incorrect path appears to hover around 0 Hz as the balance factor pulls the Doppler path towards this state. In the case without metal, the balance factor does not seem to be as effective. The Doppler path instead follows what may be a false path. However, the correct path is hard to determine as the transmitter location causes the paths to differ significantly from the simulated paths of Figures 5.16 – 5.19.

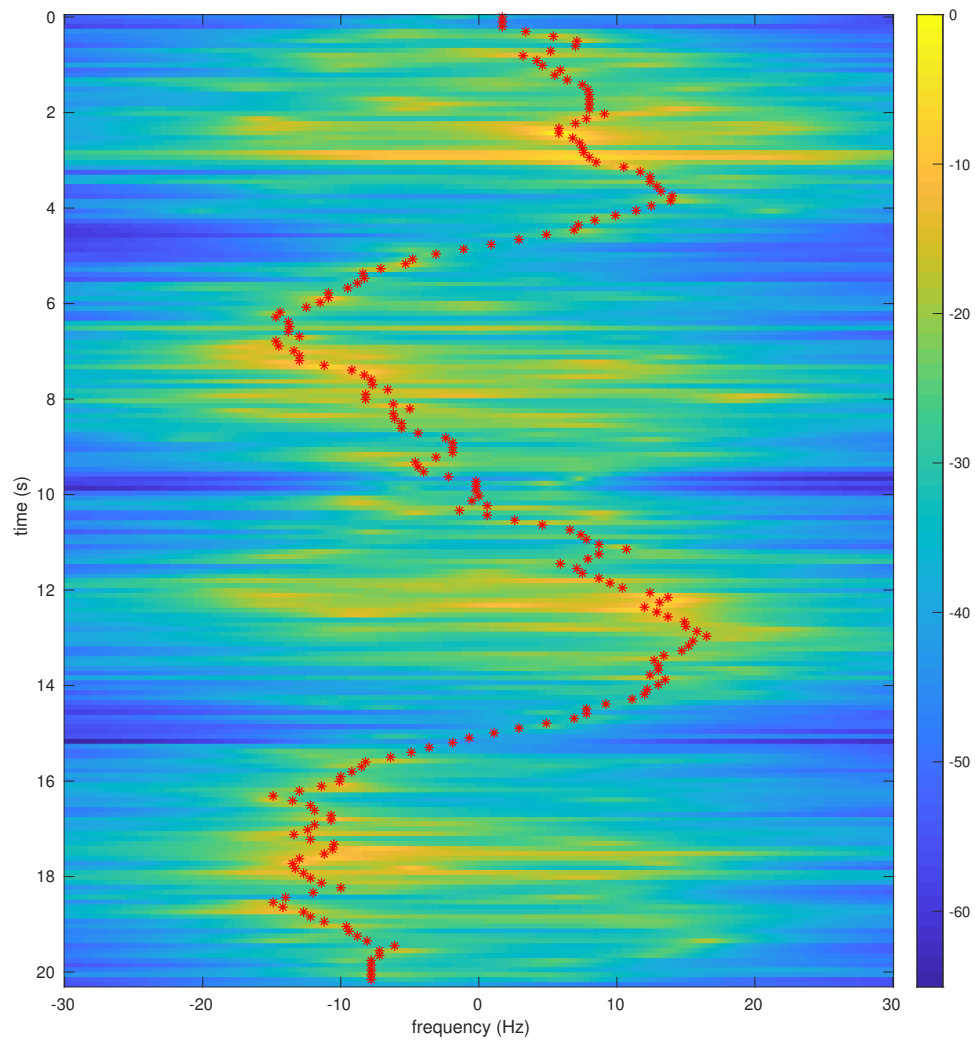


Fig. 5.30: Test 1 with the optimal path plotted on a MUSIC spectrum, SC antenna.

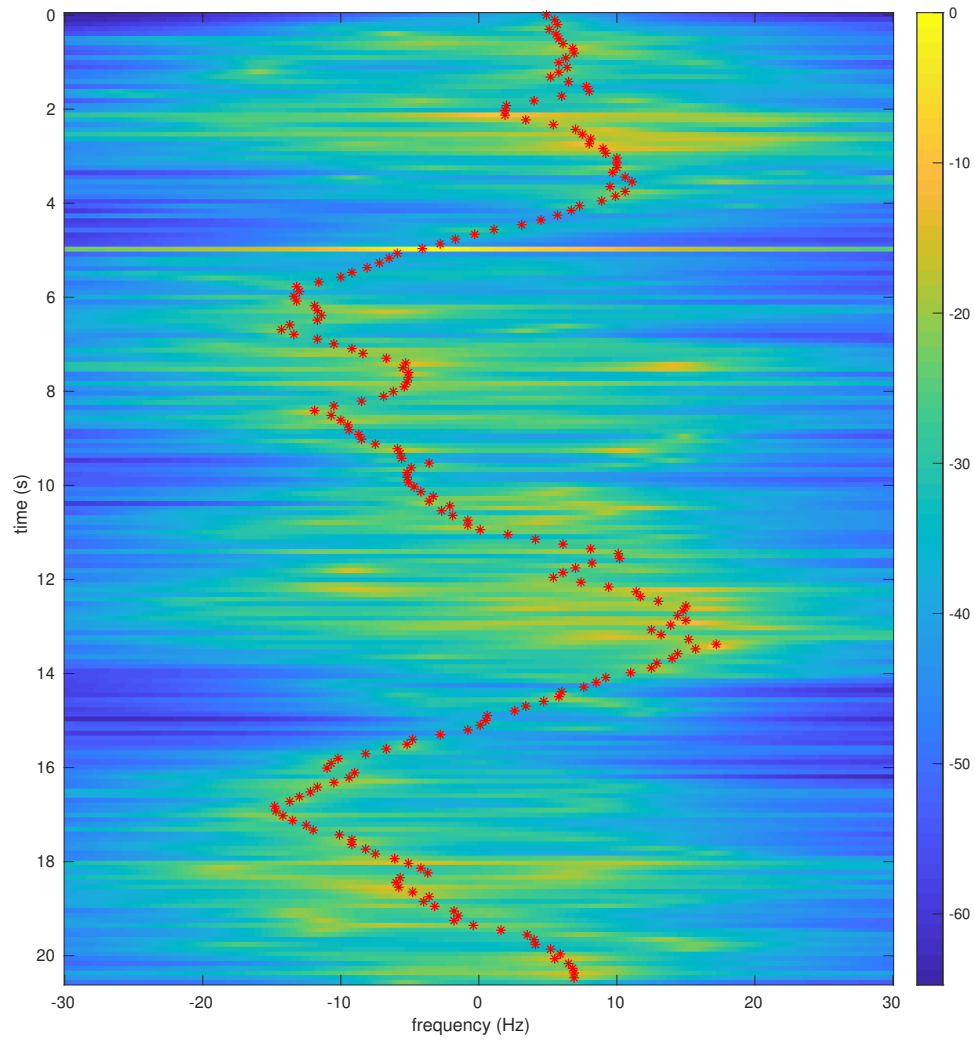


Fig. 5.31: Test 2 with the optimal path plotted on a MUSIC spectrum, SC antenna.

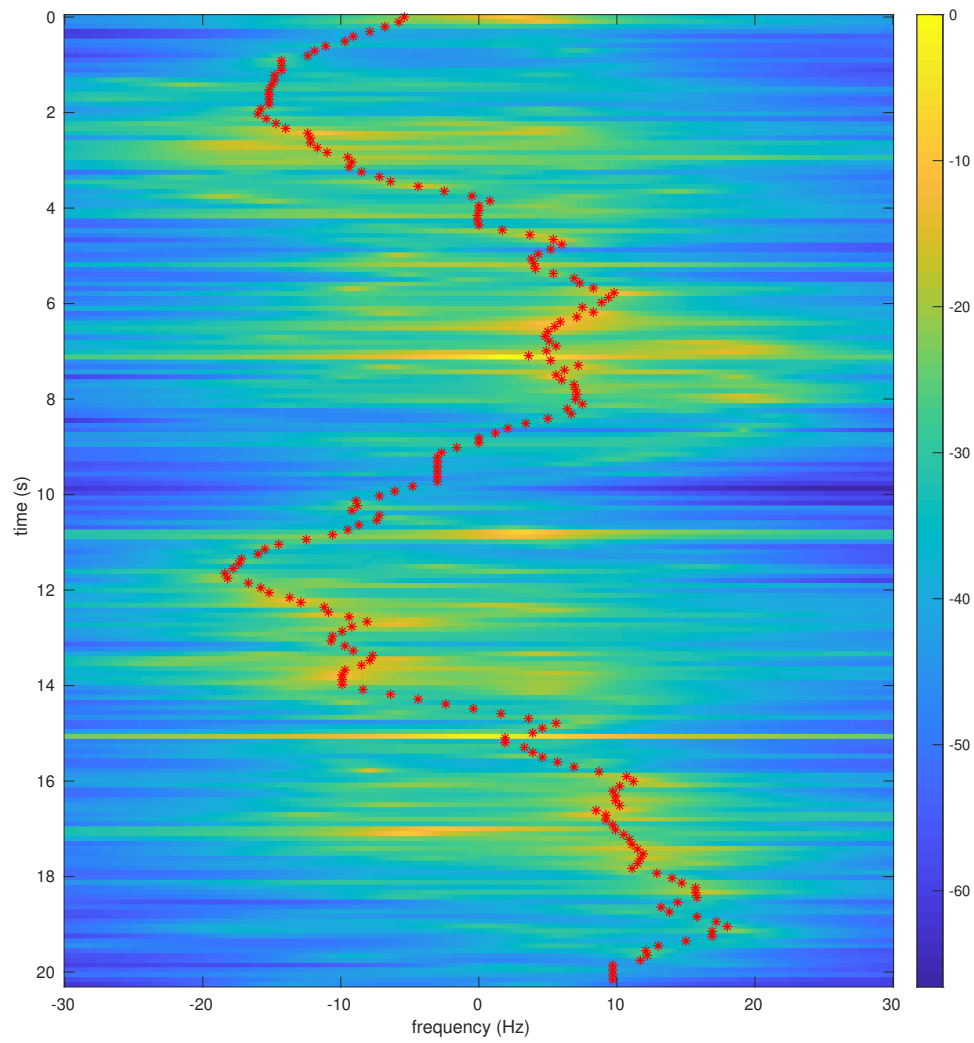


Fig. 5.32: Test 1 with the optimal path plotted on a MUSIC spectrum, SW antenna.

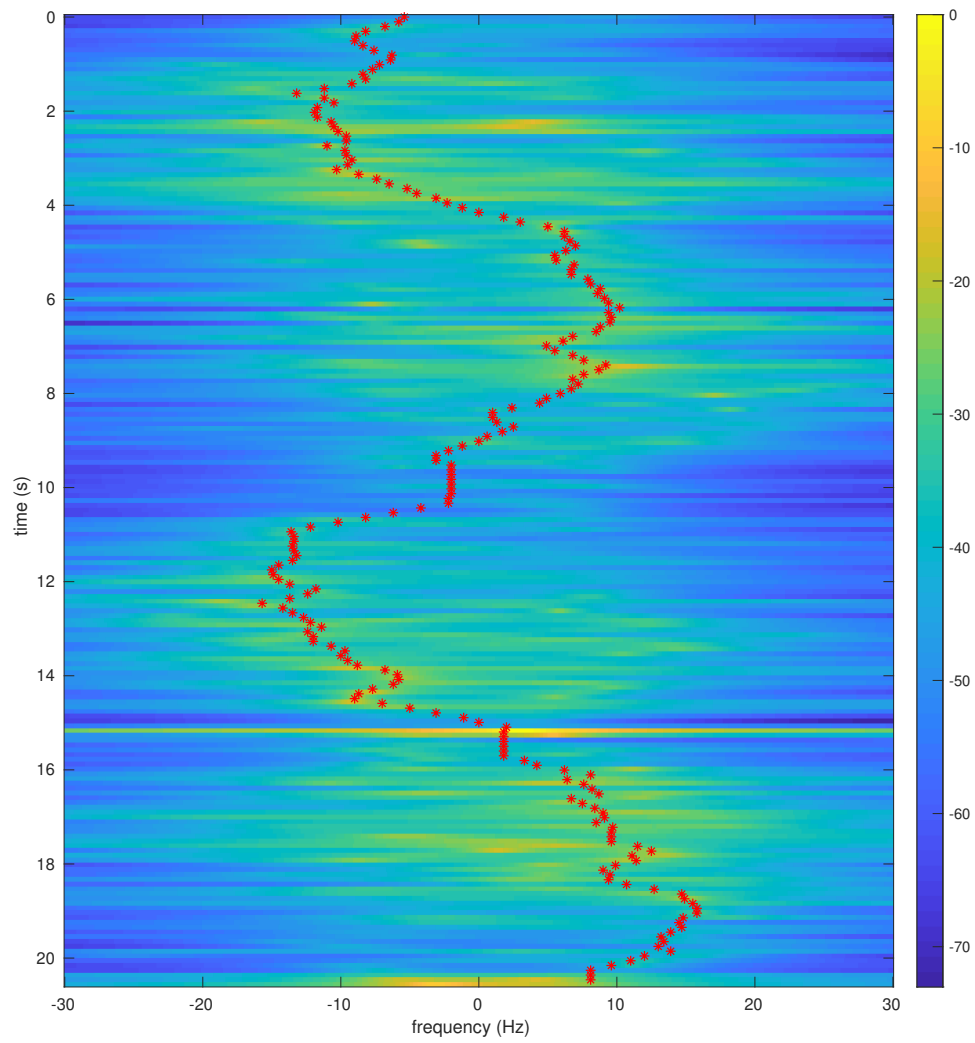


Fig. 5.33: Test 2 with the optimal path plotted on a MUSIC spectrum, SW antenna.

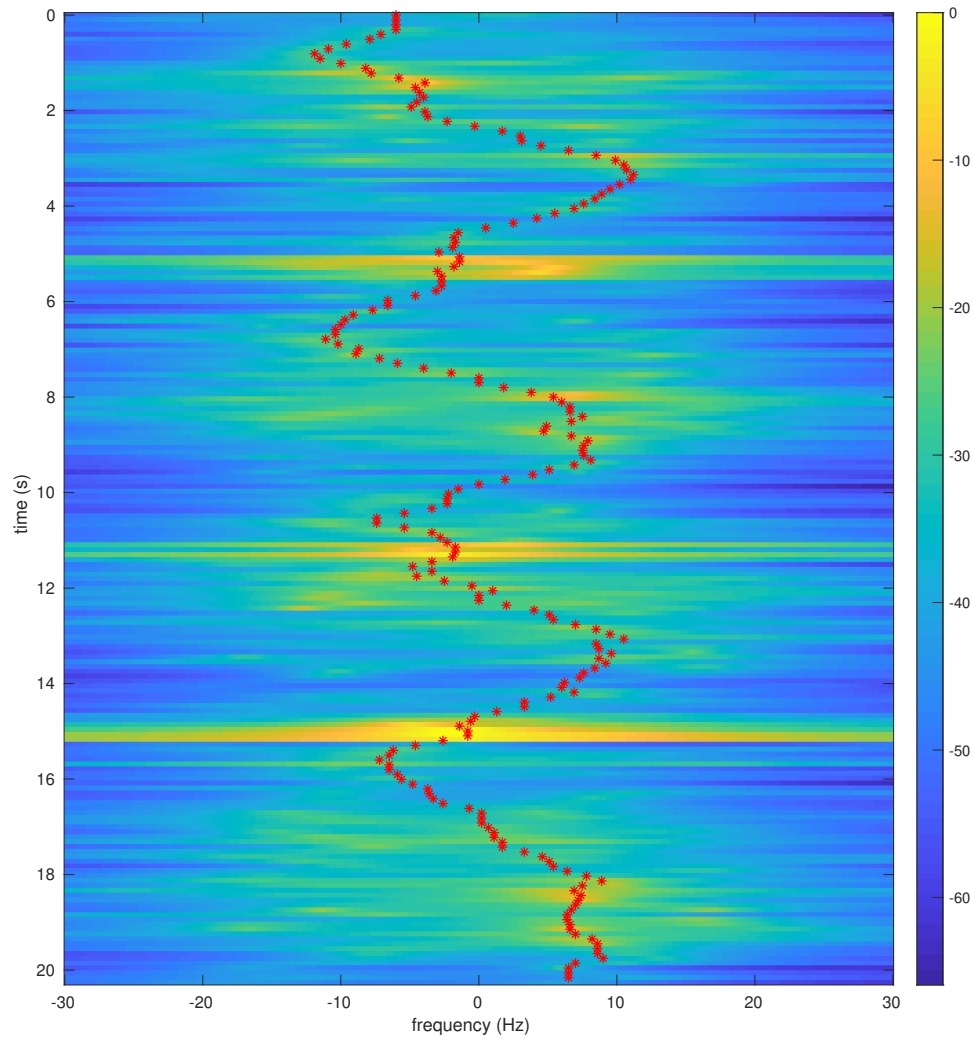


Fig. 5.34: Test 1 with the optimal path plotted on a MUSIC spectrum, NC antenna.

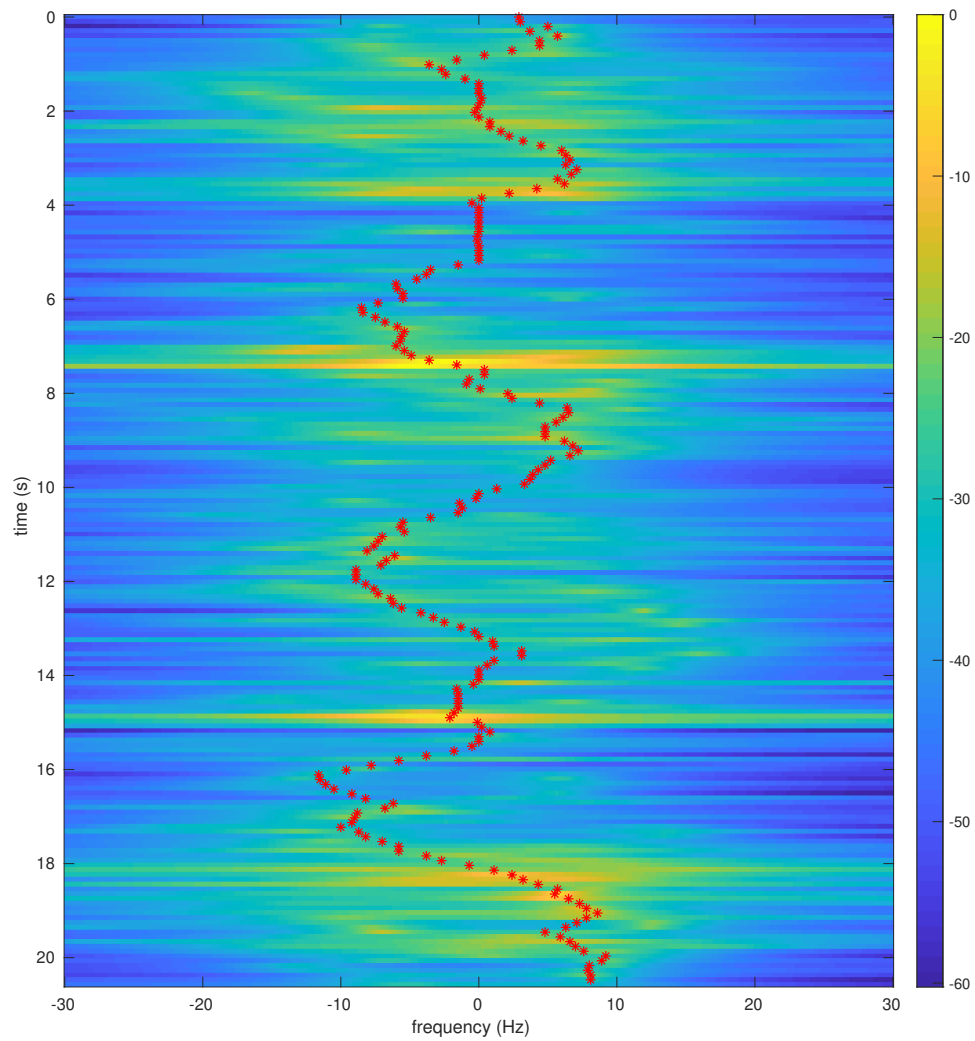


Fig. 5.35: Test 2 with the optimal path plotted on a MUSIC spectrum, NC antenna.

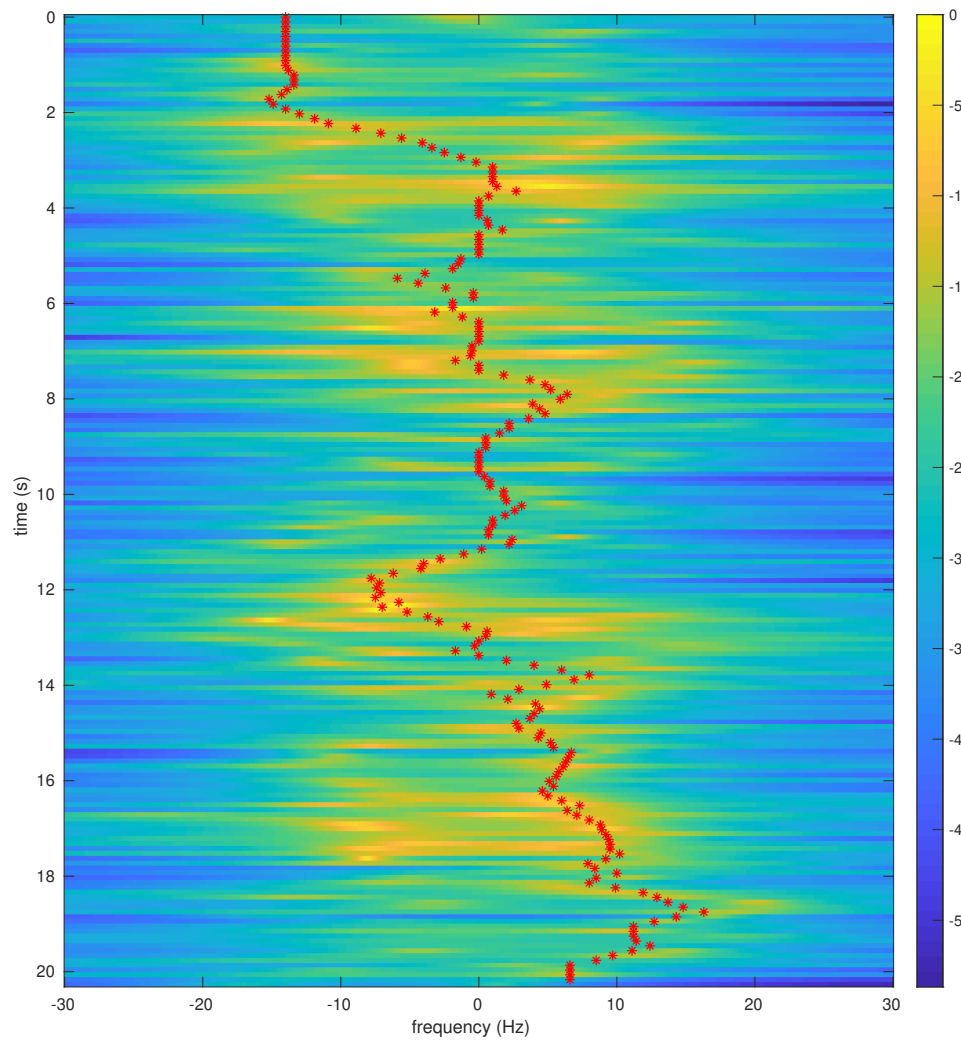


Fig. 5.36: Test 1 with the optimal path plotted on a MUSIC spectrum, NW antenna.

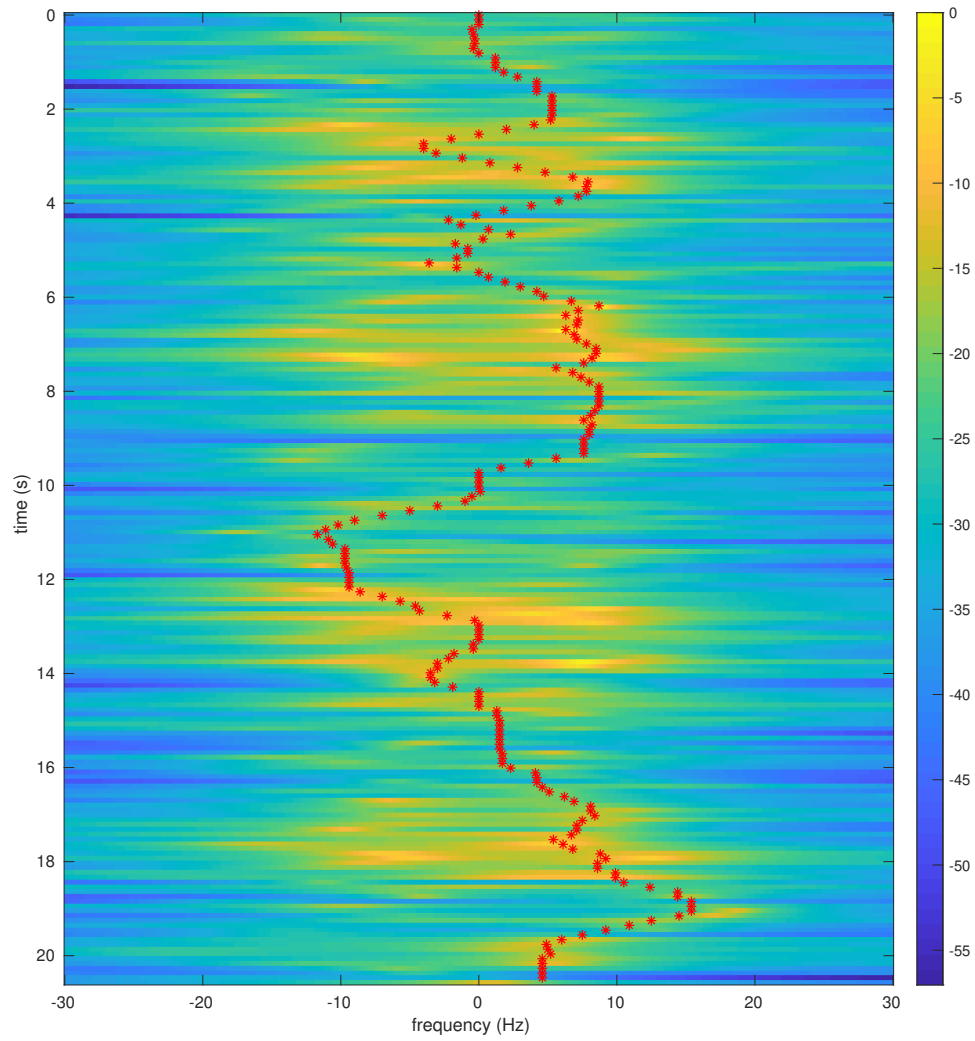


Fig. 5.37: Test 2 with the optimal path plotted on a MUSIC spectrum, NW antenna.

5.2.11 February 12, 2021

The major problem with all the previous tests to this test set was the lack of devices with good clocks. The E320 devices had phase jitter problems on their clocks, and all of the other devices' clocks were also noisy. In addition, each receiver had its own separate device, which had a separate clock from all the other receivers.

The N310s were a solution to this problem. These devices had four channels, each capable of transmitting or receiving data. This allowed one of the devices to act as the receiver for all four antennas simultaneously. In addition, these devices had more consistent clocks.

This test set used one N310 as a transmitter with the transmit antenna placed 8.23 m east and 1.83 m south of the SW antenna. Another N310 was used as the receiver for the NW, SW, NC, and SC antennas.

Figures 5.38 – 5.45 compare the results of Test 0 to the results of Test 4 at each antenna. These results show the N310 devices are more ideal for collecting data.

Table 5.37: Device Parameters, February 12, 2021

Transmitter	N310-2, RF 0, Tx/Rx
Transmitter gain	65 dB
Receiver 1	N310-1, RF 0, Tx/Rx
Receiver 1 gain	65 dB
Receiver 1 antenna	SC building antenna
Receiver 2	N310-1, RF 1, Tx/Rx
Receiver 2 gain	65 dB
Receiver 2 antenna	SW building antenna
Receiver 3	N310-1, RF 2, Tx/Rx
Receiver 3 gain	65 dB
Receiver 3 antenna	NW building antenna
Receiver 4	N310-1, RF 3, Tx/Rx
Receiver 4 gain	65 dB
Receiver 4 antenna	NC building antenna
Number of amplifiers	1
Sample rate	153600 Samples/s
Interpolation/Decimation rate	8
Carrier frequency	905 MHz
Master clock rate	122.88 MHz

Table 5.38: Symbol Timing Synchronization Parameters, February 12, 2021

Samples per symbol	8
Excess bandwidth	0.35
Matched filter length	161
TED	$\text{sgn}(y[n])y'[n]$ Maximum Likelihood
Expected TED gain	1
Loop bandwidth	0.02
Damping factor	1.3
Maximum deviation	1.5
Interpolating resampler	MMSE, 8-tap FIR

Table 5.39: Carrier phase synchronization parameters, February 12, 2021

$B_n T_s$	90 <i>m</i>
ζ	0.707
K_p	0.5
K_0	1

The similarities between the two tests at each antenna, Test 0 with metal and Test 4 without metal, suggest each path is approaching the correct path in both cases. Comparing them to the optimal paths of the simulated paths in Figures 5.16 – 5.19 also show a more similar resemblance than the previous tests had.

Table 5.40: Tests, February 12, 2021

Test	Metal	Description
0	Yes	Moving between the SW and SC antennas, running
1	Yes	Moving between the SW and SC antennas, walking
2	Yes	Moving between the SW and NW antennas, running
3	Yes	Moving between the SW and NW antennas, walking
4	No	Moving between the SW and SC antennas, running
5	No	Moving between the SW and SC antennas, walking
6	No	Moving between the SW and NW antennas, running
7	No	Moving between the SW and NW antennas, walking

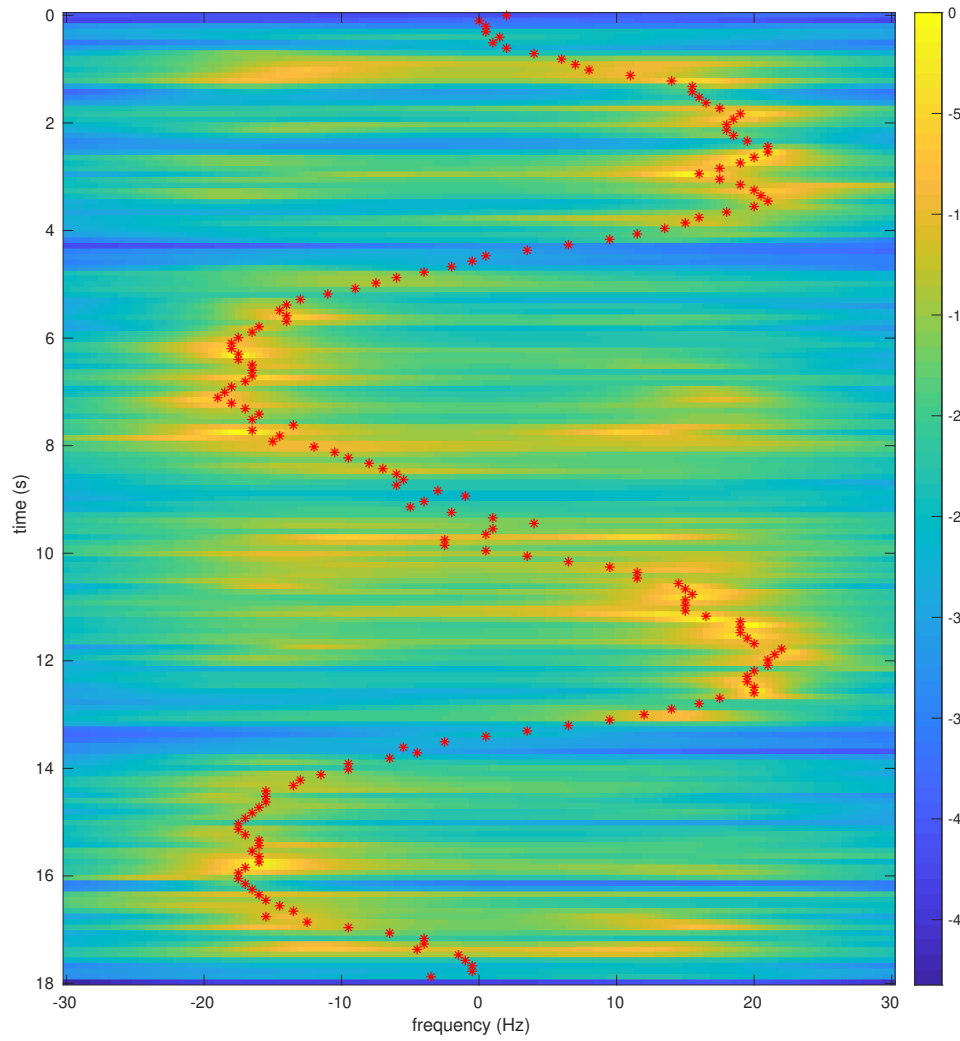


Fig. 5.38: Test 0 with the optimal path plotted on a MUSIC spectrum, SC antenna.

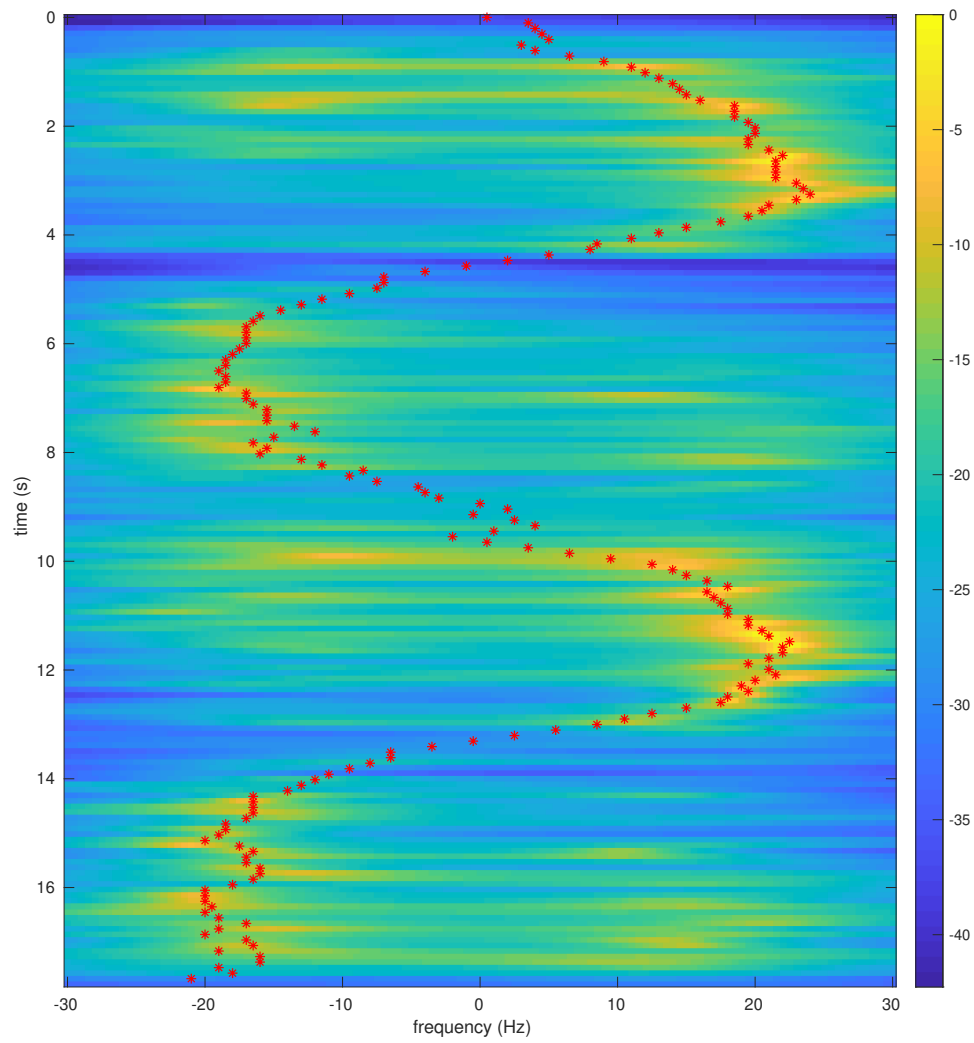


Fig. 5.39: Test 4 with the optimal path plotted on a MUSIC spectrum, SC antenna.

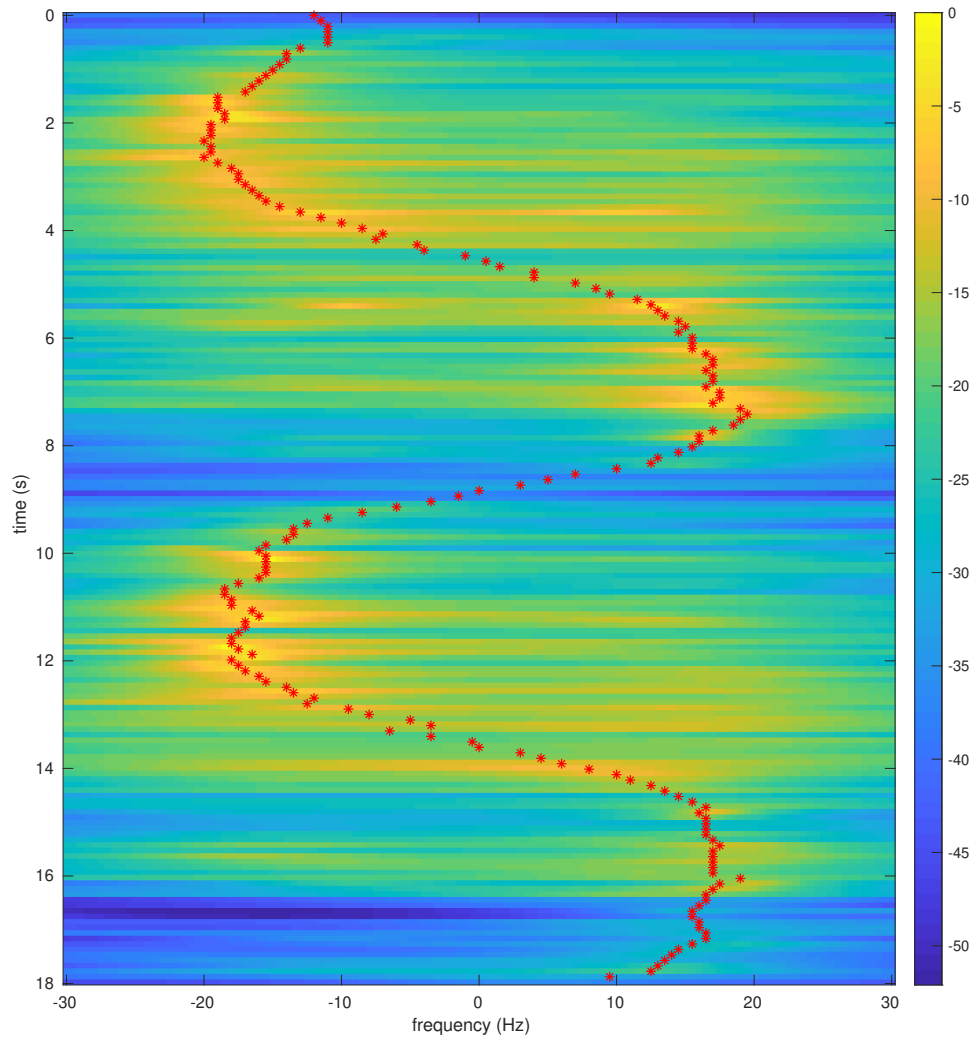


Fig. 5.40: Test 0 with the optimal path plotted on a MUSIC spectrum, SW antenna.

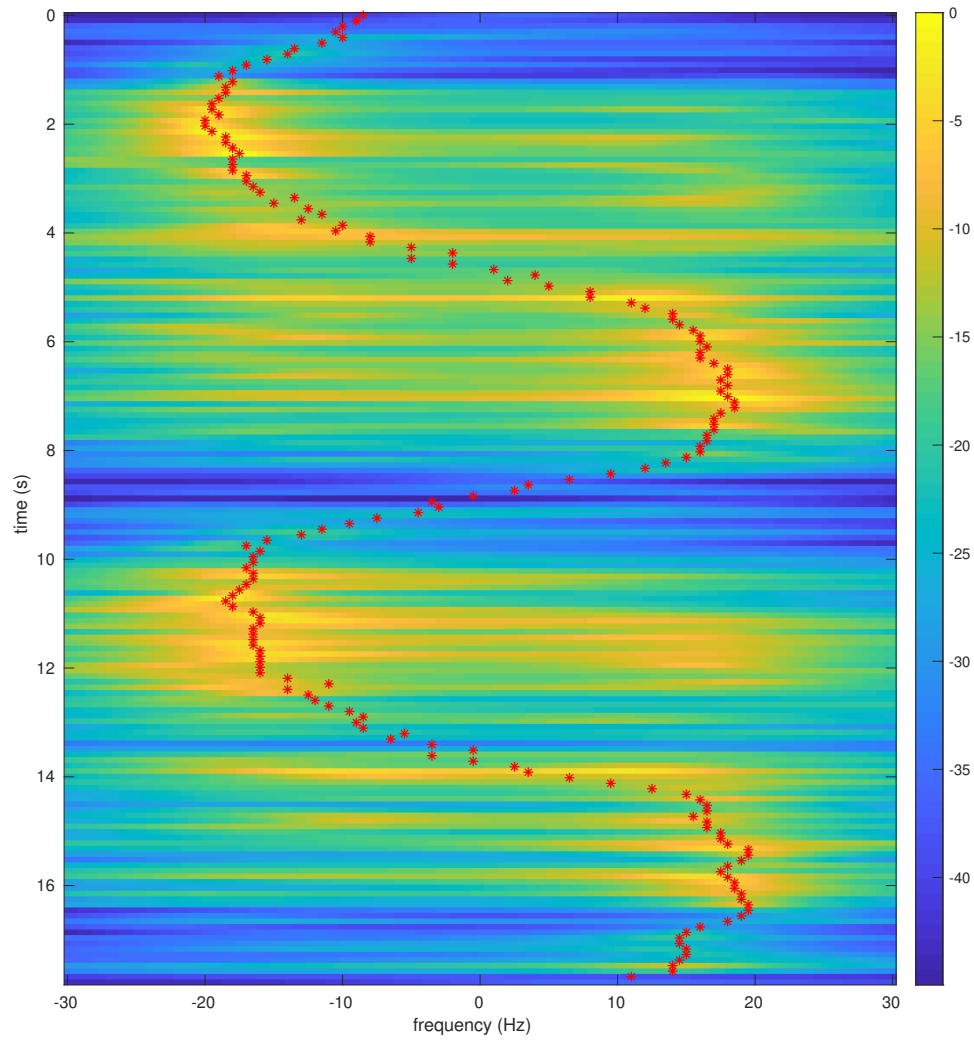


Fig. 5.41: Test 4 with the optimal path plotted on a MUSIC spectrum, SW antenna.

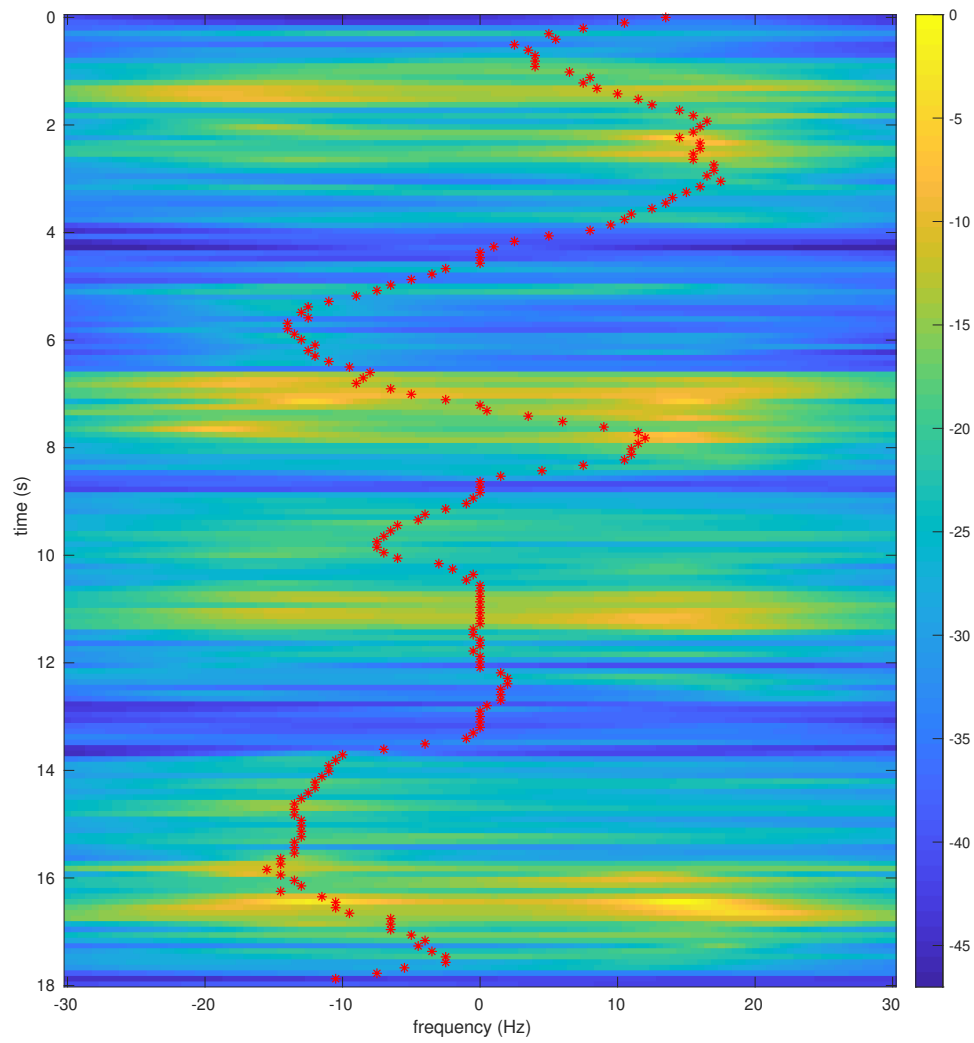


Fig. 5.42: Test 0 with the optimal path plotted on a MUSIC spectrum, NC antenna.

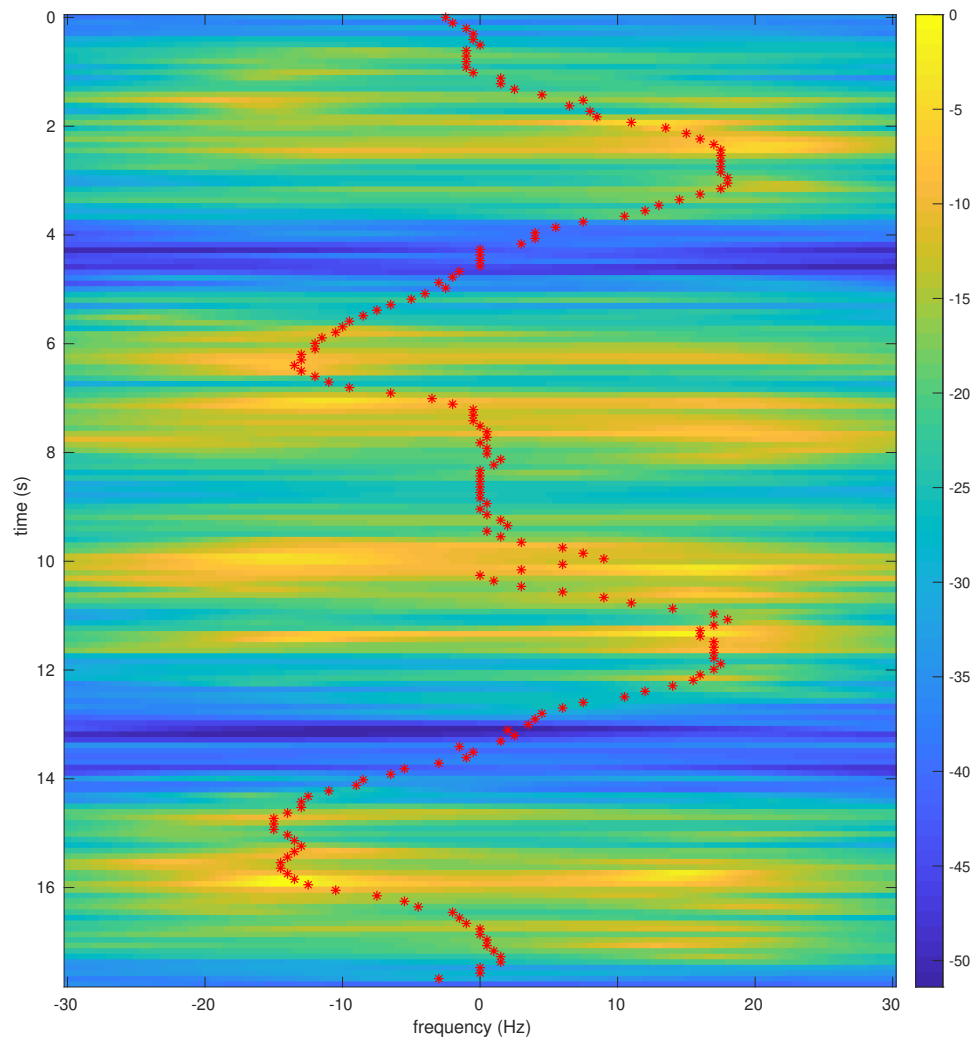


Fig. 5.43: Test 4 with the optimal path plotted on a MUSIC spectrum, NC antenna.

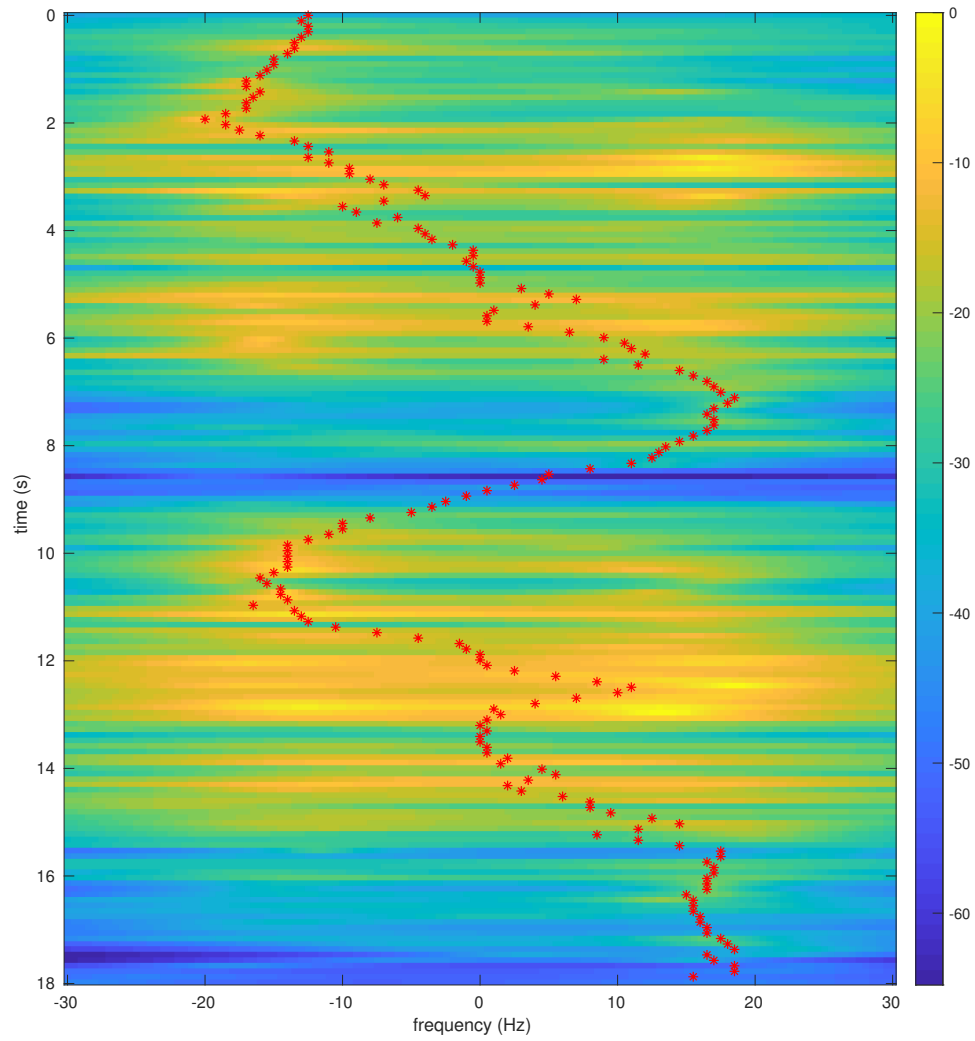


Fig. 5.44: Test 0 with the optimal path plotted on a MUSIC spectrum, NW antenna.

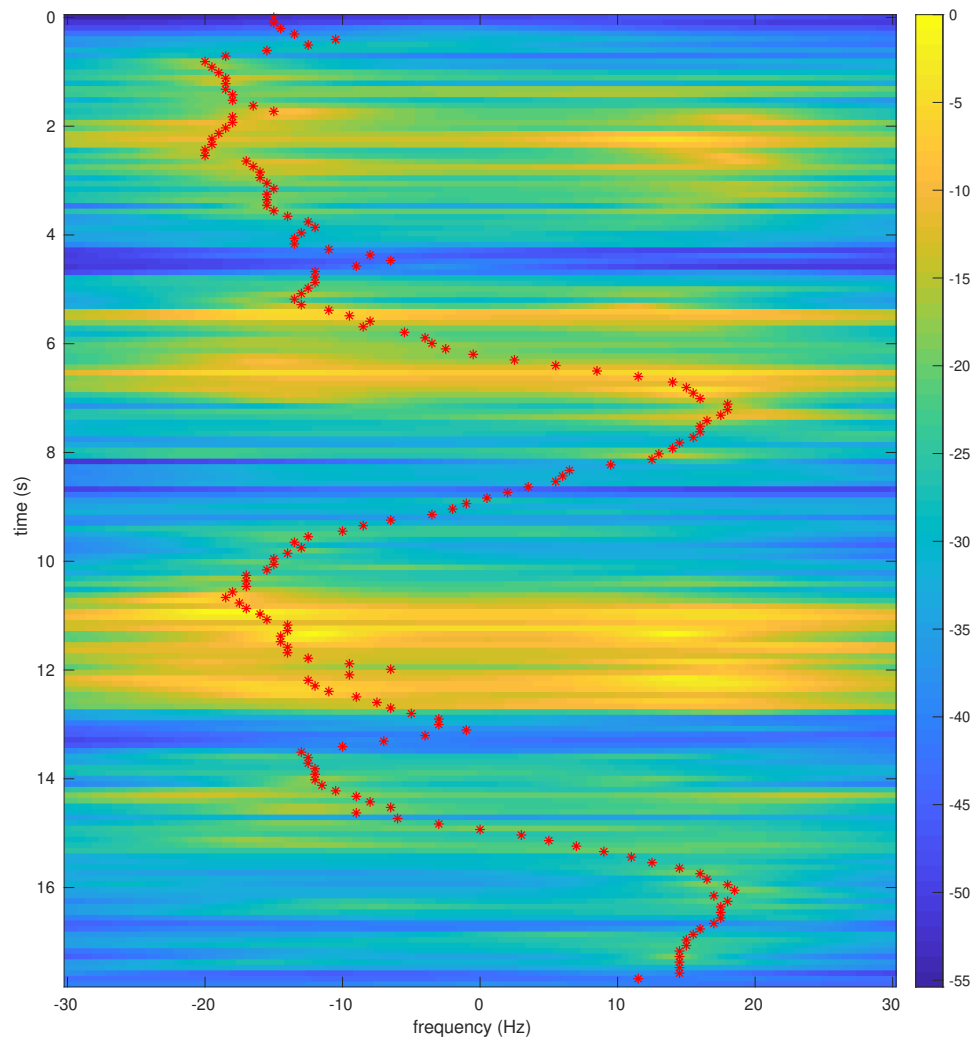


Fig. 5.45: Test 4 with the optimal path plotted on a MUSIC spectrum, NW antenna.

5.3 Discussion

The results of these tests show it is possible to extract the Doppler frequency path from a communication signal without needing a reference signal. The hardware of the system limits the applications this system can perform in, but additional tuning of the system could result in improved results for the less ideal hardware. However, finding a better tuning of all of the parameters is difficult.

In order to best set up and tune the whole system, it is best to split the system into four parts and tune in the following order: the communication system setup, the carrier frequency offset removal tuning, the spectrum estimation tuning, and the path extraction tuning. Each of these pieces can be tuned separately, but it is important to remember that each part significantly affects all the parts following it.

5.3.1 Communication System Setup

Tuning the communication system is of key importance to gathering useful Doppler echo data. The most important piece of this tuning is ensuring the system locks on to the signal constellation with the least noise on the constellation as possible.

The transmitter and receiver devices directly influence the amount of noise on the signal. The best devices are those without much noise or drift on their clocks. The less the carrier frequency offset drifts, the easier it is for the Second DEM to estimate and demodulate this frequency offset from the extracted Doppler signal.

When building the Doppler system, it is also important to keep in mind the placement of the antennas. The results suggest the further apart the receiver, transmitter, and target are from each other, the weaker the signal is. This means it is important to keep all of these as close together as possible. If the application requires them to be far apart, then using an amplifier on the transmitter may improve the results.

The parameters in GNU Radio were best set through trial and error. Since GNU Radio runs in real-time, adjusting these parameters until the signal locks on was done in real-time before collecting the data. Once the correct set of parameters are found, there is

typically little need to change them from test to test. This resulted in the parameters of the communication system changing only slightly from test to test.

5.3.2 Carrier Frequency Offset Removal Tuning

Removing the carrier frequency offset is an important step in extracting the Doppler using the Second DEM. Difficulty in removing the carrier frequency offset results in a noisier spectrogram. The key to removing the carrier frequency offset is finding the proper parameters to get an accurate estimate of the carrier frequency offset without estimating the Doppler frequency.

This becomes a trade-off problem. If the parameters are set to aggressively track the carrier frequency offset, then the Doppler frequency may negatively affect this tracking and result in the spectrogram becoming noisier. This occurs when the path extraction uses the maximum value algorithm or the Viterbi algorithm with a low K value. However, if the parameters are set to slowly track the carrier frequency offset, then the algorithm may not track all of the carrier frequency offset's changes and also result in a noisy spectrogram. This occurs when the path extraction uses the Viterbi algorithm with a high K value.

This is the only situation where a strong Doppler echo negatively affects the system. Usually, it is better to have a stronger Doppler echo, but this can become a problem when estimating the carrier frequency offset. In the situation where the carrier frequency offset is extra noisy, the stronger Doppler signal may negatively affect the system as a whole. Overall, it is best to make sure the carrier frequency offset is the least noisy it can be to make it easier to track despite a strong Doppler echo interfering with the carrier frequency extraction.

5.3.3 Spectrum Estimation Parameter Tuning

The parameters for the spectrum estimation are the hardest part of the system to tune. Each parameter in the spectrum estimation subtly influences the optimal path parameters. Often, this results in adjusting the spectrum estimation parameters at the same time as the path extraction parameters.

The MUSIC algorithm is the preferred method used among all the tests due to the ability to improve spectral resolution. All but six of the spectrograms in this chapter use this method, as it often resulted in a better path. However, the DFT was often used while generating these results to help determine the best parameters for the MUSIC algorithm.

The MUSIC algorithm works well when it is properly tuned. When MUSIC is not properly tuned, it can easily give erroneous results. It is hard to determine from looking at a MUSIC spectrogram if the results are poor, as in some of the later tests it became difficult to determine the Doppler path from simply looking at the spectrogram. The DFT is a better algorithm for seeing what is going on exactly, even with less resolution. Therefore, it is often better to compare the spectrograms from both algorithms side by side when tuning the MUSIC algorithm.

5.3.4 Doppler Path Extraction Parameter Tuning

Tuning the parameters for the Doppler path extraction is more straightforward than tuning the parameters for the spectrum estimation; it is easier to see the change each parameter makes, but it can be a little overwhelming given how many different parameters there are. There are nine parameters to adjust when using the BCJR algorithm with the balance factor adjustment. However, the Viterbi parameters K and δ can often be set to the same value as the BCJR parameters K_1 and δ . This is because these parameters control essentially the same things in both algorithms. Both K and K_1 control the weight on the branch metric, while δ in both algorithms controls the allowed state transitions.

Situations may arise where it is advantageous to give one algorithm more freedom to allow transitions while keeping the other more restrictive, but usually it is better to keep them at the same value. BCJR makes some improvements upon Viterbi, but in many ways it is redundant.

To also simplify the number of parameters, it is useful to set either K_1 or K_2 and then only adjust the other. This is because the ratio of these weights is more important than the magnitude of these weights. Therefore, by making one of these constant and changing the other, the best path can more easily be found.

Most of the balance factor parameters can usually be set and left alone. The parameters σ , Λ , and K_3 all control the shape of the artificial spectrum. These parameters can often take some trial and error to get correct. K_0 changes how much weight is given to the balance factor and can also be set through trial and error.

ϵ is the most important parameter for the balance factor, as it determines when to apply the effects. The key is to apply the balance factor when it is most probable for the Doppler to be near 0 Hz. This occurs when the balance factor is close to 0. However, this can change easily from test to test depending on many factors. Therefore, this parameter is the first to change when there appears to be problems with the path not going to DC when it should.

CHAPTER 6

CONCLUSION

The purpose of this thesis was to explore a new method for extracting the Doppler frequencies from a communication signal. Traditionally, the Doppler frequencies are extracted using the CAF function, which requires the separation of the Doppler echo and the direct path signal [12]. This research studied a new way of performing the Doppler extraction to avoid the need for this separation. This new method takes advantage of the communication signal structure.

The First DEM was tested using a GNU Radio communication system. The results revealed that this method only worked when the transmitter and receiver had synchronized clocks. However, the Second DEM was developed for the case when the transmitter and receiver did not have synchronized clocks. This allowed the system to work in the more realistic case.

GNU Radio used the First DEM and Second DEM to extract the Doppler signal, but extracting the Doppler frequency path from this signal was difficult due to the low SIR and SNR of the signal. Additional processing was developed and implemented in MATLAB in order to improve the Doppler frequency path extraction. The key processing techniques used were the MUSIC algorithm, the Viterbi algorithm, and a balance factor adjusted BCJR algorithm.

The MUSIC algorithm was able to produce a higher resolution frequency spectrum by removing some of the noise on the signal. The Viterbi algorithm was able to find a smooth continuous path of highest likelihood through the Doppler echo spectrogram. The balance factor adjusted BCJR algorithm was able to make up for the loss of the Doppler path when the Doppler echo was at 0 Hz. Together, these algorithms were able to successfully extract the expected Doppler frequency path, as shown in Chapter 5.

This thesis also explored the limitations of this system. This system is limited by the

strength of the Doppler echo and the phase noise on the carrier frequency offset. Ensuring the system has good receivers and limiting the size of the tracking area allows the system to overcome these limitations. This system also is difficult to tune because of the number of parameters for adjusting the algorithms in the system. More time optimizing the system could lead to improved results.

The purpose for extracting the Doppler echo is for indoor target tracking. Once the Doppler echo is extracted, a Kalman filter or a particle filter can use the Doppler echo to track the location of the moving target. The Doppler echo from multiple receivers is required for this tracking using either of the filters. This thesis explored the extraction of the Doppler frequency path with the goal of extracting this path at multiple receivers to use in the indoor tracking system. Implementing the tracking system as described in the paper by Chen, et al. using this Doppler data is the next step in the research [3].

Figure 6.1 shows an initial tracking plot using data from Test 0 of the February 12, 2021 test set. This test involved a person carrying an aluminum sheet jogging in between the SW and SC receivers twice. This plot was generated using an extended Kalman filter modified to correct the path from going outside of the marked hallways. The unfilled circle denotes the starting point of the person jogging up and down the hall while the star represents the ending point. This initial plot suggests promising results for future research using this new Doppler extraction system. Additional work can be done to improve the extending Kalman filter and overall indoor human tracking system.

There are also other algorithms and techniques to explore to improve this Doppler extraction system. The Second DEM was developed to overcome the carrier frequency offset problems the First DEM suffered from. However, there may be ways to overcome the problems of the First DEM. Section 4.4 explored one possibility, but more research into this method and other methods would be beneficial.

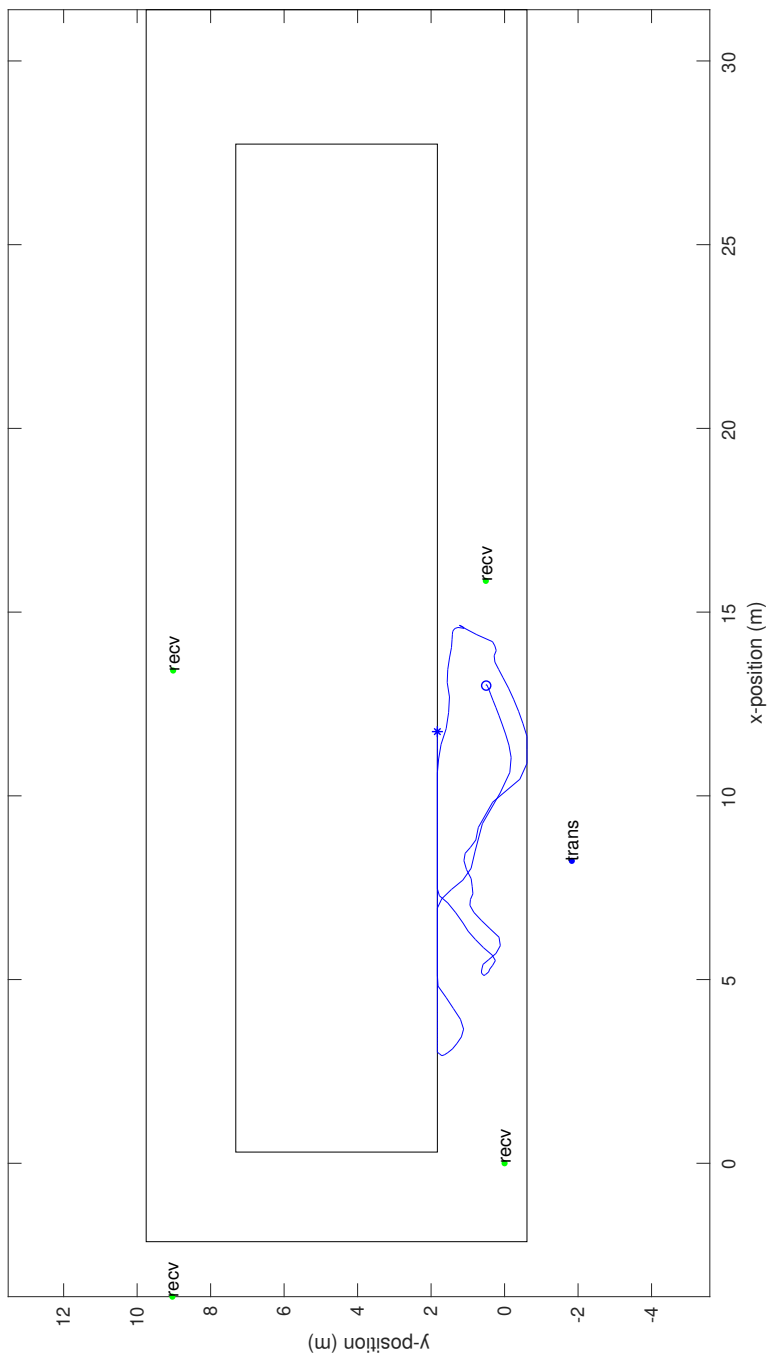


Fig. 6.1: Initial target tracking test using Doppler extracted data from the February 12, 2021 tests.

REFERENCES

- [1] H. Griffiths, “Developments in bistatic and networked radar,” in *Proceedings of 2011 IEEE CIE International Conference on Radar*, vol. 1, 2011, pp. 10–13.
- [2] K. Chetty, G. E. Smith, and K. Woodbridge, “Through-the-wall sensing of personnel using passive bistatic WiFi radar at standoff distances,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 50, no. 4, pp. 1218–1226, 2012.
- [3] Q. Chen, B. Tan, K. Woodbridge, and K. Chetty, “Indoor target tracking using high Doppler resolution passive Wi-Fi radar,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5565–5569.
- [4] B. Tan, A. Burrows, R. Piechocki, I. Craddock, Q. Chen, K. Woodbridge, and K. Chetty, “Wi-Fi based passive human motion sensing for in-home healthcare applications,” in *2015 IEEE 2nd World Forum on Internet of Things (WF-IoT)*, 2015, pp. 609–614.
- [5] Y. Li, X. Wang, and Z. Ding, “Multi-target position and velocity estimation using OFDM communication signals,” *IEEE Transactions on Communications*, vol. 68, no. 2, pp. 1160–1174, 2020.
- [6] Chenqi Zhang, Yong Wu, J. Wang, and Zhen Luo, “FM-based multi-frequency passive radar system,” in *2016 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC)*, 2016, pp. 1–4.
- [7] P. Zhang, Y. Wu, J. Wang, and J. Qiao, “Real-time signal processing for FM-based passive bistatic radar using GPUs,” in *2014 19th International Conference on Digital Signal Processing*, 2014, pp. 536–540.
- [8] P. E. Howland, D. Maksimiuk, and G. Reitsma, “FM radio based bistatic radar,” *IEEE Proceedings - Radar, Sonar and Navigation*, vol. 152, no. 3, pp. 107–115, 2005.
- [9] M. Edrich, A. Schroeder, and F. Meyer, “Design and performance evaluation of a mature FM/DAB/DVB-T multi-illuminator passive radar system,” *IET Radar, Sonar Navigation*, vol. 8, no. 2, pp. 114–122, 2014.
- [10] W. Li, B. Tan, and R. Piechocki, “Opportunistic Doppler-only indoor localization via passive radar,” in *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*, 2018, pp. 467–473.
- [11] I. Milani, F. Colone, C. Bongioanni, and P. Lombardo, “WiFi emission-based vs passive radar localization of human targets,” in *2018 IEEE Radar Conference (RadarConf18)*, 2018, pp. 1311–1316.

- [12] B. Tan, K. Woodbridge, and K. Chetty, "A real-time high resolution passive WiFi Doppler-radar and its applications," in *2014 International Radar Conference*, 2014, pp. 1–6.
- [13] L. Schauer, P. Marcus, and C. Linnhoff-Popien, "Towards feasible Wi-Fi based indoor tracking systems using probabilistic methods," in *2016 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*, 2016, pp. 1–8.
- [14] Y. Lai, C. Chou, M. Tang, T. Horng, and F. Wang, "Finger gesture sensing and recognition using a Wi-Fi-based passive radar," in *2019 IEEE MTT-S International Microwave Symposium (IMS)*, 2019, pp. 293–296.
- [15] M. Rice, *Digital Communications: A Discrete-Time Approach*. Self-published, 2018.
- [16] N. J. Willis, *Bistatic Radar*. Artech House, 1991.
- [17] R. W. S. Oppenheim, Alan V., *Discrete-Time Signal Processing*. Pearson Higher Education, Inc., 2010.
- [18] T. K. Moon and W. C. Stirling, *Mathematical Methods and Algorithms for Signal Processing*. Prentice Hall, 2000.
- [19] T. K. Moon, *Error Correction Coding*. John Wiley & Sons, Inc., 2005.

APPENDICES

APPENDIX A

Test Parameters

The following pages contain the spectrum generation and path extraction parameters used to generate the figures in Chapter 5. These parameters are kept in three separate tables.

The first table, Table A.1, lists all the parameters needed for the carrier frequency offset removal. Only tests using the Second DEM need to remove the carrier frequency offset. The parameters for this removal are the type of spectrogram generated (DFT or MUSIC), the step size between each line of the spectrogram, the length of the data frame for each line of the spectrogram, and the resolution of the spectrogram. In addition, if MUSIC was used the P and M values are recorded, and if the Viterbi algorithm was used for the path extraction its parameters are also recorded.

The second table, Table A.2, lists all the parameters needed for the Doppler signal spectrum estimation. The parameters for this generation are the type of spectrogram generated (DFT or MUSIC), the step size between each line of the spectrogram, the length of the data frame for each line of the spectrogram, and the resolution of the spectrogram. In addition, if MUSIC was used the P and M values are recorded.

The third table, Table A.3, lists all the parameters needed for the Doppler frequency path extraction. It lists the algorithms used to extract the path and any parameters for the algorithms.

All three tables list the Doppler extraction method for the test. Any unused parameters in the tables will list a n/a to note it was not used.

Table A.1: Carrier frequency offset removal parameters

Figure	Method	Type	Step Length (s)	Frame Length (s)	Resolution (Hz)	P/M	Path	K	δ
5.3	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.4	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.5	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.6	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.7	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.8	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.9	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.10	1	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.11	2	DFT	0.1	0.2	0.1	n/a	max	n/a	n/a
5.12	2	MUSIC	0.1	0.2	0.1	1/5	max	n/a	n/a
5.13	2	MUSIC	0.1	0.2	0.1	1/5	max	n/a	n/a
5.14	2	DFT	0.1	0.2	0.1	n/a	Viterbi	0	0.5
5.15	2	DFT	0.1	0.2	0.1	n/a	Viterbi	0	0.5
5.20	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	20	0.5

continued on next page

Table A.1: Carrier frequency offset removal parameters - *continued from previous page*

Figure	Method	Type	Step Length (s)	Frame Length (s)	Resolution (Hz)	P/M	Path	K	δ
5.21	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	20	0.5
5.22	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.23	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.24	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.25	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.26	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.27	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.28	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.29	2	MUSIC	0.1	0.2	0.1	1/5	Viterbi	30	0.5
5.30	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5
5.31	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5
5.32	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5
5.33	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5
5.34	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5

continued on next page

Table A.1: Carrier frequency offset removal parameters - *continued from previous page*

Figure	Method	Type	Step Length (s)	Frame Length (s)	Resolution (Hz)	P/M	Path	K	δ
5.35	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5
5.36	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5
5.37	2	MUSIC	0.1	0.2	0.1	1/6	Viterbi	50	0.5
5.38	2	MUSIC	0.05	0.05	0.1	1/5	Viterbi	30	0.25
5.40	2	MUSIC	0.05	0.05	0.1	1/5	Viterbi	30	0.25
5.42	2	MUSIC	0.05	0.05	0.1	1/5	Viterbi	30	0.25
5.44	2	MUSIC	0.05	0.05	0.1	1/5	Viterbi	30	0.25

Table A.2: Spectrum estimation parameters

Figure	Method	Type	Step Length (s)	Frame Length (s)	Resolution (Hz)	P/M
5.3	1	DFT	0.1	0.2	0.1	n/a
5.4	1	DFT	0.1	0.2	0.1	n/a
5.5	1	DFT	0.1	0.2	0.1	n/a
5.6	1	DFT	0.1	0.2	0.1	n/a
5.7	1	DFT	0.1	0.2	0.1	n/a
5.8	1	MUSIC	0.1	0.2	0.1	50/200
5.9	1	MUSIC	0.1	0.2	0.1	50/200
5.10	1	MUSIC	0.1	0.2	0.1	50/200
5.11	2	MUSIC	0.1	0.2	0.1	50/200
5.12	2	MUSIC	0.1	0.2	0.1	70/200
5.13	2	MUSIC	0.1	0.2	0.1	70/200
5.14	2	MUSIC	0.1	0.2	0.1	70/200
5.15	2	MUSIC	0.1	0.2	0.1	70/200
5.20	2	MUSIC	0.1	0.2	0.1	70/250
5.21	2	MUSIC	0.1	0.2	0.1	70/250
5.22	2	MUSIC	0.1	0.2	0.1	70/200
5.23	2	MUSIC	0.1	0.2	0.1	70/200
5.24	2	MUSIC	0.1	0.2	0.1	70/200
5.25	2	MUSIC	0.1	0.2	0.1	70/200
5.26	2	MUSIC	0.1	0.2	0.1	70/200
5.27	2	MUSIC	0.1	0.2	0.1	70/200
5.28	2	MUSIC	0.1	0.2	0.1	70/200
5.29	2	MUSIC	0.1	0.2	0.1	70/200

continued on next page

Table A.2: Spectrum estimation parameters - *continued from previous page*

Figure	Method	Type	Step Length (s)	Frame Length (s)	Resolution (Hz)	P/M
5.30	2	MUSIC	0.1	0.2	0.1	70/200
5.31	2	MUSIC	0.1	0.2	0.1	70/200
5.32	2	MUSIC	0.1	0.2	0.1	70/200
5.33	2	MUSIC	0.1	0.2	0.1	70/200
5.34	2	MUSIC	0.1	0.2	0.1	70/200
5.35	2	MUSIC	0.1	0.2	0.1	70/200
5.36	2	MUSIC	0.1	0.2	0.1	70/200
5.37	2	MUSIC	0.1	0.2	0.1	70/200
5.38	2	MUSIC	0.1	0.2	0.5	70/250
5.40	2	MUSIC	0.1	0.2	0.5	70/250
5.42	2	MUSIC	0.1	0.2	0.5	70/250
5.44	2	MUSIC	0.1	0.2	0.5	70/250

Table A.3: Path extraction parameters

Figure	Method	Path	Viterbi		BCJR			BCJR, BF				
			K	δ	K_1	K_2	δ	K_0	K_3	σ	Λ	ϵ
5.3	1	Viterbi	0.2	3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.4	1	Viterbi	0.2	3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.5	1	Viterbi	0.2	3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.6	1	Viterbi	0.5	3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.7	1	Viterbi	0.5	3	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.8	1	Viterbi	0.8	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.9	1	Viterbi	0.8	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.10	1	Viterbi	0.8	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.11	2	Viterbi	0.5	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.12	2	Viterbi	1.0	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.13	2	Viterbi	1.0	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.14	2	Viterbi	0.8	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.15	2	Viterbi	0.8	2	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
5.20	2	BCJR	0	2	0.5	0.5	2	n/a	n/a	n/a	n/a	n/a
5.21	2	BCJR	0	2	0.5	0.5	2	n/a	n/a	n/a	n/a	n/a

continued on next page

Table A.3: Path extraction parameters - *continued from previous page*

Figure	Method	Path	Viterbi		BCJR			BCJR, BF			
			K	δ	K_1	K_2	δ	K_0	K_3	σ	ϵ
5.22	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.23	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.24	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.25	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.26	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.27	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.28	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.29	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.07
5.30	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.09
5.31	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.09
5.32	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.09
5.33	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.09
5.34	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.09
5.35	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.09
5.36	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	5 0.09

continued on next page

Table A.3: Path extraction parameters - continued from previous page

Figure	Method	Path	Viterbi		BCJR			BCJR, BF			
			K	δ	K_1	K_2	δ	K_0	K_3	σ	ϵ
5.37	2	BCJR,BF	0.1	2	0.5	0.5	2	2	0.25	20	0.09
5.38	2	BCJR,BF	0.0	3	0.9	0.5	3	2	0.25	20	0.07
5.40	2	BCJR,BF	0.0	3	0.9	0.5	3	2	0.25	20	0.07
5.42	2	BCJR,BF	0.0	3	0.9	0.5	3	2	0.25	20	0.07
5.44	2	BCJR,BF	0.0	3	0.9	0.5	3	2	0.25	20	0.07

APPENDIX B

Derivation of Path Derivative

This appendix uses geometry to derive the formula for the derivative of the path length

$$\frac{dR}{dt} = \lim_{\Delta t \rightarrow 0} \frac{R(t + \Delta t) - R(t)}{\Delta t} \quad (\text{B.1})$$

in the situation shown in Figure B.1.

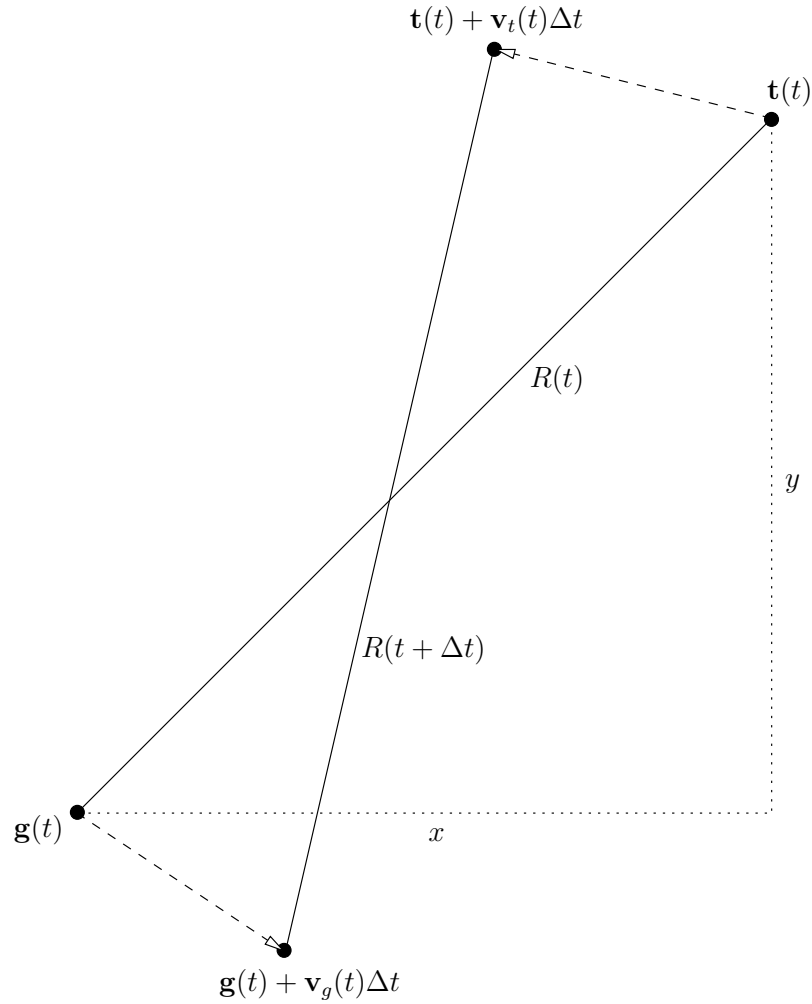


Fig. B.1: Diagram of a changing path length between two moving objects.

With the position of one moving object defined as $\mathbf{t}(t) = (t_x, t_y)$ with velocity $\mathbf{v}_t(t) = (v_{xt}, v_{yt})$, and the position of the other moving object defined as $\mathbf{g}(t) = (g_x, g_y)$ with velocity $\mathbf{v}_g(t) = (v_{xg}, v_{yg})$, we find

$$\begin{aligned}
\frac{R(t + \Delta t) - R(t)}{\Delta t} &= \frac{\left\| (\mathbf{t}(t) + \mathbf{v}_t(t)\Delta t) - (\mathbf{g}(t) + \mathbf{v}_g(t)\Delta t) \right\| - \left\| \mathbf{t}(t) - \mathbf{g}(t) \right\|}{\Delta t} \\
&= \frac{\sqrt{\left((t_x + v_{xt}\Delta t) - (g_x + v_{xg}\Delta t) \right)^2 + \left((t_y + v_{yt}\Delta t) - (g_y + v_{yg}\Delta t) \right)^2}}{\Delta t} \\
&\quad - \frac{\sqrt{(t_x - g_x)^2 + (t_y - g_y)^2}}{\Delta t} \\
&= \frac{\sqrt{\left((t_x - g_x) + \Delta t(v_{xt} - v_{xg}) \right)^2 + \left((t_y - g_y) + \Delta t(v_{yt} - v_{yg}) \right)^2}}{\Delta t} \\
&\quad - \frac{\sqrt{(t_x - g_x)^2 + (t_y - g_y)^2}}{\Delta t}.
\end{aligned}$$

Defining $x = t_x - g_x$, $y = t_y - g_y$, $\Delta v_x = v_{xt} - v_{xg}$, and $\Delta v_y = v_{yt} - v_{yg}$ leads to

$$\begin{aligned}
\frac{R(t + \Delta t) - R(t)}{\Delta t} &= \frac{\sqrt{\left(x + \Delta t(\Delta v_x) \right)^2 + \left(y + \Delta t(\Delta v_y) \right)^2} - \sqrt{x^2 + y^2}}{\Delta t} \\
&= \frac{\sqrt{x^2 + 2x\Delta t(\Delta v_x) + (\Delta t(\Delta v_x))^2 + y^2 + 2y\Delta t(\Delta v_y) + (\Delta t(\Delta v_y))^2} - \sqrt{x^2 + y^2}}{\Delta t}.
\end{aligned}$$

The terms $(\Delta t(\Delta v_x))^2$ and $(\Delta t(\Delta v_y))^2$ can be dropped as they will be insignificant in the limit as $\Delta t \rightarrow 0$,

$$\begin{aligned}
\frac{R(t + \Delta t) - R(t)}{\Delta t} &\approx \frac{\sqrt{x^2 + 2x\Delta t(\Delta v_x) + y^2 + 2y\Delta t(\Delta v_y)} - \sqrt{x^2 + y^2}}{\Delta t} \\
&= \frac{\sqrt{(x^2 + y^2) + 2\Delta t(x(\Delta v_x) + y(\Delta v_y))} - \sqrt{x^2 + y^2}}{\Delta t} \\
&= \frac{\sqrt{x^2 + y^2} \sqrt{1 + \frac{2\Delta t(x(\Delta v_x) + y(\Delta v_y))}{x^2 + y^2}} - \sqrt{x^2 + y^2}}{\Delta t}.
\end{aligned}$$

Using the identity $\sqrt{1+\alpha} \approx 1 + \alpha/2$ when $\alpha \ll 1$, the equation simplifies to

$$\begin{aligned}
 \frac{R(t+\Delta t) - R(t)}{\Delta t} &\approx \frac{\sqrt{x^2 + y^2} \left(1 + \frac{\Delta t (x(\Delta v_x) + y(\Delta v_y))}{x^2 + y^2} \right) - \sqrt{x^2 + y^2}}{\Delta t} \\
 &= \frac{\sqrt{x^2 + y^2} + \frac{\Delta t (x(\Delta v_x) + y(\Delta v_y))}{\sqrt{x^2 + y^2}} - \sqrt{x^2 + y^2}}{\Delta t} \\
 &= \frac{\Delta t (x(\Delta v_x) + y(\Delta v_y))}{\Delta t \sqrt{x^2 + y^2}} \\
 &= \frac{x(\Delta v_x) + y(\Delta v_y)}{\sqrt{x^2 + y^2}}.
 \end{aligned}$$

Plugging back in the values for x , y , Δv_x , and Δv_y , and taking the limit leads to

$$\frac{dR}{dt} = \frac{(t_x - g_x)(v_{xt} - v_{xg}) + (t_y - g_y)(v_{yt} - v_{yg})}{\sqrt{(t_x - g_x)^2 + (t_y - g_y)^2}} = \frac{(\mathbf{v}_t(t) - \mathbf{v}_g(t)) \cdot (\mathbf{t}(t) - \mathbf{g}(t))}{\|\mathbf{t}(t) - \mathbf{g}(t)\|}. \quad (\text{B.2})$$

At this point it is recognized that

$$\frac{\mathbf{t}(t) - \mathbf{g}(t)}{\|\mathbf{t}(t) - \mathbf{g}(t)\|} = \hat{\mathbf{u}}(t)$$

is a unit vector pointing from $\mathbf{g}(t)$ to $\mathbf{t}(t)$. This allows (B.2) to be written as

$$\frac{dR}{dt} = (\mathbf{v}_t(t) - \mathbf{v}_g(t)) \cdot \frac{\mathbf{t}(t) - \mathbf{g}(t)}{\|\mathbf{t}(t) - \mathbf{g}(t)\|} = (\mathbf{v}_t(t) - \mathbf{v}_g(t)) \cdot \hat{\mathbf{u}}(t). \quad (\text{B.3})$$

APPENDIX C

Code Listings

The MATLAB code used to generate the spectrograms throughout this thesis is provided in the following sections. This code is designed to use any of the methods discussed in this thesis.

Section C.1 lists the overhead files for the project. There are four overhead files for the system. These files have been split up to make it easier to control the system. Listing C.1 controls the parameters for the entire system. This file was the main file edited in adjusting the system. Listing C.2 is the overhead file for performing the spectrum estimation, and Listing C.3 is the overhead file for performing the path extraction. Listing C.4 is the overhead file responsible for plotting the results. This section also contains a function for reading in the data in Listing C.5.

Section C.2 contains the functions for generating the DFT and MUSIC spectra. Listing C.6 and Listing C.7 contain functions for the DFT. Listing C.8 and Listing C.9 contain functions for the MUSIC algorithm.

Section C.3 lists the various Viterbi functions for the different path extraction methods. The basic Viterbi algorithm used is in Listing C.10. The Viterbi algorithms for the symmetry correction of Section 4.4 are in Listing C.11 and Listing C.12.

Section C.4 lists the BCJR functions for the different path extraction methods. Listing C.13 contains the basic BCJR function, while Listing C.14 contains the balance factor adjusted BCJR algorithm. This section also includes the balance factor function in Listing C.15.

C.1 Overhead Files

Listing C.1: Overhead code for adjusting all of the parameters and controlling the entire system

```

1  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
2  %%% SET PARAMETERS %%%
3  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
4  %%% Control Variables %%%
5  % Whether to limit the length of the data
6  Data_limit = 0;
7  % This turns off the Spectrum generation when working with the same data
8  % for quiker results
9  Path_gen_only = 0;
10 % This shows the balance factor on plot as either above threshold or not
11 show_BF = 0;
12 % Which Doppler method to use (1 or 2)
13 Doppler_method = 2;
14 % Spectrum for output (DFT or MUSIC)
15 Spectrum = 'MUSIC';
16 % Method to find optimal path
17 % (none, viterbi, max, symm_viterbi, BCJR, or BCJR_BF)
18 Path = 'BCJR_BF';
19 % (METHOD1 only)
20 % Where to get the data for the sign extraction (PLL or SS)
21 Sign_ext = 'SS';
22 % (METHOD2 only)
23 % Method to find carrier frequency offset (DFT or MUSIC)
24 C_spec = 'MUSIC';
25
26 %%% Choose test %%%
27 % Date of the data set
28 date = "21_02_12";
29 % Which test to run from the data set
30 test_num_1 = 0;
31 % Which receiver if multiple (type "none" if only one receiver)

```

```

32 receiver_l = "SW";
33
34 %%% Figure numbers %%%
35 time_plt = 0;
36 carr_offset_plt = 1;
37 DFT_out_plt = 3;
38 MUSIC_out_plt = 4;
39 BCJR_plot = 5;
40
41 %%% Spectral Estimation Variables %%%
42 % Symbol Sample rate
43 Fs = 153600/8;
44 % Center frequency
45 cenF = 0;
46 % Maximum frequency extension in either +/- direction
47 maxF = 30;
48 % Spectrum resolution
49 delF = 0.5;
50 % Data frame length , in seconds
51 DL = 0.2;
52 % Data time step
53 D_step = 0.1;
54 % (MUSIC only)
55 % Estimate of the number of signals
56 P = 70;
57 % M improves the resolution of MUSIC, at expense of computations
58 M = 250;
59 % (Data limit)
60 % When to start the data (time in seconds)
61 D_start = 5;
62 % When to end data (time in seconds)
63 D_end = 15;
64
65 %%% Path Extraction Variables %%%
66 %%% (Viterbi, normal or symmetric) %%%

```

```

67 % Maximum frequency shift from state to state for viterbi
68 delta_v = 2;
69 % Branch metric weight
70 K = 1;
71 %%% (Viterbi Sign) %%%
72 % Branch metric weight
73 K_sign = 0.01;
74 % How much data should be used to find the sign
75 sign_DL = DL;
76 %%% (BCJR) %%%
77 % Maximum frequency shift from state to state for BCJR
78 delta_BCJR = 3;
79 % Weight for the branch metric
80 K1 = 0.9;
81 % Weight for the state value
82 K2 = 0.5;
83 %%% (BCJR BF) %%%
84 % Threshold to turn on adjustment spectrum
85 epsilon = 0.07;
86 % Weight given to balance factor
87 K0 = 2;
88 % Width of adjustment spectrum
89 sigma = 20;
90 % Linear drop rate away from 0 Hz
91 K3 = 0.25;
92 % Maximum value of adjustment spectrum, located at 0 Hz
93 Lambda = 5;
94 %%% (For plotting balance factor) %%%
95 % Position on spectrogram
96 graph_pos = -25;
97 % Scaling variable to make changes in BF noticable
98 dX = 10;
99
100 %%% (METHOD2) Carrier spectrum setup %%%
101 % Maximum frequency MUSIC extension in either +/- direction

```



```

102 maxF_i = 3;
103 % Spectrum resolution
104 delF_i = 0.1;
105 % Data frame length, in seconds
106 DL_i = 0.05;
107 % How much time to step forward for each data point
108 D_step_i = 0.05;
109 % How many signals you should find in data
110 P_i = 1;
111 % M improves the resolution of MUSIC, at expense of computations
112 M_i = 5;
113 %%% Viterbi parameters %%%
114 % Branch metric weight
115 K_i = 30;
116 % Furthest allowed state jump
117 delta_i = 0.25;
118
119 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
120 %%% EXECUTE THE EXTRACTION %%%
121 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
122 % Cycle through all the tests
123 for test_num = test_num_1
124     % Antenna number for outputting Doppler paths
125     antenna_num = 1;
126     clear dopp_mat;
127     % Cycle through all the receivers in this test
128     for ii = 1:length(receiver_1)
129         receiver = receiver_1(ii);
130         %Run the program and time it
131         tic
132         %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
133         %%% SPECTRUM ESTIMATION %%%
134         %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
135         if(Path_gen_only == 0)
136             doppler_spectrum_generation;

```

```

137         end
138         %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
139         %%% PATH EXTRACTION %%%
140         %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
141         doppler_path_extraction;
142
143         % Report time
144         elapsedTime = toc;
145         disp(newline);
146         disp("Program took "+elapsedTime+" seconds to complete");
147         disp(newline);
148
149         %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
150         %%% MAKE PLOTS %%%
151         %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
152         doppler_plots;
153
154         % Update plot numbers so you don't lose them all for multiple runs
155         carr_offset_plt = carr_offset_plt+1;
156         DFT_out_plt = DFT_out_plt+1;
157         MUSIC_out_plt = MUSIC_out_plt+1;
158         BCJR_plot = BCJR_plot+1;
159
160         %         %%% Output the plots to files %%%
161         %         spec_path = output_plot_path+"/"+date+"_test"+test_num+"_"+receiver
162         %         +"_"+Spectrum+"spec.png";
163         %         %prob_path = output_plot_path+"/"+date+"_test"+test_num+"_"+receiver
164         %         +"_"+Spectrum+"prob.png";
165         %         saveas(spec,spec_path);
166         %         %saveas(probs,prob_path);
167
168         %         %%% Save path in matrix and move on to next antenna %%%
169         %         dopp_mat(antenna_num,:) = best_path;
170         %         % Update antenna number
171         %         antenna_num = antenna_num + 1;

```

```
170     end
171 %     %%% Output Doppler matrix to a file for these tests %%%
172 %     output_file = output_data_path+"/"+date+"_test"+test_num+"_"+Spectrum+".
      mat";
173 %     save(output_file , 'dopp-mat' , 'receiver-1 ');
174 end
```

Listing C.2: Overhead code to perform the spectrum estimation

```

1  % This file performs the spectrum estimation and produces a spectrogram
2  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
3  %%% GET NEEDED DATA FIRST %%%
4  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
5  %%% Determine if setup is correct or throw error %%%
6  if(Doppler_method == 2)
7      if(strcmp(Path, 'symm_viterbi'))
8          error('Method 2 does not support the symmertic viterbi algorithm');
9      end
10 else
11     error('Please choose a correct Doppler method of 1 or 2');
12 end
13
14 %%% Determine file name %%%
15 data_path = "~/Documents/research_2020-21/data/";
16 if(receiver == "none")
17     data_set = date + "_data/";
18 else
19     data_set = date + "_data/"+receiver+"/";
20 end
21 % Generic file location
22 file_start = data_path+data_set+"test_"+test_num;
23 % Get the Doppler signal collected from method 1
24 file_dopp = file_start+"_doppler.bin";
25 % Get the Doppler signal collected form method 2
26 file_new_dopp = file_start+"_new_doppler.bin";
27 % Get zk from the carrier frequency synchronization block
28 file_PLL = file_start+"_carr_phase.bin";
29 % Get sk from the symbol synchronization block
30 file_sym_sync = file_start+"_sym_sync.bin";
31 % Get ak fromt he symbol decision block
32 file_sym_dec = file_start+"_sym_decision.bin";
33
34 %%% Read in the data %%%

```

```

35 if(Doppler_method == 1)
36     d = get_complex_data(file_dopp);
37     % Remove the mean from the data
38     d = d - mean(d);
39     len = length(d);
40 elseif(Doppler_method == 2)
41     dn = get_complex_data(file_new_dopp);
42     len = length(dn);
43 end
44 % Read in extra data for the symmetry correction, if needed
45 if(strcmp(Path, 'symm_viterbi'))
46     ss = get_complex_data(file_sym_sync);
47     ahat = get_complex_data(file_sym_dec);
48     if(strcmp(Sign_ext, 'PLL'))
49         PLL = get_complex_data(file_PLL);
50     elseif(strcmp(Sign_ext, 'SS'))
51         ss = get_complex_data(file_sym_sync);
52     end
53 end
54
55 %%% Limit the data length, if needed %%%
56 if(Data_limit)
57     % Compute the start and stop from the time given in seconds
58     start = floor(D_start*Fs + 1);
59     finish = floor(D_end*Fs);
60     % Try block to catch error if requested data out of range
61     try
62         if(Doppler_method == 1)
63             d = d(start:finish);
64             len = length(d);
65         elseif(Doppler_method == 2)
66             dn = dn(start:finish);
67             len = length(dn);
68         end
69         if(strcmp(Path, 'symm_viterbi'))

```

```

70         ss = ss(start:finish);
71         ahat = ahat(start:finish);
72     end
73     catch ME
74         msg_start = "Unable to start and stop data at given parameters";
75         msg_end = "Data is "+len/Fs+" seconds long";
76         msg = msg_start + newline + msg_end;
77         error(msg);
78     end
79 end
80
81
82 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
83 %%% DETERMINE OTHER VARIABLES %%%
84 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
85 %%% Set up the data frames %%%
86 % How many data points in each frame
87 frame_length = floor(DL*Fs);
88 % How many frames to cycle through
89 n_points = floor((len-frame_length)/(Fs*D_step));
90 % How many points to advance each frame
91 dT = floor(Fs*D_step);
92
93 %%% Setup frequency bins %%%
94 % List of all frequency bins in continuous frequencies
95 F = -maxF+cenF:delF:maxF+cenF;
96 % Discrete frequency bins
97 f = F./Fs;
98
99 %%% (METHOD2) Setup frequency bins for finding carrier frequency offset %%%
100 if(Doppler_method == 2)
101     % Number of data points in each frame
102     frame_length_i = floor(DL_i*Fs);
103     % How many frames to cycle through
104     n_points_i = floor((len-frame_length_i)/(Fs*D_step_i));

```



```

138 %%% (METHOD2) REMOVE CARRIER FREQUENCY OFFSET %%%
139 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
140 % This method currently calculates the carrier frequency offset separately
141 % and removes it. This will not translate to real-time and will need to be
142 % integrated into other loops
143 if(Doppler_method == 2)
144     % Create empty arrays to hold data
145     y_mat = zeros(frame_length_i, n_points_i);
146
147     %%% Find the frequency spectrum for the first time interval %%%
148     % x is now the first frame of dn
149     x = dn(1:frame_length_i);
150     if(strcmp(C_spec, 'DFT'))
151         X_f_i = DFT(x, Wn_tot);
152     elseif(strcmp(C_spec, 'MUSIC'))
153         X_f_i = MUSIC(x, P_i, M_i, S_tot);
154     else
155         error('Only supported frequency spectrum algorithms are the DFT or
156             MUSIC');
157     end
158
159     %%% Estimate the carrier frequency offset and use to find the rest easier
160     %%%
161     % Find the max postion, this is most likely within Fs_i/2 Hz of all the
162     % other frequency offsets
163     [~, max_pos] = max(X_f_i);
164     % Continuous frequency bins
165     F_i = -maxF_i+f_tot(max_pos)*Fs:delF_i:maxF_i+f_tot(max_pos)*Fs;
166     % Discrete frequencies
167     f_i = F_i./Fs;
168     % Recompute weights with less data for faster computations
169     if(strcmp(C_spec, 'DFT'))
170         Wn_i = compute_Wn(f_i, frame_length_i);
171     elseif(strcmp(C_spec, 'MUSIC'))
172         S_i = compute_S(f_i, M_i);

```



```

170     end
171
172     %%% Use new frequency values to calculate the rest of the Spectrums %%%
173     % Empty array to hold spectrogram
174     Xlog_i = zeros(length(f_i), n_points_i);
175     for i = 1:n_points_i
176         % Get the current data frame
177         vals = (i-1)*dT_i+1:(i-1)*dT_i+frame_length_i;
178         % x is now the frame of dn we are interested in
179         x = dn(vals);
180         % Apply the proper algorithm to get the frequency spectrum
181         if(strcmp(C_spec, 'DFT'))
182             X_f_i = DFT(x, Wn_i);
183         elseif(strcmp(C_spec, 'MUSIC'))
184             X_f_i = MUSIC(x, P_i, M_i, S_i);
185         else
186             error('Only supported frequency spectrum algorithms are the DFT or MUSIC');
187         end
188         % Store X_f_i
189         Xlog_i(:, i) = 10*log10(abs(X_f_i));
190     end
191
192     %%% Now remove the carrier frequency offset %%%
193     cpr_path = viterbi(Xlog_i, K_i, F_i, delta_i);
194     for i = 1:n_points_i
195         % Get the current data frame
196         vals = (i-1)*dT_i+1:(i-1)*dT_i+frame_length_i;
197         % x is now the frame of dn we are interested in
198         x = dn(vals);
199         % Modulate x
200         x_mod = x.*exp(1j*2*pi*cpr_path(i)*vals'/Fs);
201         % Store the modulation in a matrix
202         y_mat(:, i) = x_mod;
203     end

```

```

204
205     %%% Now combine data back together %%%
206     % Store new data in variable y
207     y = zeros(len, 1);
208     for i = 1:n_points-1
209         % Get the current data frame
210         vals = (i-1)*dT+1:(i-1)*dT+frame_length-i;
211         % Remove mean first to prevent spectral leakage
212         y_win = y_mat(:, i) - mean(y_mat(:, i));
213         % The plus allows overlap
214         y(vals) = y(vals) + y_win;
215     end
216 end
217
218
219 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
220 %%% CREATE SPECTROGRAM OF THE DATA %%%
221 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
222 % Create empty spectrograms to hold data
223 X_f = zeros(length(f), n_points);
224 X_f_bf = zeros(length(f), n_points);
225 for i = 1:n_points
226     %%% Get the correct data frame depending on the method %%%
227     if(Doppler_method == 1)
228         % Get the current data frame
229         vals = (i-1)*dT+1:(i-1)*dT+frame_length;
230         x = d(vals);
231     elseif(Doppler_method == 2)
232         % Get the current data frame
233         vals = (i-1)*dT+1:(i-1)*dT+frame_length;
234         x = y(vals);
235     else
236         error('I do not know how you evaded all my other checks...');
237     end
238     %%% Transform to frequency domain

```

```

239     if(strcmp(Spectrum, 'DFT'))
240         % Compute the DFT
241         X_f(:, i) = DFT(x, Wn);
242         % Scale up DC term (visual effect only)
243         X_f(floor(size(X_f, 1)/2)+1, i) = mean(X_f(:, i));
244         % Store DFT for the balance factor computation
245         X_f_bf(:, i) = X_f(:, i);
246     elseif(strcmp(Spectrum, 'MUSIC'))
247         % Compute MUSIC spectrum
248         X_f(:, i) = MUSIC(x, P, M, S);
249         % Compute DFT for the balance factor computation
250         X_f_bf(:, i) = DFT(x, Wn_cf);
251     else
252         error('Only supported frequency spectrum algorithms are the DFT or
                MUSIC');
253     end
254 end
255 X_log = 10*log10(abs(X_f));

```

Listing C.3: Overhead code to perform the path extraction

```

1  % This file contains the code to extract the Doppler path from a
2  % Spectrogram
3  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
4  %%% FIND BALANCE FACTOR %%%
5  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
6  BF = balance_factor(abs(X_f_bf), F);
7
8  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
9  %%% APPLY PATH FINDING ALGORITHMS %%%
10 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
11 %%% Viterbi path %%%
12 if(strcmp(Path, 'viterbi'))
13     % Run viterbi algorithm
14     best_path = viterbi(X_log, K, F, max_delta_v);
15 %%% Maximum value path %%%
16 elseif(strcmp(Path, 'max'))
17     % Find position of the maximum values
18     [~, max_pos] = max(X_log);
19     % Path is the frequencies of these maximum values
20     best_path = F(max_pos);
21 %%% Symmetric Viterbi path %%%
22 elseif(strcmp(Path, 'symm-viterbi'))
23     % Run the symmetric viterbi algorithm
24     symm_path = symmetric_viterbi(X_log, K, F, max_delta_v);
25     % Try and find the sign using data from carrier frequency
26     % synchronization block
27     if(strcmp(Sign_ext, 'PLL'))
28         best_path = viterbi_sign(symm_path, PLL, ahat, K_sign, dT, floor(sign_DL*Fs)
29             );
30     % Try and find the sign using data from the symbol synchronization
31     % block
32     elseif(strcmp(Sign_ext, 'SS'))
33         best_path = viterbi_sign(symm_path, ss, ahat, K_sign, dT, floor(sign_DL*Fs)
34             );

```

```

33     end
34     %%% BCJR path %%%
35     elseif(strcmp(Path, 'BCJR'))
36         % Run spectrogram through BCJR
37         Probs = BCJR(X_log, F, K1, K2, delta_BCJR);
38         % Run probability plot through Viterbi
39         best_path = viterbi(Probs, K, F, delta_v);
40     %%% BCJR with balance factor path %%%
41     elseif(strcmp(Path, 'BCJR_BF'))
42         % Run spectrogram through BCJR, with the balance factor adjustments
43         Probs = BCJR_BF(X_log, F, K1, K2, delta_BCJR, BF, epsilon, K0, sigma, K3,
44             Lambda);
45         % Run probability plot through Viterbi
46         best_path = viterbi(Probs, K, F, delta_BCJR);
47     %%% No path desired %%%
48     elseif(strcmp(Path, 'none'))
49         % Set path to 0 to signify no path
50         best_path = 0;
51     %%% Error check %%%
52     else
53         msg_start = 'Please check that your chosen path is one of the following
54             options: ';
55         msg_end = 'viterbi, max, symm-viterbi, weighted-ave';
56         msg = [msg_start newline msg_end];
57         error(msg);
58     end

```

Listing C.4: Overhead code to create spectrogram plots

```

1 % This file takes care of creating all of the plots from the data output by
2 % the Doppler extraction system
3 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
4 %%% PLOT THE TIME DOMAIN SIGNAL %%%
5 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
6 % First grab the correct data
7 if(Doppler_method == 1)
8     x = d;
9 elseif(Doppler_method == 2)
10    x = dn;
11 end
12 % Plot the original signal
13 if(time_plt ~= 0)
14     figure(time_plt); clf();
15     % create the time axis values for the whole signal
16     t = (0:length(x)-1)/Fs;
17     % Plot real portion of the signal
18     plot(t,real(x));
19     hold on;
20     % Plot imaginary portion of the signal
21     plot(t,imag(x));
22     hold off;
23     xlabel('time (s)');
24     ylabel('signal magnitude');
25 end
26
27
28 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
29 %%% PLOT THE OUTPUT SPECTROGRAM AND BEST PATH %%%
30 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
31 if((DFT_out_plt ~= 0 && strcmp(Spectrum, 'DFT')) || (MUSIC_out_plt ~= 0 && strcmp
    (Spectrum, 'MUSIC')))
32     if(strcmp(Spectrum, 'DFT'))
33         spec = figure(DFT_out_plt);

```



```

69  %%% (METHOD2) PLOT THE CARRIER FREQUENCY OFFSET %%%
70  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
71  % Plot spectrogram of the full spectrum
72  if(Doppler_method == 2 && carr_offset_plt ~= 0)
73      figure(carr_offset_plt); clf();
74      imagesc([f_i(1)*Fs, f_i(end)*Fs],[0, length(x)/Fs],Xlog_i. ');
75      colorbar;
76      % Plot the Viterbi path through
77      hold on;
78      maxT_i = length(x)/Fs;
79      dopp_t_i = (0:maxT_i/length(cpr_path):maxT_i - maxT_i/length(cpr_path));
80      plot(cpr_path, dopp_t_i, '*r');
81      hold off;
82      % Graph Aesthetics
83      ylabel('time (s)');
84      xlabel('frequency (Hz)');
85  end
86
87
88  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
89  %%% BCJR Algorithm Probability output plot %%%
90  %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
91  if((strcmp(Path, 'BCJR') || strcmp(Path, 'BCJR.BF')) && BCJR_plot ~= 0)
92      probs = figure(BCJR_plot);
93      imagesc([-maxF+cenF, maxF+cenF],[0, length(x)/Fs],10*log10(abs(Probs)). ');
94      colorbar;
95      hold on;
96      maxT = length(x)/Fs;
97      dopp_t = (0:maxT/length(best_path):maxT - maxT/length(best_path));
98      plot(best_path, dopp_t, '*r');
99      % Plot crest factor decision
100     if(show_BF)
101         t = 0;
102         hold on;
103         for bf = BF

```



```
104         if(abs(bf) > threshold)
105             plot(graph_pos+bf*dX,t,'ok','MarkerSize',8);
106         else
107             plot(graph_pos+bf*dX,t,'xr','MarkerSize',8);
108         end
109         plot(graph_pos,t,'.b','MarkerSize',8);
110         t = t + D_step;
111     end
112     hold off;
113 end
114 hold off;
115 ylabel('time (s)');
116 xlabel('frequency (Hz)');
117 end
```

Listing C.5: Function to get the saved GNU radio output data

```
1 % GET.COMPLEX.DATA gets complex data from a file
2 function [data] = get_complex_data(file_name)
3     % Open the file
4     file = fopen(file_name, 'r');
5     % read data into 2 columns
6     read = fread(file, [2,Inf], 'float32').';
7     % Close the file
8     fclose(file);
9     % convert data to complex
10    data = complex(read(:,1),read(:,2));
11 end
```

C.2 Spectrum Estimation Functions

Listing C.6: Function to compute the twiddle factors for the DFT

```

1  %COMPUTE_WN Compute the DFT twiddle factors
2  %   This function computes the twiddle factors for a certain frequency
3  %   spectrum to be used on a nfft point DFT
4  %       f:           The frequencies to compute X_f over (need to be in
5  %                   discrete form)
6  %       nfft:        DFT size
7  %   Outputs:
8  %       Wn:          The twiddle factors to use
9  function [Wn] = compute_Wn(f, nfft)
10     % Compute bin frequencies from discrete frequencies
11     k = f*nfft;
12     % Make an empty matrix for Wn
13     Wn = zeros(nfft, length(f));
14
15     % Compute the Wn values
16     for n = 1:nfft
17         Wn(n,:) = exp(-1j*2*pi*k*n/nfft);
18     end
19 end

```

Listing C.7: Function to compute the DFT spectrum

```

1  %DFT This function computes the DFT algorithm on a signal and returns
2  %the frequency spectrum
3  %   This function expects to be given a chunk of data and a set of
4  %   frequencies that the program will implement the DFT on. It will then
5  %   return the frequency response
6  %       input:      A chunk of the input signal to perform the DFT on
7  %       f:          The frequencies to compute X_f over (need to be in
8  %                   discrete form)
9  %       Wn:          The twiddle factors to use
10 %   Outputs:
11 %       X_f:         DFT of the signal
12 function [X_f] = DFT(input , Wn)
13     % Check to see that input is a column vector
14     if(iscolumn(input))
15         % Transpose to a column vector
16         input = input';
17     end
18
19     % Compute the DFT
20     X_f = input*Wn;
21 end

```

Listing C.8: Function to compute the S vectors for MUSIC

```

1  %COMPUTE_S Compute the MUSIC S matrix
2  %   This function computes the S matrix for the music algorithm to speed
3  %   up the process of computing the algorithm
4  %       f:           The frequencies to compute X_f over (need to be in
5  %                   discrete form)
6  %       M:           M is how big each S vector needs to be
7  %   Outputs:
8  %       S:           S matrix for the MUSIC algorithm
9  function [S] = compute_S(f, M)
10     % This is for the length of the y(ti) vectors
11     n = 0:M-1;
12     % Compute the S matrix
13     S = exp(-1j*2*pi*n.'*f);
14 end

```

Listing C.9: Function to compute the MUSIC spectrum

```

1  %MUSIC This function computes the MUSIC algorithm on a signal
2  %   This function expects to be given a chunk of data, from which the
3  %   program will determine the approximate Ry matrix and use this to apply
4  %   the MUSIC algorithm to determine the MUSIC spectrum.
5  %       input:      A chunk of the input signal to perform MUSIC on
6  %       P:          An expected value for P, the number of expected
7  %                   signals.
8  %       M:          A number greater than P.
9  %       f:          The frequencies to compute P_a over (need to be in
10 %                   discrete form).
11 %   Outputs:
12 %       P_a:         P(f)=1/H(f), the music spectrum of the input
13 %                   signal.
14 function [P_a] = MUSIC(input , P, M, S)
15     % Check to see that input is a column vector
16     if(isrow(input))
17         % Transpose to a column vector
18         input = input';
19     end
20
21     %%% Approximate Ry %%%
22     % Find the mean of the signal
23     mean_input = mean(input);
24     % Subtract mean to get zero-mean noise
25     input = input - mean_input;
26     % Determine the number of iterations
27     N = length(input) - M + 1;
28     % Initialize the Ry matrix as an MxM
29     Ry_hat = zeros(M);
30     for i = 1:N
31         %MAC the matrices
32         Ry_hat = Ry_hat + input(i:i+M-1)*input(i:i+M-1)';
33     end
34     % Scale by N

```

```

35 Ry_hat = Ry_hat./N;
36
37 %%% Now get the eigenvalues and eigenvectors %%%
38 [vecs, vals] = eig(Ry_hat);
39 % Sort eigenvalues from largest to smallest
40 [~, eig_order] = sort(real(diag(vals)), 'descend');
41 % Reorder eigenvectors to match reordering of eigenvalues
42 eigvecs = vecs(:, eig_order);
43
44 %%% Compute MUSIC spectrum %%%
45 size_S = size(S);
46 % Empty array to hold frequency response of MUSIC
47 M_a = zeros(size_S(2),1);
48 for i = P+1:M
49     % MAC the |S(f)'uk|^2 values
50     M_a = M_a + abs(S'*eigvecs(:,i)).^2;
51 end
52 % This create P(f) from M(f)
53 P_a = 1./M_a;
54 end

```

C.3 Viterbi Functions

Listing C.10: Function to compute the Viterbi algorithm for finding the Doppler path

```

1  %VITERBI This function computes the Viterbi algorithm on a spectrogram
2  % and returns the optimal path.
3  %   This function expects a spectrogram input. This function uses this
4  %   spectrogram to find the optimal path of the Doppler frequency.
5  %   Inputs:
6  %       input:      A MxN matrix where M is a collection of frequency bins
7  %                   and N refers to a time step
8  %       K:          A weight to determine how much credence to give to the
9  %                   branch metric
10 %       Freqs:       The frequency bins used (in continuous time values)
11 %       delta:       The maximum frequency difference allowed from current
12 %                   state to the next state.
13 %   Output:
14 %       opt_path     An array containing the optimal path through the data.
15 function [opt_path] = viterbi(input, K, Freqs, delta)
16     % Number of states
17     num_states = size(input,1);
18     % Number of time steps for Viterbi algorithm
19     v_points = size(input,2);
20     % Number of paths from one state to the next based on the delta value
21     paths_to_state = round(delta/abs(Freqs(1) - Freqs(2)));
22
23     % Empty array for the state paths
24     paths = zeros(num_states, v_points);
25     % Empty array for the state path values
26     M = zeros(num_states,1);
27
28     %%% Compute the initial states %%%
29     for m = 1:num_states
30         % Start at the given frequency
31         paths(m,1) = Freqs(m);
32         % Path value initialized to the magnitude of the current frequency

```



```

33         M(m) = input(m,1);
34     end
35
36     %%% Find the most probably path %%%
37     % Start at 2 because first point was for initial state
38     for j=2:v_points
39         % Now loop through each state
40         new_M = zeros(num_states, 1);
41         temp_paths = zeros(num_states, v_points);
42         for m = 1:num_states
43             % Get the magnitude at this state
44             v = input(m,j);
45             % Loop through each path to state m
46             for i = -paths_to_state:paths_to_state
47                 % Check to see if the previous state is within the frequency
48                     range
49                 if((m + i > 0) && (m + i <= num_states))
50                     % Compute the branch metric
51                     branch_metric = K*abs(Freqs(m+i) - Freqs(m));
52                     % Get the previous path value
53                     Mprev = M(m+i);
54                     % Compute the path value
55                     Mnext = Mprev - branch_metric + v;
56                     % Compare the path value to the best path value
57                     if(Mnext > new_M(m))
58                         % If this path value is greater, then record
59                         new_M(m) = Mnext;
60                         temp_paths(m,1:j) = [paths(m+i,1:j-1), Freqs(m)];
61                     end
62                 end
63             end
64         % Store the new paths and path values
65         M = new_M;
66         paths = temp_paths;

```

```
67     end
68
69     %%% Retreive the state with the best path %%%
70     [~, state] = max(M);
71
72     %%% Finally output the best path %%%
73     opt_path = paths(state,:);
74 end
```

Listing C.11: Function to compute the Viterbi algorithm for finding the Doppler path in the presence of symmetry

```

1  %SYMMETRIC_VITERBI This function computes the Viterbi algorithm on a Matrix of
2  % frequency data over time and returns the optimal path. This assumes
3  % symmetry in the negative and positive frequencies.
4  %   This function expects an input of a spectrogram. This function uses
5  %   this data to find the optimal path a Doppler frequency. This function
6  %   assumes symmetry in the positive and negative frequencies, and it only
7  %   tracks the positive frequency, using the symmetry as additional
8  %   information to help track.
9  %   Inputs:
10 %       input:      A MxN matrix where M is a collection of frequency bins
11 %                   and N refers to a time step
12 %       K:          A weight to determine how much credence to give to the
13 %                   branch metric
14 %       Freqs:       The frequency bins used (in continuous time values)
15 %       delta:       The maximum frequency difference allowed from current
16 %                   state to the next state.
17 %   Output:
18 %       opt_path     An array containing the optimal path through the data.
19 function [opt_path] = symmetric_viterbi(input, K, Freqs, delta)
20     % Number of states, assuming 0 is the center frequency and there are
21     % the same number of frequencies on each side
22     num_states = floor(size(input,1)/2)+1;
23     % Number of time steps for Viterbi algorithm
24     v_points = size(input,2);
25     % Number of paths from one state to the next based on the delta value
26     paths_to_state = round(delta/abs(Freqs(1) - Freqs(2)));
27
28     % Empty array for the state paths
29     paths = zeros(num_states, v_points);
30     % Empty array for the state path values
31     M = zeros(num_states,1);
32

```

```

33  %%% Compute the initial states %%%
34  for m = 1:num_states
35      % Start at 0 (num_states) and then add m-1 to it to get all positive
        values
36      paths(m,1) = Freqs(num_states+m-1);
37      % Initial path value adds both the positive and negative path values
38      M(m) = input(num_states+m-1,1)+input(num_states-(m-1),1);
39  end
40
41  %%% Find the most probably path %%%
42  % Start at 2 because first point was for initial state
43  for j=2:v_points
44      % Now loop through each state
45      new_M = zeros(num_states, 1);
46      temp_paths = zeros(num_states, v_points);
47      for m = 1:num_states
48          % Loop through each path to state m
49          for i = -paths_to_state:paths_to_state
50              % Check to see if the previous state is within the frequency
                range
51              if((m + i > 0) && (m + i <= num_states))
52                  % Compute the branch metric
53                  branch_metric = K*abs(Freqs(num_states+m-1+i) - Freqs(
                        num_states+m-1));
54                  % Get the previous path value
55                  Mprev = M(m+i);
56                  % Compute V(f) = magnitude at both (+) and (-) f
57                  V_f = input(num_states+m-1,j) + input(num_states-(m-1),j);
58                  % Compute the path value
59                  Mnext = Mprev - branch_metric + V_f;
60                  % Compare the path value to the best path value
61                  if(Mnext > new_M(m))
62                      % If this path value is greater, then record
63                      new_M(m) = Mnext;

```

```

64         temp_paths(m,1:j) = [paths(m+i,1:j-1), Freqs(num_states
        +m-1)];
65     end
66 end
67 end
68 end
69 % Store the new paths and path values
70 M = new_M;
71 paths = temp_paths;
72 end
73
74 %%% Retrieve the state with the best path %%%
75 [~, state] = max(M);
76
77 %%% Finally output the best path %%%
78 opt_path = paths(state,:);
79 end

```

Listing C.12: Function to compute the Viterbi algorithm for finding the sign of the Doppler

```

1  %VITERBLSIGN implements a viterbi algorithm to find the sign of the
2  % Doppler frequencies
3  %   This function takes in a collection of Doppler frequencies and the
4  %   symbol synchronizer output to determine the sign of the Doppler
5  %   frequencies.
6  %   Inputs:
7  %       dopp_frequencies:  A list of the absolute values of the Doppler
8  %                           frequencies
9  %       sk:                The output data of the symbol synchronizer
10 %       ak:                The output data of the symbol decision block
11 %       K:                 Constant to adjust how much weight to give the
12 %                           branch metric
13 %       dT:                Step size from data point to data point
14 %       Data_length:       Length of data that goes into each Doppler
15 %                           frequency.
16 %   Outputs:
17 %       best_path:         The Doppler frequency path
18 function [best_path] = viterbi_sign(dopp_frequencies,sk,ak,K,dT,Data_length)
19     % Number of states, one for (+) and one for (-)
20     num_states = 2;
21     % Number of time steps for Viterbi algorithm
22     v_points = size(dopp_frequencies,2);
23
24     % Empty array for the state paths
25     paths = zeros(num_states, v_points);
26     % Empty array for the state path values
27     M = zeros(num_states,1);
28
29     %%% Find the most probably path %%%
30     for i = 1:v_points
31         % Get the indecies for the data points that goes into this frequency
32         val = (i-1)*dT+1:(i-1)*dT+Data_length;
33         % Grab the data from the symbol synchronizer needed for this Doppler
           frequency

```

```

34     si = sk(val);
35     % Grab the data from the symbol decision needed for this Doppler
        frequency
36     ai = ak(val);
37
38     %%% Calculate values for the radius and theta %%%
39     % Average radius
40     r = sum(abs(si))/length(si);
41     % The radius of the doppler rotation
42     rad = abs(si) - r;
43     % The angle of the doppler rotation
44     theta = unwrap(angle(si) - angle(ai));
45
46     %%% Use a linear line fit to theta to remove the linear portion of the
        data %%%
47     % Create a A vector, 2 x N where the first column is the x values and
        the second all ones
48     A = [val.', ones(length(val),1)];
49     % This computes  $((A^T A)^{-1})(A^T y)$ , which solves for m, the slope,
        and b, the offset, of the line
50     c = A\theta;
51     % Remove the linear fit to get only the Doppler and noise on top
52     theta = theta - (val.'*c(1) + c(2));
53
54     %%% Load in an advance filter and use to advance theta and rad %%%
55     % (filter designed using the matlab filter design tool and stored)
56     load('advance_filter.mat');
57     adv_theta = filter(adv_filt{1}, adv_filt{2}, theta);
58     adv_rad = filter(adv_filt{1}, adv_filt{2}, rad);
59
60     %%% Correlate the advanced signals to determine the probable sign %%%
61     advan = corr(adv_theta, rad);
62     delay = corr(adv_rad, theta);
63
64     %%% Viterbi to determine most likely sign

```



```
97         end
98         % Replace path value and path with the temporary variables
99         M = new_M;
100         paths = temp_paths;
101     end
102 end
103 % Retrieve the state with the best path
104 [~, state] = max(M);
105
106 % Return the best path calculation
107 best_path = paths(state,:);
108 end
```

C.4 BCJR Functions

Listing C.13: Function to compute the BCJR algorithm

```

1  %BCJR creates a probabilty matrix from a spectrogram of data
2  %   This function takes in a spectrogram of data and calculates the
3  %   probabilty of the Doppler at each frequency overtime.
4  %   Inputs:
5  %       input:      Frequency data over time
6  %       Freqs:      The frequency states at each time period
7  %       K1:         Constant to give weight to the branch metric
8  %       K2:         Constant to give weight to the state value
9  %       delta:      The maximum distance for a frequency change
10 %                   from time period to time period
11 %   Output:
12 %       P:          Matrix of the probabilities of each state at each time
13 %                   step
14 function [P] = BCJR(input , Freqs , K1, K2, delta)
15     % Total number of states
16     num_states = size(input,1);
17     % The number of time steps
18     v_points = size(input,2);
19
20     %%% Pre-calculate part of the gammas, the branch metric portion %%%
21     % Create an empty array
22     G = zeros(num_states, num_states);
23     for i = 1:num_states
24         for j = 1:num_states
25             % This is the frequency shift from state i to state j
26             del = abs(Freqs(i) - Freqs(j));
27             % If within the maximum state shift values allowed
28             if(del <= delta)
29                 % Set the G value at this state difference to  $e^{(-K|f_1-f_2|)}$ 
30                 G(i,j) = exp(-K1*del);
31             % If not within the maximum states shift values allowed
32             else

```

```

33         % Set G to 0
34         G(i,j) = 0;
35     end
36 end
37 end
38
39 %%% Initialize alpha and betas %%%
40 % Matrix to store the alpha values over time
41 A = zeros(num_states, v_points);
42 % Set all of the initial alphas to 1/Q
43 A(:,1) = 1/num_states;
44 % Matrix to store the beta values over time
45 B = zeros(num_states, v_points);
46 % Set all the initial betas to 1/Q
47 B(:,end) = 1/num_states;
48
49 %%% Run forward and backward passes at the same time %%%
50 for t = 1:v_points-1
51     %%% Forward pass %%%
52     % Column vector of data at time t
53     forward_data = input(:,t);
54     % Column vector of current alphas
55     alpha = A(:,t);
56     % QxQ matrix of gamma at time t, p indexed along row q along column
57     gamma = G.*exp(K2*forward_data).';
58     % Next alpha is a sum of all the paths from p to q
59     alpha_tilde = gamma*alpha;
60     % Normalize
61     A(:,t+1) = (1/sum(alpha_tilde))*alpha_tilde;
62     %%% Backward pass %%%
63     % Column vector of data at time N-t
64     backward_data = input(:,v_points-t);
65     % Column vector of current betas
66     beta = B(:,v_points-t+1);
67     % QxQ matrix of gamma at time N-t, p indexed along row q along column

```

```

68     gamma = G.*exp(K2*backward_data. ');
69     % Next beta is a sum of all the paths from p to q
70     beta_tilde = gamma*beta;
71     % Normalize
72     B(:,v_points-t) = (1/sum(beta_tilde))*beta_tilde;
73 end
74
75 %%% Now calculate the path probabilities %%%
76 % Probability of the state at this time
77 P = zeros(num_states,v_points);
78 for t = 1:v_points-1
79     % Column vector of data at time t
80     data = input(:,t);
81     % Column vector of alpha at time t
82     alpha = A(:,t);
83     % Column vector of beta at time t+1
84     beta = B(:,t+1);
85     % Matrix of gamma using current data
86     gamma = G.*exp(K2*data);
87     % Probabilities of each state at this time
88     P(:,t) = alpha.*(gamma*beta);
89 end
90 end

```

Listing C.14: Function to compute the BCJR algorithm with balance factor adjustment

```

1  %BCJR.BF creates a probabilty matrix from a spectrogram of data
2  %   This function takes in a spectrogram of data and calculates the
3  %   probabilty of the Doppler at each frequency overtime, taking into
4  %   account the balance factor.
5  %   Inputs:
6  %       input:      Frequency data over time
7  %       Freqs:      The frequency states at each time period
8  %       K1:         Constant to give weight to the branch metric
9  %       K2:         Constant to give weight to the state value
10 %       delta:      The maximum distance for a frequency change
11 %                   from time period to time period
12 %       BF:         A vector containing the balance factor for each time
13 %                   step
14 %       epsilon:    The threshold variable for determining whether or not
15 %                   to pay attention to the balance factor
16 %       K0:         Constant to give weight to the balance factor
17 %       sigma:      Width of the adjustment spectrum
18 %       K3:         Controls how fast the magnitude of the adjustment
19 %                   spectrum linearly drops away from the maximum at 0 Hz
20 %       Lambda:     The maximum height of the adjustment spectrum at 0 Hz
21 %   Output:
22 %       P:          Matrix of the probabilities of each state at each time
23 %                   step
24 function [P] = BCJR.BF(input ,Freqs ,K1,K2, delta ,BF, epsilon ,K0, sigma ,K3, Lambda)
25     % Total number of states
26     num_states = size(input,1);
27     % The number of time steps
28     v_points = size(input,2);
29
30     %%% Create the adjustment spectrum %%%
31     V0 = zeros(num_states,1);
32     % Cycle through all values you want to give weight to
33     for i = -sigma:sigma
34         V0(floor(num_states/2)+1+i) = Lambda-abs(i*K3);

```

```

35     end
36
37     %%% Pre-calculate part of the gammas, the branch metric portion %%%
38     % Create an empty array
39     G = zeros(num_states,num_states);
40     for i = 1:num_states
41         for j = 1:num_states
42             % This is the frequency shift from state i to state j
43             del = abs(Freqs(i) - Freqs(j));
44             % If within the maximum state shift values allowed
45             if(del <= delta)
46                 % Set the G value at this state difference to  $e^{(-K|f_1-f_2|)}$ 
47                 G(i,j) = exp(-K1*del);
48             % If not within the maximum states shift values allowed
49             else
50                 % Set G to 0
51                 G(i,j) = 0;
52             end
53         end
54     end
55
56     %%% Initialize alpha and betas %%%
57     % Matrix to store the alpha values over time
58     A = zeros(num_states,v_points);
59     % Set all of the initial alphas to 1/Q
60     A(:,1) = 1/num_states;
61     % Matrix to store the beta values over time
62     B = zeros(num_states,v_points);
63     % Set all the initial betas to 1/Q
64     B(:,end) = 1/num_states;
65
66     %%% Run forward and backward passes at the same time %%%
67     for t = 1:v_points-1
68         %%% Forward pass %%%
69         if(abs(BF(t)) > epsilon)

```

```

70         % Column vector of data at time t
71         forward_data = input(:,t);
72     else
73         % Column vector of data at time t taking into account the
74         % balance factor
75         BF_tilde = abs(BF(t))*K0;
76         forward_data = BF_tilde*input(:,t) + (1-BF_tilde)*V0;
77     end
78     % Column vector of current alphas
79     alpha = A(:,t);
80     % QxQ matrix of gamma at time t, p indexed along row q along column
81     gamma = G.*exp(K2*forward_data).';
82     % Next alpha is a sum of all the paths from p to q
83     alpha_tilde = gamma*alpha;
84     % Normalize
85     A(:,t+1) = (1/sum(alpha_tilde))*alpha_tilde;
86     %%% Backward pass %%%
87     if(abs(BF(t)) > epsilon)
88         % Column vector of data at time N-t
89         backward_data = input(:,v_points-t);
90     else
91         % Column vector of data at time N-t taking into account the
92         % balance factor
93         BF_tilde = abs(BF(t))*K0;
94         backward_data = BF_tilde*input(:,v_points-t) + (1-BF_tilde)*V0;
95     end
96     % Column vector of current betas
97     beta = B(:,v_points-t+1);
98     % QxQ matrix of gamma at time N-t, p indexed along row q along column
99     gamma = G.*exp(K2*backward_data. ');
100    % Next beta is a sum of all the paths from p to q
101    beta_tilde = gamma*beta;
102    % Normalize
103    B(:,v_points-t) = (1/sum(beta_tilde))*beta_tilde;
104 end

```

```

105
106     %%% Now calculate the path probabilities %%%
107     % Probability of the state at this time
108     P = zeros(num_states, v_points);
109     for t = 1:v_points-1
110         if(abs(BF(t)) > epsilon)
111             % Column vector of data at time t
112             data = input(:, t);
113         else
114             % Column vector of data at time t taking into account the
115             % balance factor
116             BF_tilde = abs(BF(t))*K0;
117             data = BF_tilde*input(:, t) + (1-BF_tilde)*V0;
118         end
119         % Column vector of alpha at time t
120         alpha = A(:, t);
121         % Column vector of beta at time t+1
122         beta = B(:, t+1);
123         % Matrix of gamma using current data
124         gamma = G.*exp(K2*data);
125         % Probabilities of each state at this time
126         P(:, t) = alpha.*(gamma*beta);
127     end
128 end

```


Listing C.15: Function to compute the balance factors

```

1  %BALANCEFACTOR Computes a spectrum balance factor
2  %   This takes in a frequency spectrum and finds the energy balance between
3  %   the positive and negative frequencies.
4  function [BF] = balance_factor(freq_spectrum, freqs)
5      % Find where the the DC term is
6      zero_pos = find(freqs==0);
7
8      % Find the sum of each side and the total spectrum
9      L_sum = sum(freq_spectrum(1:zero_pos, :), 1);
10     R_sum = sum(freq_spectrum(zero_pos:end, :), 1);
11     T_sum = sum(freq_spectrum, 1);
12
13     % Figure out how much energy is assigned to each side
14     fl = L_sum./T_sum;
15     fr = R_sum./T_sum;
16
17     % Subtract the energy balances to get a balance factor
18     % BF ranges from -1 to 1, -1 meaning all negative, 1 meaning all
19     % positive, -1 meaning all negative, 0 meaning balanced
20     BF = fr - fl;
21 end

```